BAN CƠ YẾU CHÍNH PHỦ

HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG THÔNG TIN HỖ TRỢ QUẢN LÝ VĂN BẰNG CHỨNG CHỈ DỰA TRÊN NỀN TẢNG BLOCKCHAIN

Ngành: **An toàn thông tin**

Mã số: **7.48.02.02**

Sinh viên thực hiện:

Phạm Đức Khải – MSSV: AT190226

Hà Nội, 2026

BAN CƠ YẾU CHÍNH PHỦ

HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG THÔNG TIN HỖ TRỢ QUẢN LÝ VĂN BẰNG CHỨNG CHỈ DỰA TRÊN NỀN TẢNG BLOCKCHAIN

Ngành: **An toàn thông tin**

Mã số: **7.48.02.02**

Sinh viên thực hiện:

Phạm Đức Khải – MSSV: AT190226

Người hướng dẫn:

TS. Trần Trung

Khoa CNTT – Trường Đại học Điện lực

Hà Nội, 2026

MỤC LỤC

LỜI CẢM ƠN.....

LỜI NÓI ĐẦU.....

CHƯƠNG I. CƠ SỞ LÝ THUYẾT VỀ CÔNG NGHỆ BLOCKCHAIN.....

1.1. Tổng quan về công nghệ Blockchain.....1

1.1.1. Khái niệm và mục tiêu hình thành.....

1.1.2. Quá trình phát triển.....

1.1.3. Đặc trưng, giới hạn và những cách hiểu cần tránh.....

1.1.4. Phân biệt với DLT và cơ sở dữ liệu phân tán.....

1.2. Cấu trúc dữ liệu và nền tảng mật mã.....3

1.2.1. Giao dịch, khối và liên kết băm.....

1.2.2. Hàm băm mật mã và cây Merkle.....

1.2.3. Mật mã khóa công khai và chữ ký số.....

1.2.4. Định danh, dấu thời gian và nonce.....

1.3. Kiến trúc và nguyên lý hoạt động.....7

1.3.1. Mạng ngang hàng và vai trò của nút.....

1.3.2. Vòng đời giao dịch.....

1.3.3. Sao chép trạng thái và thực thi xác định.....

1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống.....

1.4. Cơ chế đồng thuận.....9

1.4.1. Bài toán đồng thuận và mô hình lỗi.....

1.4.2. Proof of Work và Proof of Stake.....

1.4.3. Đồng thuận CFT và BFT trong mạng định danh.....

1.4.4. Tính hoàn tất và đánh đổi thiết kế.....

1.5. Phân loại Blockchain và điều kiện lựa chọn.....11

1.5.1. Các trục phân loại.....

1.5.2. So sánh các mô hình triển khai.....

1.5.3. Khi nên và không nên sử dụng Blockchain.....

1.6. An toàn, quyền riêng tư và khả năng mở rộng.....13

1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp.....

1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài.....

1.6.3. Quyền riêng tư và dữ liệu cá nhân.....

1.6.4. Khả năng mở rộng và khả năng tương tác.....
1.7. Định hướng ứng dụng trong quản lý văn bằng, chứng chỉ.....15
1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng.....
1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi.....
1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm.....
1.7.4. Mô hình Blockcerts V3 và quan hệ với Verifiable Credentials.....
1.7.5. Bitcoin OP_RETURN và môi trường regtest.....
1.7.6. Bối cảnh pháp lý và ranh giới áp dụng tại Việt Nam.....
1.7.7. Đối chiếu các phương án và phạm vi đóng góp.....
1.8. Tổng kết Chương I.....19
1.8.1. Kết quả lý luận chính.....
1.8.2. Cơ sở chuyển sang Chương II.....
CHƯƠNG II. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....
2.1. Phân tích bài toán.....20
2.1.1. Bối cảnh nghiệp vụ.....
2.1.2. Phạm vi.....
2.2. Tác nhân và yêu cầu chức năng.....20
2.2.1. Tác nhân.....
2.2.2. Yêu cầu chức năng.....
2.2.3. Quy tắc nghiệp vụ.....
2.3. Yêu cầu phi chức năng và mô hình đe dọa.....22
2.3.1. Yêu cầu phi chức năng.....
2.3.2. Tài sản và đối thủ.....
2.4. Lựa chọn công nghệ và kiến trúc.....24
2.4.1. Lựa chọn mô hình Blockcerts–Bitcoin.....
2.4.2. Kiến trúc thành phần.....
2.4.3. Luồng dữ liệu.....
2.5. Thiết kế Maker–Checker và máy trạng thái.....26
2.5.1. Luồng phát hành.....
2.5.2. Trạng thái.....
2.6. Thiết kế phát hành Blockcerts V3.....28

2.6.1. Cấu trúc chứng thư.....
2.6.2. Gom lô và ánh xạ kết quả.....
2.6.3. Tách nhóm yêu cầu và lô phát hành.....
2.7. Thiết kế xác minh và thu hồi.....29
2.7.1. Xác minh.....
2.7.2. Thu hồi.....
2.8. Thiết kế dữ liệu ngoài chuỗi.....30
2.8.1. Bảng users.....
2.8.2. Bảng issued_certificates.....
2.8.3. verification_logs và audit_logs.....
2.8.4. Bảng issuance_batches.....
2.8.5. Bảng issuance_outbox.....
2.9. Thiết kế API và phân quyền.....33
2.10. Thiết kế ba cổng giao diện.....35
2.11. Thiết kế an toàn và triển khai.....35
2.11.1. Ranh giới tin cậy và dữ liệu nhạy cảm.....
2.11.2. Bảo vệ trạng thái, outbox và phục hồi.....
2.11.3. Audit và giới hạn tính toàn vẹn.....
2.12. Tiêu chí chấp nhận và kế hoạch kiểm thử.....36
2.13. Kết luận Chương 2.....37
CHƯƠNG III. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG.....
3.1. Môi trường và mục tiêu tái lập.....38
3.1.1. Môi trường thực nghiệm và mốc mã nguồn.....
3.1.2. Kiến trúc triển khai.....
3.2. Tái lập hệ thống từ máy Ubuntu mới.....39
3.2.1. Chuẩn bị công cụ và kiểm tra tài nguyên.....
3.2.2. Lấy mã nguồn và cố định phiên bản.....
3.2.3. Tạo cấu hình runtime và bảo vệ bí mật.....
3.2.4. Dựng PostgreSQL, Redis và Bitcoin regtest.....
3.2.5. Áp dụng migration, cài phụ thuộc và build.....
3.2.6. Khởi tạo Blockcerts regtest và preflight.....
3.2.7. Triển khai backend và ba cổng web.....
3.2.8. Khởi tạo tài khoản thử nghiệm và kiểm tra quyền.....

3.2.9. Chạy kiểm thử chức năng, an toàn và phục hồi.....
3.2.10. Chạy đo hiệu năng và cố định bộ bằng chứng.....
3.2.11. Dừng dịch vụ và làm sạch môi trường tái lập.....
3.3. Hiện thực backend, dữ liệu và hàng đợi.....50
3.3.1. Các mô-đun nghiệp vụ.....
3.3.2. Mô hình dữ liệu và liên kết người học.....
3.3.3. Outbox giao dịch và phục hồi phát hành.....
3.3.4. Audit và dữ liệu xác minh.....
3.4. Hiện thực phát hành, xác minh và thu hồi Blockcerts.....51
3.4.1. Phát hành theo lô và gom lô Merkle.....
3.4.2. Xác minh công khai.....
3.4.3. Thu hồi có điểm kiểm soát và đối soát trạng thái.....
3.5. Tái hiện quy trình nghiệp vụ trên web từ đầu đến cuối.....52
3.5.1. Khởi tạo tài khoản Checker, Maker và Student.....
3.5.2. Maker đăng nhập, tra cứu Student và lập phiếu.....
3.5.3. Checker kiểm tra, phê duyệt và Worker phát hành.....
3.5.4. Student xem và tải chứng thư.....
3.5.5. Xác minh chứng thư hợp lệ.....
3.5.6. Sửa nội dung chứng thư và kiểm tra lại.....
3.5.7. Checker thu hồi và xác minh lại tệp gốc.....
3.6. Kiểm soát an toàn và khả năng phục hồi.....60
3.7. Phương pháp kiểm thử và chuỗi bằng chứng.....61
3.7.1. Phân lớp kiểm thử.....
3.7.2. Cố định bộ bằng chứng.....
3.8. Kết quả Kịch bản 1 – tính toàn vẹn và Maker–Checker.....62
3.9. Kết quả Kịch bản 2 – hiệu năng phát hành theo lô Merkle.....63
3.9.1. Kết quả chín lần chạy.....
3.9.2. Giá trị tổng hợp.....
3.9.3. Tài nguyên.....
3.9.4. Kiểm toán tệp kết quả.....
3.10. Kết quả Kịch bản 3 – thu hồi, RBAC và audit.....67
3.11. Đánh giá kết quả, hạn chế và hướng phát triển.....68
3.11.1. Mức đáp ứng.....

3.11.2. Hạn chế.....
3.11.3. Hướng phát triển.....
3.12. Kết luận Chương 3.....70
KẾT LUẬN VÀ KIẾN NGHỊ.....
1. Kết quả đạt được và mức hoàn thành mục tiêu.....71
2. Đóng góp của đồ án.....71
3. Hạn chế và hướng phát triển.....72
TÀI LIỆU THAM KHẢO.....
PHỤ LỤC.....
Phụ lục 1. Danh mục API chính và quyền truy cập.....76
Phụ lục 2. Khung cấu hình tái lập và bảo vệ bí mật.....77
Phụ lục 3. Quy trình tái lập và lệnh đại diện.....78
Phụ lục 4. Bộ bằng chứng thực nghiệm chính thức.....79
Phụ lục 5. Ma trận truy vết yêu cầu – hiện thực – bằng chứng.....80

DANH MỤC KÝ HIỆU VÀ TỪ VIẾT TẮT

Từ viết tắt	Từ đầy đủ
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BFT	Byzantine Fault Tolerance
CFT	Crash Fault Tolerance
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DB	Database
DBA	Database Administrator
DLT	Distributed Ledger Technology
DNS	Domain Name System
DTO	Data Transfer Object
E2E	End-to-End
HSM	Hardware Security Module
HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON-LD	JavaScript Object Notation for Linked Data
JWT	JSON Web Token
KMS	Key Management Service/System
MFA	Multi-Factor Authentication
P2P	Peer-to-Peer
PBFT	Practical Byzantine Fault Tolerance
PKI	Public Key Infrastructure
PoS	Proof of Stake
PoW	Proof of Work
RAM	Random Access Memory
RBAC	Role-Based Access Control
RPC	Remote Procedure Call
RSS	Resident Set Size
SHA	Secure Hash Algorithm
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UTXO	Unspent Transaction Output
UUID	Universally Unique Identifier
VBCC	Văn bằng, chứng chỉ
VC	Verifiable Credential
W3C	World Wide Web Consortium
WIF	Wallet Import Format

DANH MỤC HÌNH VẼ

Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp.....

Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch.....

Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát.....

Hình 1.4. So sánh khái quát các nhóm cơ chế đồng thuận.....

Hình 1.5. Phân loại Blockchain theo phạm vi tham gia và quyền truy cập.....

Hình 1.6. Các lớp bảo vệ trong một hệ thống Blockchain.....

Hình 1.7. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bằng.....

Hình 2.1. Tác nhân và nhóm chức năng.....

Hình 2.2. Kiến trúc thành phần.....

Hình 2.3. Trình tự phát hành theo lô.....

Hình 2.4. Máy trạng thái yêu cầu phát hành.....

Hình 2.5. Trình tự xác minh công khai.....

Hình 2.6. Mô hình dữ liệu.....

Hình 3.1. Kiến trúc triển khai của hệ thống trong môi trường thực nghiệm.....

Hình 3.2. Chuỗi tái lập từ Ubuntu mới đến bộ bằng chứng thực nghiệm.....

Hình 3.3. Màn hình đăng nhập cổng quản trị.....

Hình 3.4. Maker đối chiếu hồ sơ Student và nhập dữ liệu văn bằng.....

Hình 3.5. Phiếu được tạo và chờ Checker phê duyệt.....

Hình 3.6. Checker kiểm tra phiếu đang chờ phê duyệt.....

Hình 3.7. Danh sách văn bằng sau khi Worker phát hành.....

Hình 3.8. Student xem và tải chứng thư thuộc đúng tài khoản.....

Hình 3.9. Cổng Verify xác nhận chứng thư hợp lệ.....

Hình 3.10. Verifier phát hiện chứng thư đã bị thay đổi.....

Hình 3.11. Checker thu hồi chứng thư và hệ thống cập nhật trạng thái.....

Hình 3.12. Cổng Verify trả trạng thái REVOKED cho chứng thư gốc.....

Hình 3.13. Thời gian phát hành trung bình theo kích thước lô.....

Hình 3.14. Thông lượng và thời gian trung bình trên mỗi chứng thư.....
Hình 3.15. Mức sử dụng tài nguyên theo kích thước lô.....

DANH MỤC BẢNG

Bảng 1.1. So sánh khái quát các mô hình triển khai Blockchain.....
Bảng 1.2. Đối chiếu các phương án quản lý và kiểm chứng văn bằng.....
Bảng 2.1. Danh sách yêu cầu chức năng.....
Bảng 2.2. Đe dọa và kiểm soát.....
Bảng 2.3. Ý nghĩa trạng thái.....
Bảng 2.4. Mô hình dữ liệu chính.....
Bảng 2.5. API chính và quyền.....
Bảng 2.6. Tiêu chí chấp nhận.....
Bảng 3.1. Thành phần phần mềm của lượt tái kiểm chứng.....
Bảng 3.2. Nhóm cấu hình runtime và nguyên tắc bảo vệ.....
Bảng 3.3. Điều kiện hoàn thành giai đoạn triển khai.....
Bảng 3.4. Các thực thể dữ liệu trực tiếp tham gia luồng phát hành.....
Bảng 3.5. Kiểm soát an toàn và bằng chứng kiểm thử.....
Bảng 3.6. Kết quả kiểm thử tự động và E2E.....
Bảng 3.7. Chỉ số kiểm soát cuối của bộ bằng chứng.....
Bảng 3.8. Kết quả Kịch bản 1.....
Bảng 3.9. Chín lần chạy hiệu năng tái kiểm chứng.....
Bảng 3.10. Tổng hợp theo kích thước lô.....
Bảng 3.11. Tài nguyên hệ thống theo kích thước lô.....
Bảng 3.12. Kết quả kiểm toán tệp kết quả.....
Bảng 3.13. Kết quả Kịch bản 3.....
Bảng 3.14. Một số kiểm tra E2E bổ sung của phiên bản hiện tại.....
Bảng 3.15. Đối chiếu yêu cầu chính và bằng chứng.....
Bảng PL1.1. API chính của nguyên mẫu.....
Bảng PL2.1. Nhóm cấu hình và mức nhạy cảm.....
Bảng PL4.1. Thành phần bộ bằng chứng và mục đích.....
Bảng PL5.1. Ma trận truy vết yêu cầu và bằng chứng.....

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp, em đã nhận được sự hướng dẫn tận tình của TS. Trần Trung – Giảng viên Khoa Công nghệ thông tin, Trường Đại học Điện lực. Em trân trọng cảm ơn ThS. Nguyễn Đức Ngân – Giảng viên Khoa An toàn thông tin, Học viện Kỹ thuật Mật mã – cùng Hệ Quản lý sinh viên đã hỗ trợ em trong quá trình học tập và hoàn thiện đồ án. Em cũng chân thành cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện để em hoàn thành công việc này.

SINH VIÊN THỰC HIỆN ĐỒ ÁN

Phạm Đức Khải

LỜI NÓI ĐẦU

Chuyển đổi số trong giáo dục làm cho hồ sơ học tập, văn bằng và chứng chỉ ngày càng được tạo, trao đổi và xác minh trên môi trường mạng. Cơ sở dữ liệu tập trung vẫn là thành phần cần thiết để quản lý nghiệp vụ, nhưng việc xác minh bởi bên thứ ba thường phụ thuộc vào cổng tra cứu của đơn vị cấp hoặc quy trình đối soát thủ công. Khi dữ liệu bị sửa trái phép, bản sao bị làm giả hoặc dịch vụ tra cứu gián đoạn, việc đánh giá nguồn gốc và trạng thái hiệu lực của tài liệu trở nên khó khăn. Vì vậy, bài toán quản lý văn bằng không chỉ là số hóa tài liệu mà còn phải bảo đảm tính toàn vẹn, khả năng truy vết, phân tách thẩm quyền và cơ chế xác minh độc lập.

Blockchain cung cấp một cơ chế ghi nhận bằng chứng khó sửa đổi hồi tố khi được kết hợp đúng với hàm băm, chữ ký số, cây Merkle và quy tắc đồng thuận [1], [2]. Trong hệ sinh thái Blockcerts, dữ liệu đầy đủ của chứng thư được giữ ngoài chuỗi, còn giá trị đại diện của một lô có thể được neo lên blockchain để hỗ trợ kiểm tra độc lập [11], [14]. Cách tiếp cận này không thay thế cơ sở dữ liệu nghiệp vụ và cũng không bảo đảm dữ liệu đầu vào là đúng; đơn vị phát hành vẫn chịu trách nhiệm về danh tính, quyền phê duyệt, quản lý khóa và quyết định thu hồi.

Tại Việt Nam, Thông tư 10/2026/TT-BGDĐT đã bổ sung khuôn khổ đối với văn bằng, chứng chỉ số và hoạt động quản lý, khai thác, xác minh thông tin văn bằng, chứng chỉ [25]. Đồng thời, Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 và Nghị định 356/2025/NĐ-CP đặt ra yêu cầu đối với việc xử lý dữ liệu cá nhân [26], [27]. Vì vậy, nguyên mẫu trong đồ án được định vị là một lớp kỹ thuật hỗ trợ phát hành và xác minh chứng thư Blockcerts, không tự tạo ra giá trị pháp lý cho văn bằng số. Dữ liệu cá nhân chi tiết được giữ ngoài blockchain; môi trường thực nghiệm sử dụng Bitcoin regtest để kiểm chứng logic mà không phát sinh giao dịch tài chính thật [18].

Từ bối cảnh trên, đồ án "Nghiên cứu phát triển hệ thống thông tin hỗ trợ quản lý văn bằng chứng chỉ dựa trên nền tảng Blockchain" đặt mục tiêu phân tích, thiết kế và xây dựng một nguyên mẫu có quy trình Maker–Checker, phân quyền theo vai trò và đối tượng, phát hành Blockcerts V3 theo lô, cổng Holder cho sinh viên, cổng Verify công khai, cơ chế thu hồi và nhật ký kiểm toán. Báo cáo gồm ba chương: Chương I trình bày cơ sở lý thuyết và bối cảnh áp dụng; Chương II phân tích, thiết kế hệ thống và các kiểm soát an toàn; Chương III tái lập hệ thống từ môi trường mới, trình bày luồng Web đầu cuối, kiểm thử chức năng, phục hồi, hiệu năng và bộ bằng chứng thực nghiệm.

CHƯƠNG I. CƠ SỞ LÝ THUYẾT VỀ CÔNG NGHỆ BLOCKCHAIN

1.1. Tổng quan về công nghệ Blockchain

1.1.1. Khái niệm và mục tiêu hình thành

Công nghệ chuỗi khối (Blockchain) là một dạng kiến trúc sổ cái phân tán (Distributed Ledger Technology - DLT), trong đó dữ liệu giao dịch được đóng gói thành các khối (blocks), liên kết tuần tự theo thời gian bằng các hàm băm mật mã và được đồng bộ giữa các nút mạng thông qua giao thức đồng thuận [1], [2]. Thay vì phụ thuộc vào một máy chủ trung tâm, sổ cái Blockchain được sao chép và duy trì độc lập tại nhiều nút tham gia. Khi dữ liệu quá khứ bị thay đổi, các liên kết băm phía sau không còn khớp; sự sai lệch vì thế có thể được phát hiện khi kiểm tra lại lịch sử. Kết hợp với sao chép sổ cái và đồng thuận, cơ chế này làm cho việc sửa đổi hồi tố mà không để lại dấu vết trở nên khó khăn trong các giả định an toàn của hệ thống [1], [10].

Mục tiêu cốt lõi của Blockchain là giúp các thực thể thuộc nhiều miền quản trị thống nhất một lịch sử trạng thái dùng chung và giảm mức phụ thuộc vào một bên trung gian duy nhất [1]. Trong các quy trình nghiệp vụ truyền thống, việc đối soát liên tổ chức qua trung gian thường làm phát sinh chi phí, độ trễ và nguy cơ xuất hiện điểm lỗi đơn lẻ (Single Point of Failure). Blockchain giải quyết bài toán này nhờ kết hợp mật mã khóa công khai, cây Merkle, mạng ngang hàng và thuật toán đồng thuận để tạo ra nhật ký kiểm toán minh bạch, cho phép các bên tự kiểm chứng tính toàn vẹn dữ liệu [2], [10].

Tuy nhiên, Blockchain không nhằm thay thế các hệ thống lưu trữ dung lượng lớn hay cơ sở dữ liệu quan hệ truyền thống. Mọi hệ thống Blockchain trong thực tế đều đòi hỏi một khung quản trị xác định, bao gồm quy định về quyền truy cập, chính sách quản lý khóa mật mã, quy tắc phê duyệt giao dịch và cơ chế nâng cấp giao thức [1]. Công nghệ này phù hợp hơn với bài toán cộng tác đa bên, trong đó các thực thể cần chia sẻ trạng thái có thể kiểm chứng nhưng không muốn một bên duy nhất toàn quyền quyết định lịch sử dữ liệu [10].

1.1.2. Quá trình phát triển

Nền tảng lý thuyết của Blockchain là kết quả kế thừa từ nhiều thập kỷ nghiên cứu về mật mã học, cấu trúc dữ liệu và hệ thống phân tán. Từ cuối những năm 1970 và đầu thập niên 1980, các công trình về hệ mật mã khóa công khai, cấu trúc cây băm Merkle [3] và các giải pháp đóng dấu thời gian số (digital timestamping) đã đặt nền móng cho việc xác thực tính toàn vẹn dữ liệu. Song song với đó, lý thuyết hệ thống phân tán phát triển các mô hình đồng thuận chịu lỗi

dừng (CFT) và chịu lỗi tùy ý Byzantine (BFT), cung cấp cơ sở toán học để duy trì tính nhất quán trạng thái giữa các nút độc lập khi xảy ra sự cố mạng hoặc có nút sai hỏng [6], [7].

Năm 2008, Satoshi Nakamoto kết hợp chuỗi liên kết băm, mạng ngang hàng và cơ chế Bằng chứng công việc (Proof of Work) để giới thiệu Bitcoin, qua đó đề xuất một cơ chế hạn chế chi tiêu kép mà không cần máy chủ thanh toán trung tâm [1], [2]. Các nền tảng về sau mở rộng khả năng lập trình trạng thái thông qua hợp đồng thông minh (smart contract), cho phép biểu diễn và tự động thực thi một số quy tắc nghiệp vụ [1], [10]. Đồng thời, các kiến trúc Blockchain cấp quyền (permissioned) ra đời, ứng dụng các thuật toán CFT và BFT để tối ưu hóa thông lượng và đáp ứng yêu cầu bảo mật, quyền riêng tư nghiêm ngặt trong môi trường liên tổ chức [1], [8].

1.1.3. Đặc trưng, giới hạn và những cách hiểu cần tránh

Hệ thống Blockchain sở hữu các đặc trưng kỹ thuật cốt lõi gồm phân tán dữ liệu, khả năng kiểm chứng độc lập, tính truy vết giao dịch và cơ chế chống can thiệp dữ liệu hồi tố [1]. Mọi giao dịch hợp lệ đều gắn liền với định danh mật mã của bên khởi tạo và được neo chặt vào cấu trúc chuỗi khối, giúp người tham gia đối soát lịch sử một cách minh bạch mà không phụ thuộc một chủ thể duy nhất [2], [10]. Mặc dù vậy, việc nghiên cứu và ứng dụng đòi hỏi phải nhận diện rõ các giới hạn kỹ thuật và tránh ba ngộ nhận phổ biến:

Thứ nhất, dữ liệu ghi trên chuỗi không đồng nghĩa với chân lý thực tế; hệ thống chỉ bảo đảm tính toàn vẹn của dữ liệu sau khi đã được chấp nhận vào sổ cái. Nếu dữ liệu đầu vào bị sai lệch hoặc cố tình gian lận từ khâu thu thập ngoại chuỗi (off-chain), hệ thống vẫn lưu trữ và bảo vệ dữ liệu sai đó một cách bền vững theo nguyên lý "rác vào, rác ra" (Garbage-In, Garbage-Out) [1]. Thứ hai, không nên tuyệt đối hóa tính chất dữ liệu thành "bất biến vĩnh cửu". Về bản chất, dữ liệu trên chuỗi chỉ mang tính kháng sửa đổi dựa trên các giả định an toàn mật mã và ngưỡng tài nguyên đồng thuận; nếu kẻ xấu kiểm soát vượt ngưỡng năng lực tính toán hoặc vượt số nút lỗi cho phép của thuật toán đồng thuận, lịch sử giao dịch vẫn có nguy cơ bị phân nhánh hoặc đảo ngược [1], [6].

Thứ ba, Blockchain không đồng nghĩa với tính ẩn danh hoàn toàn hay việc loại bỏ trách nhiệm pháp lý. Đa phần các mạng lưới chỉ cung cấp tính năng bí danh (pseudonymity) dựa trên địa chỉ khóa công khai; thông qua phân tích đồ thị luồng giao dịch và đối chiếu siêu dữ liệu ngoại vi, danh tính thực của thực thể vẫn có thể bị truy vết và định danh [1]. Ngoài ra, việc nhiều nút cùng kiểm tra và lưu bản sao

thường làm tăng độ trễ, chi phí lưu trữ và chi phí phối hợp so với cơ sở dữ liệu tập trung được tối ưu cho cùng một bài toán [10].

1.1.4. Phân biệt với DLT và cơ sở dữ liệu phân tán

Công nghệ sổ cái phân tán (DLT) là khái niệm bao trùm, chỉ chung mọi kiến trúc hệ thống lưu trữ và đồng bộ hóa sổ cái ghi chép trạng thái trên nhiều thực thể mạng mà không phụ thuộc vào một máy chủ trung tâm [1]. DLT không bắt buộc dữ liệu phải tổ chức thành khối liên kết chuỗi tuyến tính, mà có thể sử dụng cấu trúc đồ thị có hướng không chu trình (DAG) hoặc các dạng cây phi tuyến khác. Do đó, Blockchain là một tập con đặc thù của DLT, nổi bật với cấu trúc phân khối tuần tự và các liên kết mật mã chặt chẽ giữa các khối [1], [10].

Cơ sở dữ liệu phân tán truyền thống (chẳng hạn các hệ quản trị RDBMS hoặc NoSQL phân tán) cũng thực hiện việc phân mảnh và nhân bản dữ liệu qua nhiều máy chủ nhằm tối ưu thông lượng, độ sẵn sàng (Availability) và tính chịu lỗi phần cứng (Fault Tolerance). Tuy nhiên, các hệ thống này vận hành trong cùng một miền quản trị tin cậy (Single Trust Domain) dưới sự kiểm soát của một tổ chức [10]. Quản trị viên hệ thống (DBA) có toàn quyền can thiệp, xóa bản ghi hoặc chỉnh sửa lược đồ dữ liệu trực tiếp mà không cần sự đồng thuận ngang hàng của các bên độc lập.

Blockchain hướng tới môi trường có nhiều chủ thể cùng duy trì lịch sử theo một bộ quy tắc đã thống nhất [1], [8]. Trong thiết kế liên tổ chức, quyền cập nhật và quản trị nên được phân bổ sao cho một tài khoản đơn lẻ không thể âm thầm sửa lịch sử đã xác nhận; tuy nhiên, mức phân quyền thực tế vẫn phụ thuộc số tổ chức vận hành, chính sách nâng cấp và khả năng thông đồng. Khác biệt cốt lõi so với cơ sở dữ liệu phân tán truyền thống vì thế nằm ở ranh giới tin cậy và quy trình chấp nhận thay đổi, không chỉ ở số lượng máy lưu bản sao [10].

1.2. Cấu trúc dữ liệu và nền tảng mật mã

1.2.1. Giao dịch, khối và liên kết băm

Giao dịch (Transaction) là đơn vị nguyên tử biểu diễn một yêu cầu thay đổi trạng thái trong hệ thống sổ cái phân tán [1], [10]. Tùy nền tảng, giao dịch có thể chứa định danh hoặc địa chỉ bên gửi, dữ liệu đầu vào, số thứ tự chống phát lại, tham số phí và chữ ký số. Khi được phát tán vào mạng lưới, các nút tiếp nhận tiến hành kiểm tra độc lập tính hợp lệ về mặt cú pháp, tính toàn vẹn chữ ký số và điều kiện logic trạng thái hiện hành trước khi chấp nhận đưa vào hàng đợi xử lý [1], [2].

Các giao dịch đã được sắp thứ tự có thể được đóng gói vào Khối (Block), gồm phần đầu chứa siêu dữ liệu giao thức và phần thân chứa giao dịch [1], [2].

Tùy thiết kế, phần đầu khối có thể bao gồm giá trị băm của khối trước $(H_i - 1)$, gốc Merkle ($MerkleRoo\,t_i$), dấu thời gian $(Timestam\,p_i)$và dữ liệu phục vụ đồng thuận ($Metadat\,a_i$). Một biểu diễn khái quát của liên kết băm là: Mối liên kết mật mã giữa các khối liên tiếp được thiết lập bằng cách tính toán giá trị băm H_i của toàn bộ phần đầu khối thông qua hàm băm mật mã H:

$$H_i = H(H_{i-1} \parallel MerkleRoot_i \parallel Timestamp_i \parallel Metadata_i) \quad (1.1)$$

Trong biểu thức (1.1), toán tử $\parallel$ biểu thị phép nối chuỗi. Đây là mô hình minh họa; tập trường và quy tắc mã hóa byte cụ thể do từng giao thức quy định. Giá trị $H_i - 1$ tạo liên kết kiểm chứng giữa hai khối liên tiếp. Cấu trúc liên kết này được minh họa trên Hình 1.1.



Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp

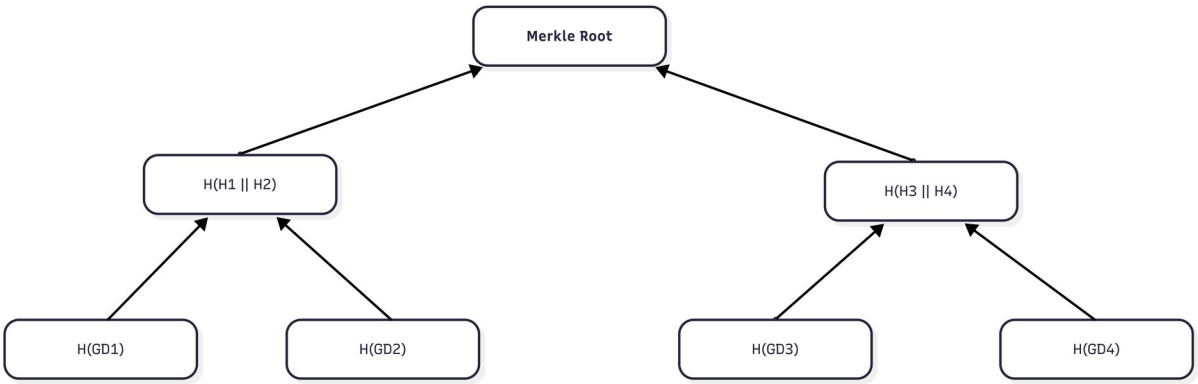
Nếu dữ liệu trong khối quá khứ *k* bị sửa, giá trị băm giao dịch, gốc Merkle và H_k thay đổi do tính nhạy của hàm băm [4]. Tham chiếu tại khối kế tiếp vì thế không còn khớp và sự can thiệp có thể được phát hiện. Để lịch sử đã sửa được các nút chấp nhận, đối phương còn phải đáp ứng quy tắc đồng thuận và chính sách quản trị của mạng; điều kiện cụ thể phụ thuộc loại Blockchain và mô hình đe dọa, không chỉ phụ thuộc liên kết băm [1], [2].

1.2.2. Hàm băm mật mã và cây Merkle

Hàm băm mật mã là công cụ toán học nền tảng đảm bảo tính toàn vẹn dữ liệu trong kiến trúc chuỗi khối [1], [4]. Một hàm băm mật mã H ánh xạ một thông điệp đầu vào M có độ dài tùy ý thành một chuỗi nhị phân cố định $h = H(M)$, điển hình là 256-bit đối với chuẩn SHA-256 (FIPS 180-4) [4]. Về mặt an toàn thông tin, hàm băm mật mã phải thỏa mãn ba thuộc tính cốt lõi: tính kháng tiền ảnh (Pre-image Resistance: cho trước y, khó về mặt tính toán để tìm x sao cho $H(x) = y$), tính kháng tiền ảnh thứ hai (Second Pre-image Resistance: cho trước x_1, khó tìm $x_2 \neq x_1$ sao cho $H(x_2) = H(x_1)$), và tính kháng va chạm mạnh (Collision Resistance: khó tìm

bất kỳ cặp (x_1, x_2) phân biệt nào sao cho $H(x_1) = H(x_2)$). Đồng thời, hiệu ứng tuyết lở (avalanche effect) đảm bảo mọi biến đổi dù nhỏ nhất ở đầu vào cũng làm thay đổi ngẫu nhiên và toàn diện giá trị băm đầu ra [4].

Để quản lý và xác thực danh sách giao dịch trong khối với chi phí tối ưu, giao thức sử dụng cấu trúc Cây Merkle (Merkle Tree) [3]. Cây Merkle thường được biểu diễn dưới dạng cây băm nhị phân, trong đó mỗi nút lá lưu giá trị băm của một giao dịch $(H_j = H(T x_j))$, còn mỗi nút trung gian lưu giá trị băm của phép nối hai nút con trực tiếp . Tiến trình băm cặp này thực hiện đệ quy từ các nút lá lên đỉnh để tạo ra một giá trị đại diện duy nhất là Gốc Merkle (*Merkle Root*) được ghi vào phần đầu khối [3]. Cây nhị phân là mô hình minh họa phổ biến; cấu trúc và cách xử lý số nút lẻ có thể khác giữa các nền tảng. Cấu trúc cây và đường dẫn kiểm chứng được minh họa trên Hình 1.2.



Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch

Cấu trúc cây Merkle cho phép kiểm tra một giao dịch thuộc tập giao dịch đã được cam kết thông qua bằng chứng Merkle. Với cây cân bằng, kích thước bằng chứng và số phép băm khi xác minh tăng theo logarit của số phần tử [3], [10]. Nút kiểm tra kết hợp giá trị băm của giao dịch với các giá trị trên đường dẫn để tính lại Merkle root và so sánh với root đã cam kết. Kết quả này chứng minh tính thuộc tập và tính toàn vẹn đối với cam kết, không tự chứng minh chữ ký, điều kiện chi tiêu hoặc mọi quy tắc hợp lệ của giao dịch; các điều kiện đó cần được kiểm tra riêng theo giao thức [2].

1.2.3. Mật mã khóa công khai và chữ ký số

Mật mã khóa công khai và chữ ký số thiết lập cơ chế xác thực nguồn gốc và kiểm soát ủy quyền đối với các giao dịch thay đổi trạng thái sổ cái [1], [5]. Theo chuẩn NIST FIPS 186-5 [5], một lược đồ chữ ký số an toàn được định nghĩa hình thức qua bộ ba thuật toán *(KeyGen, Sign, Verify)*: thuật toán sinh khóa $KeyGen(1^\lambda) \to (sk, pk)$ nhận tham số an ninh λ để tạo cặp khóa bất đối xứng gồm khóa

bí mật *sk* (dùng để ký) và khóa công khai *pk* (dùng để xác minh); thuật toán ký $Sign(sk, m) \rightarrow \sigma$ tạo chữ ký số σ từ thông điệp *m* và khóa *sk*; thuật toán xác minh $Verify(pk, m, \sigma) \rightarrow Valid, Invalid$ kiểm tra tính hợp lệ của σ dựa trên *pk* và *m*. Mô hình tổng quát được biểu diễn theo công thức (1.2):

$$\sigma \leftarrow Sign(sk, m); Verify(pk, m, \sigma) \in Hợp\ lệ, Không\ hợp\ lệ \quad (1.2)$$

Không nên diễn giải chữ ký số nói chung như việc "mã hóa giá trị băm bằng khóa bí mật rồi giải mã bằng khóa công khai". Cách mô tả này không phản ánh mô hình của nhiều lược đồ hiện đại. Ở mức tổng quát, thuật toán *Verify* chỉ trả về kết quả hợp lệ hoặc không hợp lệ dựa trên khóa công khai, thông điệp và chữ ký; nó không khôi phục thông điệp bằng một phép giải mã [5].

Về phương diện an toàn thông tin, chữ ký số cung cấp ba thuộc tính cơ bản: xác thực nguồn gốc (Authentication), bảo toàn tính toàn vẹn (Integrity) và chống chối bỏ kỹ thuật (Non-repudiation) [1], [5]. Tuy nhiên, cần nhận thức rõ giới hạn biên của công nghệ: chữ ký số chỉ chứng minh tính hợp lệ về mặt toán học giữa bộ dữ liệu *(m, σ, pk)*, chứ không thể chứng minh thực thể hợp pháp thực sự trực tiếp ký nếu khóa bí mật *sk* đã bị lộ lọt, bị mã độc đánh cắp hoặc trích xuất trái phép từ thiết bị lưu trữ [1], [5]. Do đó, giá trị chứng minh của chữ ký trong thực tế phụ thuộc vào quản lý vòng đời khóa, bảo vệ thiết bị ký, gắn khóa với đúng danh tính, ghi nhật ký sử dụng và thu hồi kịp thời khi có sự cố [1], [5].

1.2.4. Định danh, dấu thời gian và nonce

Định danh trong Blockchain cho phép gắn giao dịch với một khóa hoặc tư cách thành viên có quyền [1]. Ở mạng không cấp quyền, địa chỉ thường được tạo từ khóa công khai hoặc dữ liệu dẫn xuất và chủ yếu mang tính bí danh. Ở mạng cấp quyền, khóa công khai có thể được gắn với danh tính tổ chức thông qua chứng thư số và hạ tầng khóa công khai [8]. Trong cả hai trường hợp, chữ ký hợp lệ chỉ chứng minh việc sử dụng khóa tương ứng; mối liên hệ giữa khóa và cá nhân hoặc tổ chức ngoài đời vẫn phụ thuộc quy trình đăng ký, cấp phát và quản trị danh tính.

Dấu thời gian hỗ trợ sắp xếp sự kiện và áp dụng các quy tắc có yếu tố thời hạn, nhưng không nên mặc nhiên coi thời gian do một nút cung cấp là bằng chứng thời gian pháp lý. Mạng phân tán không có đồng hồ vật lý chung tuyệt đối; từng giao thức phải quy định nguồn thời gian, khoảng sai lệch chấp nhận và cách xử lý nút có đồng hồ sai [1], [2]. Khi nghiệp vụ cần bằng chứng thời gian có thẩm quyền, hệ thống phải kết hợp cơ chế xác nhận phù hợp ngoài Blockchain.

Nonce là giá trị được giao thức sử dụng để tạo tính duy nhất hoặc thứ tự. Trong PoW, nút khai thác thay đổi nonce để tìm giá trị băm thỏa điều kiện độ khó

[2]. Trong mô hình tài khoản, nonce thường là bộ đếm tăng dần để các nút phát hiện giao dịch cũ hoặc sai thứ tự. Cơ chế này hỗ trợ chống phát lại khi được kiểm tra cùng chữ ký và ngữ cảnh giao dịch; nó không thay thế toàn bộ biện pháp bảo vệ phiên và giao thức [1], [10].

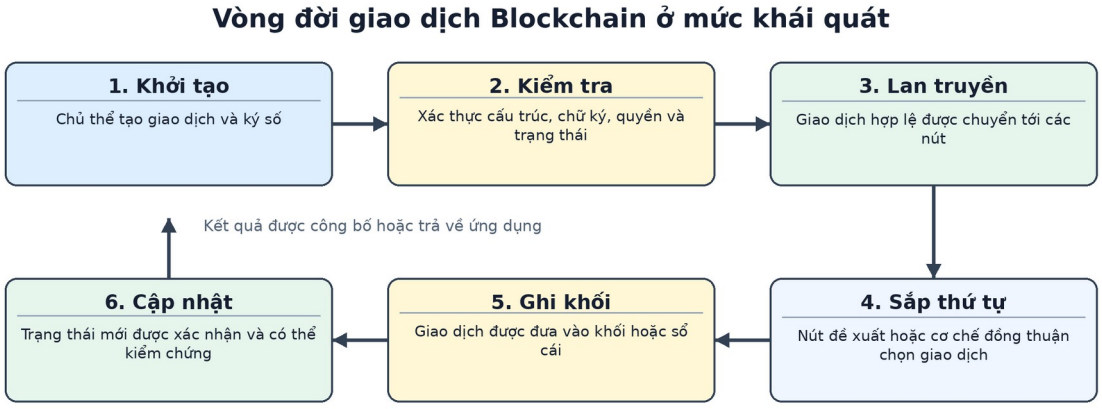
1.3. Kiến trúc và nguyên lý hoạt động

1.3.1. Mạng ngang hàng và vai trò của nút

Mạng ngang hàng (Peer-to-Peer - P2P) cho phép các nút trao đổi trực tiếp giao dịch, khối và thông tin trạng thái mà không bắt buộc mọi thông điệp phải đi qua một máy chủ trung tâm [1], [2]. Mỗi nút kiểm tra dữ liệu nhận được theo quy tắc giao thức trước khi chuyển tiếp cho các nút lân cận. Cách tổ chức này hỗ trợ nhân bản dữ liệu, tăng khả năng tiếp tục hoạt động khi một số kết nối gặp sự cố và làm giảm phụ thuộc vào một điểm truyền tin duy nhất. Tuy nhiên, P2P không đồng nghĩa với phân quyền tuyệt đối: các nút vẫn có thể tập trung tại cùng nhà cung cấp hạ tầng, cùng miền quản trị hoặc chịu ảnh hưởng của một nhóm vận hành nhỏ [1], [10].

Vai trò của nút phụ thuộc nền tảng. Nút đầy đủ thường lưu đủ dữ liệu cần thiết để tự kiểm tra lịch sử; nút nhẹ chỉ giữ một phần thông tin và dựa vào bằng chứng mật mã; nút đề xuất hoặc xác thực tham gia đồng thuận; một số nền tảng còn tách riêng nhiệm vụ sắp thứ tự, thực thi và lưu trữ [1], [8]. Vì vậy, các tên gọi như nút khai thác, nút xác thực hay nút lưu trữ không phải bộ phân loại áp dụng cho mọi Blockchain. Khi đánh giá kiến trúc, cần xác định rõ mỗi nút lưu gì, kiểm tra gì, thuộc tổ chức nào và có quyền tác động đến trạng thái ra sao.

1.3.2. Vòng đời giao dịch



Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát

Hình 1.3 khái quát vòng đời giao dịch từ khởi tạo, kiểm tra, sắp thứ tự đến khi trạng thái được cập nhật. Vòng đời giao dịch bắt đầu khi chủ thể tạo yêu cầu

thay đổi trạng thái và ký bằng khóa bí mật tương ứng. Nút tiếp nhận kiểm tra cấu trúc, chữ ký, số thứ tự chống phát lại, quyền thao tác và điều kiện trạng thái hiện tại [1]. Giao dịch hợp lệ được truyền tới các thành phần tham gia sắp thứ tự hoặc đồng thuận. Trong mạng công khai, giao dịch có thể được giữ trong vùng chờ trước khi nút đề xuất lựa chọn; mạng cấp quyền có thể sử dụng quy trình đề xuất, chứng thực và sắp thứ tự khác mà không có hoạt động khai thác [2], [8].

Sau khi đạt điều kiện chấp nhận của giao thức, giao dịch được đưa vào khối hoặc lô dữ liệu, truyền tới các nút và kiểm tra lần cuối trước khi cập nhật trạng thái. Ứng dụng chỉ nên thông báo thành công khi đã nhận bằng chứng giao dịch được cam kết hợp lệ, thay vì coi việc tạo mã giao dịch hay phát tán yêu cầu là hoàn tất. Mức độ chắc chắn của kết quả phụ thuộc cơ chế đồng thuận: một số mạng cần nhiều khối xác nhận, trong khi mạng dựa trên túc số có thể đạt hoàn tất xác định dưới các giả định vận hành đã nêu [1], [6].

1.3.3. Sao chép trạng thái và thực thi xác định

Blockchain có thể được phân tích như một dạng sao chép máy trạng thái. Gọi S là trạng thái, T là giao dịch và δ là hàm chuyển trạng thái; cùng trạng thái đầu vào và cùng thứ tự giao dịch phải tạo cùng kết quả $S'=\delta(S,T)$ tại các nút trung thực [10]. Điều kiện xác định này đòi hỏi chương trình tránh phụ thuộc trực tiếp vào đồng hồ cục bộ, số ngẫu nhiên không được thống nhất, phản hồi mạng bên ngoài hoặc cách làm tròn khác nhau giữa các máy.

Nhiều nền tảng tách nhật ký giao dịch nối tiếp khỏi phần biểu diễn trạng thái mới nhất để vừa bảo toàn lịch sử vừa truy vấn hiệu quả. Nhật ký cho phép kiểm toán và tái dựng; trạng thái hiện thời phục vụ nghiệp vụ đọc thường xuyên. Đây là mô hình kiến trúc phổ biến nhưng cách tổ chức dữ liệu, xử lý giao dịch không hợp lệ và phục hồi trạng thái thay đổi theo từng nền tảng [8], [10].

1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống

Hợp đồng thông minh là chương trình biểu diễn quy tắc chuyển trạng thái và được các nút thực thi theo cơ chế của nền tảng [1], [10]. Nó có thể chuẩn hóa điều kiện nghiệp vụ và tạo dấu vết nhất quán, nhưng không tự bảo đảm logic được viết đúng. Lỗi phân quyền, kiểm tra đầu vào, xử lý trạng thái hoặc cơ chế nâng cấp có thể gây hậu quả trên toàn mạng. Vì vậy, mã cần được kiểm thử, rà soát an toàn, quản lý phiên bản và triển khai theo quy trình phê duyệt phù hợp.

Môi trường thực thi xác định không thể tùy ý lấy dữ liệu từ Internet vì các nút có thể nhận kết quả khác nhau. Dữ liệu ngoài hệ thống thường được đưa vào qua nguồn cung cấp có thẩm quyền hoặc cơ chế oracle. Chữ ký của nguồn chỉ

chứng minh dữ liệu đến từ khóa tương ứng và không bị sửa sau khi ký; nó không chứng minh nội dung phản ánh đúng sự thật ngoài đời. Thiết kế an toàn cần quy định nguồn tin cậy, thời hạn hiệu lực, cách xử lý sai lệch và cơ chế dừng an toàn khi dữ liệu ngoài không đáng tin [1].

1.4. Cơ chế đồng thuận

1.4.1. Bài toán đồng thuận và mô hình lỗi

Đồng thuận giúp các nút thống nhất thứ tự giao dịch và trạng thái được chấp nhận trong điều kiện thông điệp có thể đến không đồng thời hoặc một số nút gặp sự cố [1]. Hai thuộc tính thường được xem xét là tính an toàn, nghĩa là các nút trung thực không chấp nhận hai quyết định mâu thuẫn trong cùng ngữ cảnh, và tính sống, nghĩa là hệ thống có thể tiếp tục đưa ra quyết định khi các giả định về mạng và số nút lỗi còn được thỏa mãn [6], [7]. Không giao thức nào bảo đảm mọi thuộc tính trong mọi điều kiện; mỗi thiết kế phải nêu mô hình mạng, loại lỗi và ngưỡng đối phương mà nó chịu được.

Mô hình chịu lỗi dừng (Crash Fault Tolerance - CFT) giả định nút lỗi chỉ dừng hoặc mất kết nối, không chủ động tạo thông điệp sai. Mô hình chịu lỗi Byzantine (Byzantine Fault Tolerance - BFT) mạnh hơn vì cho phép nút bị chiếm quyền gửi thông điệp khác nhau tới các bên, thông đồng hoặc cố ý vi phạm giao thức [6], [7]. Việc lựa chọn CFT hay BFT phải dựa trên ranh giới tin cậy: nút trong cùng miền quản trị và được bảo vệ tốt có thể chấp nhận giả định lỗi dừng, còn mạng liên tổ chức có nguy cơ hành vi tùy ý cần đánh giá BFT hoặc biện pháp kiểm soát bổ sung.

1.4.2. Proof of Work và Proof of Stake

Trong mạng không cấp quyền, một thực thể có thể tạo nhiều định danh với chi phí thấp để thao túng biểu quyết. Proof of Work (PoW) gắn quyền đề xuất khối với công việc tính toán, còn Proof of Stake (PoS) gắn quyền tham gia với lượng tài sản cam kết và các quy tắc khuyến khích hoặc xử phạt [1], [2]. Hai nhóm cơ chế này tạo chi phí cho tấn công Sybil mà không cần danh sách thành viên cố định.

PoW có lịch sử vận hành rõ trong Bitcoin nhưng tiêu thụ nhiều năng lượng và thường đạt hoàn tất theo xác suất [2]. PoS giảm nhu cầu tính toán lặp lại, song độ an toàn phụ thuộc cách chọn người đề xuất, cơ chế xử lý nhánh cạnh tranh, phân phối cổ phần và quản trị giao thức. Không phải mọi PoS đều có cùng mô hình hoàn tất hoặc hiệu năng. Đối với mạng gồm các tổ chức đã định danh, PoW và PoS thường không phải lựa chọn đầu tiên vì quyền tham gia có thể được kiểm soát trực tiếp bằng danh tính và chính sách [1], [8].

1.4.3. Đồng thuận CFT và BFT trong mạng định danh

Raft là thuật toán CFT sử dụng nút dẫn dắt để sắp thứ tự và sao chép nhật ký; một mục được cam kết khi đạt đa số theo quy tắc của giao thức [7]. PBFT cổ điển hướng tới mô hình Byzantine và sử dụng các pha trao đổi thông điệp để các bản sao thống nhất thứ tự yêu cầu ngay cả khi có một số nút hành xử tùy ý [6]. So với PoW hoặc PoS, các giao thức này dựa trên tập thành viên đã biết và cơ chế túc số, nên phù hợp hơn với nhiều mạng cấp quyền.

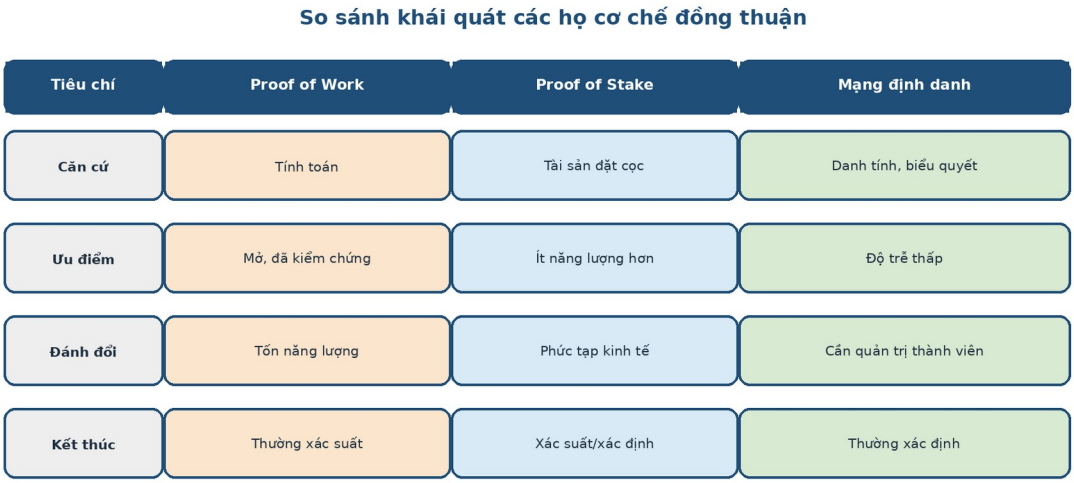
Các ngưỡng chịu lỗi phụ thuộc giả định và giao thức, không phải công thức phổ quát cho mọi thuật toán CFT/BFT. Với cấu hình đa số CFT và mô hình PBFT cổ điển, quan hệ số nút thường được viết:

$$n \geq 2f+1 (CFT); n \geq 3f+1 (BFT) \quad (1.3)$$

Trong đó n là tổng số nút và f là số nút lỗi tối đa. Với $n=3f+1$, một túc số BFT thường có $2f+1$ phiếu; hai túc số như vậy giao nhau ít nhất $f+1$ nút, nên phần giao chứa ít nhất một nút trung thực khi có tối đa f nút Byzantine [6]. PBFT cổ điển còn có chi phí truyền thông bậc hai $O(n^2)$ ở các pha nhiều nút trao đổi chéo, do đó chi phí tăng nhanh theo quy mô mạng [6].

1.4.4. Tính hoàn tất và đánh đổi thiết kế

Tính hoàn tất (finality) biểu thị mức độ chắc chắn rằng một giao dịch đã được chấp nhận sẽ không bị đảo ngược trong điều kiện an toàn giả định. Trong cơ chế hoàn tất theo xác suất, khả năng tổ chức lại lịch sử giảm dần khi có thêm khối xác nhận nhưng không bằng không về mặt tuyệt đối [2]. Trong cơ chế hoàn tất xác định dựa trên túc số, giao dịch được coi là hoàn tất sau khi đạt đủ phiếu theo giao thức; kết luận này vẫn phụ thuộc vào giả định số nút lỗi, an toàn khóa và quy tắc quản trị [6], [7]. Sự khác biệt khái quát giữa các nhóm cơ chế đồng thuận được trình bày trên Hình 1.4.

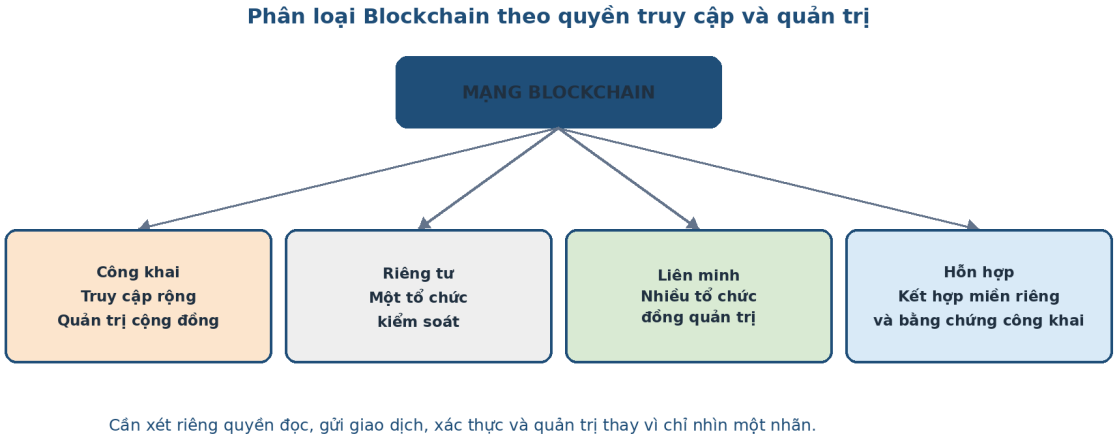


Hình 1.4. So sánh khái quát các nhóm cơ chế đồng thuận

Lựa chọn đồng thuận là bài toán đánh đổi giữa mức phân tán quyền quyết định, khả năng chịu lỗi, độ trễ, thông lượng và chi phí vận hành [10]. "Bộ ba phân quyền – an toàn – khả năng mở rộng" có thể dùng như một khung trực giác, không phải định luật buộc mọi hệ thống chỉ được chọn hai thuộc tính. Cơ chế phù hợp phải được đánh giá trên tải thực tế, số tổ chức vận hành, loại hành vi đối phương và yêu cầu phục hồi, thay vì dựa trên tên gọi hoặc tuyên bố hiệu năng của nền tảng.

1.5. Phân loại Blockchain và điều kiện lựa chọn

1.5.1. Các trục phân loại



Hình 1.5. Phân loại Blockchain theo phạm vi tham gia và quyền truy cập

Hình 1.5 khái quát các trục phân loại theo quyền tham gia và phạm vi quản trị. Blockchain có thể được phân loại theo nhiều trục độc lập. Trục quyền tham gia phân biệt mạng không cấp quyền, nơi chủ thể có thể tham gia theo quy tắc mở, với mạng cấp quyền, nơi danh tính và vai trò được phê duyệt trước [1]. Trục phạm vi quản trị phân biệt mạng công khai, mạng riêng do một miền quản trị chi phối và

mạng liên minh do nhiều tổ chức cùng quản trị. Quyền đọc dữ liệu lại là một quyết định khác: một mạng cấp quyền có thể cho công chúng đọc bằng chứng tối thiểu, trong khi một mạng công khai vẫn có thể sử dụng cơ chế bảo vệ nội dung riêng tư. Vì vậy, không nên đồng nhất "công khai" với "không cấp quyền", hoặc "riêng tư" với "cấp quyền" [1], [8].

1.5.2. So sánh các mô hình triển khai

Bảng 1.1. So sánh khái quát các mô hình triển khai Blockchain

Tiêu chí	Công khai, không cấp quyền	Riêng, cấp quyền	Liên minh, cấp quyền
Quản trị	Cộng đồng và quy tắc giao thức	Một tổ chức hoặc một miền quản trị	Nhiều tổ chức theo chính sách chung
Thành viên xác thực	Mở theo điều kiện chống Sybil	Danh tính được phê duyệt nội bộ	Danh tính thuộc các tổ chức thành viên
Khả năng kiểm chứng	Rộng, thường công khai	Chủ yếu trong nội bộ	Giữa các thành viên và bên được cấp quyền
Quyền riêng tư	Khó kiểm soát phạm vi phổ biến dữ liệu	Dễ giới hạn hơn theo chính sách nội bộ	Có thể phân vùng theo tổ chức và nghiệp vụ
Hiệu năng	Phụ thuộc mạng mở và cơ chế kinh tế	Có thể tối ưu trong phạm vi nhỏ	Phụ thuộc số tổ chức, túc số và hạ tầng
Bài toán phù hợp	Hệ sinh thái mở cần kiểm chứng rộng	Nhật ký và quy trình nội bộ	Phối hợp và kiểm toán liên tổ chức

Bảng 1.1 cho thấy không có mô hình triển khai nào mặc nhiên tốt hơn; lựa chọn phụ thuộc ranh giới tin cậy, yêu cầu kiểm chứng, riêng tư và vận hành. Mạng mở tạo khả năng kiểm chứng rộng và chống Sybil bằng nguồn lực kinh tế, nhưng thường phát sinh phí, độ trễ và khó kiểm soát phạm vi phổ biến dữ liệu. Mạng riêng dễ quản trị và tối ưu hiệu năng, song giá trị phân tán niềm tin bị hạn chế nếu một tổ chức vẫn quyết định toàn bộ lịch sử. Mạng liên minh phân bổ quyền giữa nhiều tổ chức đã định danh, thuận lợi cho kiểm toán liên tổ chức và chính sách dữ liệu có kiểm soát, nhưng đòi hỏi thỏa thuận quản trị, vận hành nút và xử lý tranh chấp [1], [8], [10].

Không mô hình nào mặc nhiên tốt hơn. Hiệu năng thực tế phụ thuộc thuật toán đồng thuận, số nút, chính sách xác nhận, hạ tầng mạng, kích thước giao dịch và cách tổ chức dữ liệu. Vì vậy, các con số thông lượng hoặc độ trễ chỉ có ý nghĩa khi được đo trên cấu hình và tải xác định [10].

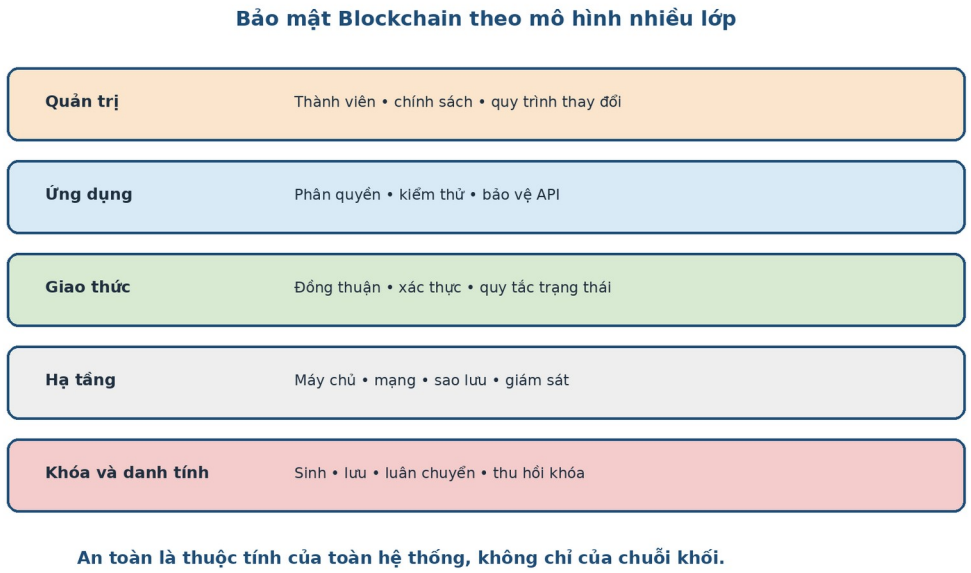
1.5.3. Khi nên và không nên sử dụng Blockchain

Nên xem xét Blockchain khi nhiều tổ chức cần cùng cập nhật trạng thái, lịch sử phải được kiểm chứng độc lập và không bên nào được chấp nhận làm chủ thể duy nhất có quyền sửa dữ liệu. Quy trình cũng phải đủ rõ để biểu diễn thành giao dịch, các bên có động lực vận hành chung và dữ liệu có thể được tối thiểu hóa phù hợp với yêu cầu riêng tư [1], [10].

Không nên sử dụng Blockchain nếu chỉ một tổ chức chịu trách nhiệm và được các bên tin cậy, nếu dữ liệu phải sửa hoặc xóa thường xuyên, hoặc nếu tải và độ trễ vượt khả năng của cơ chế đồng thuận. Một cơ sở dữ liệu truyền thống có nhật ký kiểm toán và chữ ký số có thể đơn giản, rẻ và dễ quản trị hơn trong các trường hợp đó. Blockchain cũng không sửa được một quy trình nghiệp vụ mơ hồ, dữ liệu đầu vào kém chất lượng hay tranh chấp trách nhiệm giữa các tổ chức.

1.6. An toàn, quyền riêng tư và khả năng mở rộng

1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp



Hình 1.6. Các lớp bảo vệ trong một hệ thống Blockchain

Hình 1.6 khái quát các lớp bảo vệ cần xem xét trong một hệ thống Blockchain. An toàn Blockchain không chỉ phụ thuộc hàm băm hoặc đồng thuận mà phải được xem xét theo nhiều lớp: mật mã, mạng, đồng thuận, hợp đồng thông minh, ứng dụng, lưu trữ ngoài chuỗi và quản trị vận hành [1], [10]. Tài sản cần bảo vệ gồm khóa bí mật, danh tính thành viên, trạng thái sổ cái, dữ liệu nghiệp vụ, cấu

hình nút và nhật ký. Đối phương có thể là người ngoài tấn công hạ tầng, người dùng bị chiếm tài khoản, quản trị viên lạm quyền hoặc một nhóm nút thông đồng.

Biện pháp bảo vệ phải bám vào mô hình đe dọa. Kênh truyền cần được mã hóa và xác thực; nút cần vá lỗi, phân đoạn mạng, giám sát và sao lưu; chính sách đồng thuận phải giới hạn quyền của một tổ chức; ứng dụng phải kiểm soát đầu vào, phiên đăng nhập và phân quyền. Việc nhiều nút lưu sổ cái không thay thế kế hoạch ứng phó sự cố: nếu các nút cùng phụ thuộc một hạ tầng, cùng dùng cấu hình sai hoặc cùng bị lộ khóa, lỗi có thể lan rộng trên toàn hệ thống.

1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài

Khóa bí mật đại diện cho quyền ký giao dịch nên cần được quản lý suốt vòng đời: sinh bằng nguồn ngẫu nhiên an toàn, lưu trong kho được bảo vệ, giới hạn mục đích sử dụng, luân chuyển, sao lưu có kiểm soát và thu hồi khi nghi ngờ lộ [5]. Mô-đun bảo mật phần cứng, chữ ký nhiều bên hoặc mật mã ngưỡng có thể giảm rủi ro một khóa hay một cá nhân trở thành điểm lỗi đơn, nhưng không loại bỏ nhu cầu quản trị và phục hồi.

Hợp đồng thông minh cần kiểm tra quyền, dữ liệu đầu vào, chuyển trạng thái và các trường hợp đồng thời; phiên bản mới phải có quy trình rà soát, kiểm thử và phê duyệt [10]. Nguồn dữ liệu ngoài phải được xác định thẩm quyền, ký số khi phù hợp và có thời hạn hiệu lực. Dùng nhiều nguồn chỉ hữu ích khi các nguồn đủ độc lập; nếu tất cả cùng lấy dữ liệu từ một hệ thống gốc, biểu quyết không làm dữ liệu đáng tin hơn.

1.6.3. Quyền riêng tư và dữ liệu cá nhân

Sổ cái được nhân bản và khó sửa đặt ra xung đột với các nguyên tắc bảo vệ dữ liệu cá nhân như giới hạn mục đích, tối thiểu hóa dữ liệu và quản lý thời hạn lưu giữ. Đưa trực tiếp họ tên, ngày sinh, email, mã sinh viên, điểm số hoặc toàn bộ tệp văn bằng lên blockchain làm tăng phạm vi sao chép và khiến việc chỉnh sửa, hạn chế xử lý hoặc xóa dữ liệu trở nên khó thực hiện. Mã hóa cũng không loại bỏ hoàn toàn rủi ro vì bản mã vẫn tồn tại lâu dài và khóa giải mã có thể bị lộ trong tương lai [1], [9].

Đối với bối cảnh Việt Nam, Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 có hiệu lực từ ngày 01/01/2026 và Nghị định 356/2025/NĐ-CP quy định chi tiết một số nội dung thi hành [26], [27]. Vì vậy, thiết kế của đồ án áp dụng nguyên tắc tối thiểu hóa dữ liệu trên chuỗi: hồ sơ người học, thông tin tài khoản và dữ liệu nghiệp vụ được giữ ngoài chuỗi trong PostgreSQL; Bitcoin chỉ nhận cam kết mật mã tối thiểu phục vụ kiểm chứng. Giá trị băm không mặc nhiên là dữ liệu ẩn danh

tuyệt đối, do đó việc công bố tệp kết quả, verification log, audit log và dữ liệu thử nghiệm vẫn phải được rà soát nguy cơ liên kết hoặc suy đoán ngược.

1.6.4. Khả năng mở rộng và khả năng tương tác

Khi nhiều nút cùng lưu và kiểm tra giao dịch, thông lượng, độ trễ, băng thông và dung lượng trở thành các giới hạn cần đo [10]. Có thể cải thiện bằng cách xử lý theo lô, tối ưu chính sách xác nhận, phân vùng dữ liệu hoặc đưa một phần xử lý ra ngoài chuỗi; mỗi cách tạo thêm giả định tin cậy và độ phức tạp. Các giải pháp lớp 2 chỉ nên được xem xét khi thật sự kế thừa an toàn từ chuỗi cơ sở, không nên đồng nhất mọi sidechain hoặc cầu nối với lớp 2.

Khả năng tương tác không chỉ là truyền thông điệp giữa hai mạng mà còn là thống nhất ý nghĩa dữ liệu, định danh tổ chức, phiên bản lược đồ và trạng thái nghiệp vụ. Cầu nối hoặc dịch vụ chuyển tiếp bổ sung điểm tin cậy mới và từng là mục tiêu của nhiều sự cố. Các chuẩn dữ liệu chứng thực như W3C Verifiable Credentials giúp thống nhất cách biểu diễn vai trò phát hành, chủ thể và bên xác minh, nhưng không tự giải quyết quản trị khóa, quyền truy cập hay tranh chấp pháp lý [9].

1.7. Định hướng ứng dụng trong quản lý văn bằng, chứng chỉ

1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng

Quản lý văn bằng, chứng chỉ liên quan ba vai trò chính: cơ sở đào tạo phát hành, người học nắm giữ hoặc chia sẻ và tổ chức bên ngoài xác minh [9]. Quy trình truyền thống thường dựa vào bản giấy, bản sao chứng thực hoặc yêu cầu xác nhận thủ công; khi dữ liệu nằm ở nhiều hệ thống, việc đối soát có thể chậm và khó phát hiện tài liệu đã bị chỉnh sửa. Một số bằng chứng dùng chung có thể hỗ trợ bên xác minh kiểm tra nguồn phát hành, tính toàn vẹn và trạng thái hiện thời mà giảm số bước liên hệ thủ công. Mô hình khái niệm của bài toán được trình bày trên Hình 1.7.



Hình 1.7. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bằng

Yêu cầu kiểm chứng không chỉ là so sánh mã băm. Hệ thống phải xác định đơn vị nào có thẩm quyền phát hành, khóa nào hợp lệ tại thời điểm ký, dữ liệu nào tạo thành nội dung văn bằng và trạng thái nào cho phép sử dụng. Kết quả không tìm thấy bằng chứng cũng không đủ để kết luận văn bằng giả, bởi hồ sơ cũ có thể chưa được số hóa hoặc nằm ngoài phạm vi hệ thống.

1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi

Ở mức khái niệm, sổ cái chỉ nên giữ lượng dữ liệu tối thiểu phục vụ kiểm chứng, chẳng hạn cam kết mật mã, tham chiếu đơn vị phát hành và trạng thái hiệu lực. Hồ sơ đầy đủ cùng dữ liệu cá nhân được quản lý ngoài chuỗi trong hệ thống có thẩm quyền, với mã hóa, phân quyền, nhật ký truy cập và sao lưu. Hai miền được liên kết bằng quy tắc tạo bằng chứng xác định để tài liệu bị thay đổi tạo kết quả kiểm tra khác [1], [9].

Cách phân tách này giảm việc sao chép dữ liệu nhạy cảm nhưng không làm rủi ro riêng tư biến mất. Bằng chứng trên chuỗi vẫn có thể bị liên kết hoặc dò đoán; kho ngoài chuỗi vẫn có thể bị mất, sửa hoặc ngừng phục vụ. Vì vậy, Chương II cần xác định mô hình dữ liệu, quy tắc chuẩn hóa, quyền truy cập và cơ chế xử lý khi dữ liệu nguồn sai mà không giả định Blockchain tự giải quyết các vấn đề đó.

1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm

Giá trị kỳ vọng của hướng tiếp cận là rút ngắn thời gian xác minh, tăng khả năng phát hiện tài liệu bị sửa và tạo dấu vết trạng thái có thể kiểm chứng. Công nghệ có thể hỗ trợ ghi nhận phát hành hoặc thay đổi hiệu lực, nhưng không tự quyết định người học đủ điều kiện tốt nghiệp, không chứng minh người xuất trình là chủ văn bằng và không thay thế thẩm quyền cấp văn bằng của cơ sở giáo dục.

Cơ sở đào tạo vẫn chịu trách nhiệm về dữ liệu đầu vào, quyết định công nhận tốt nghiệp và thẩm quyền phát hành; người vận hành chịu trách nhiệm bảo vệ

khóa và hạ tầng; bên xác minh phải diễn giải kết quả đúng phạm vi. Thông tư 10/2026/TT-BGDĐT đã bổ sung khái niệm và cơ chế quản lý văn bằng, chứng chỉ số, đồng thời xác định vai trò của cơ sở dữ liệu văn bằng, chứng chỉ trong hoạt động cấp và xác minh [25]. Vì vậy, chứng thư Blockcerts trong đồ án được định vị là tệp kết quả kỹ thuật của nguyên mẫu hỗ trợ phát hành và kiểm chứng, không mặc nhiên thay thế văn bằng, chứng chỉ số có giá trị pháp lý theo quy định hiện hành.

1.7.4. Mô hình Blockcerts V3 và quan hệ với Verifiable Credentials

Blockcerts V3 tổ chức chứng thư dưới dạng JSON-LD, gồm định danh chứng thư, thông tin chủ thể, issuer và bằng chứng mật mã; cú pháp JSON-LD được quy định trong [12], còn cấu trúc Blockcerts được đối chiếu với schema/context của dự án [11]. Báo cáo dùng W3C Verifiable Credentials Data Model v2.0 để mô tả ba vai trò issuer–holder–verifier [9]. Tuy nhiên, toolchain thực nghiệm phát hành credential với context https://www.w3.org/2018/credentials/v1; do đó tệp kết quả được đối chiếu thêm với VC Data Model v1.1 [24]. Phân biệt này tránh diễn giải rằng nguyên mẫu đã chuyển hoàn toàn sang context VC v2.

Trong quá trình phát hành, nội dung cần bảo vệ được chuẩn hóa theo quy tắc của suite, tính target hash và đưa vào cây Merkle. Merkle root đại diện cho toàn bộ lô; mỗi chứng thư có một đường dẫn riêng cùng thông tin anchor. Nhiều chứng thư vì thế có thể dùng chung một transaction nhưng vẫn có bằng chứng để phát hiện thay đổi nội dung [11], [14]. Chữ ký giao dịch Bitcoin chứng minh quyền sử dụng khóa phát hành, còn việc gắn khóa đó với cơ sở đào tạo có thẩm quyền phụ thuộc Issuer Profile và quy trình quản trị danh tính.

1.7.5. Bitcoin OP_RETURN và môi trường regtest

Bitcoin cho phép ghi một lượng dữ liệu nhỏ vào output OP_RETURN không thể chi tiêu. Nguyên mẫu chỉ neo Merkle root hoặc dấu vết thu hồi tối thiểu, không ghi toàn bộ hồ sơ người học lên blockchain. Cách tiếp cận này giảm dữ liệu công khai và phù hợp với nguyên tắc tối thiểu hóa dữ liệu; hồ sơ đầy đủ vẫn phải được bảo vệ, sao lưu và quản lý ngoài chuỗi.

Môi trường regtest được lựa chọn để chủ động tạo block, không dùng tiền thật và tái lập thí nghiệm ổn định [18]. Regtest chỉ phù hợp để chứng minh logic phát hành, xác minh, confirmation và recovery trong môi trường kiểm soát; nó không phản ánh phí, độ trễ mempool, mức phân tán nút hay rủi ro tổ chức lại chuỗi của Bitcoin mainnet.

1.7.6. Bối cảnh pháp lý và ranh giới áp dụng tại Việt Nam

Thông tư 10/2026/TT-BGDĐT có hiệu lực từ ngày 15/04/2026, bổ sung quy định về văn bằng, chứng chỉ số, cơ sở dữ liệu văn bằng, chứng chỉ và hoạt động cập nhật, khai thác, xác minh thông tin [25]. Quy định này cho thấy bài toán văn bằng số hiện đã có khuôn khổ pháp lý cụ thể hơn so với giai đoạn trước. Tuy nhiên, một chứng thư kỹ thuật do nguyên mẫu tự phát hành không tự động trở thành văn bằng số hợp pháp; giá trị pháp lý còn phụ thuộc chủ thể có thẩm quyền, quy trình nghiệp vụ, dữ liệu được cập nhật vào hệ thống/cơ sở dữ liệu theo quy định và các điều kiện pháp lý liên quan.

Do hệ thống xử lý họ tên, ngày sinh, email, mã sinh viên và thông tin văn bằng, việc triển khai thực tế còn phải tuân thủ Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 và các quy định hướng dẫn thi hành [26], [27]. Trong phạm vi đồ án, dữ liệu benchmark là dữ liệu giả lập; bí mật, credential runtime và dữ liệu cá nhân không cần thiết không được công bố trong repository hoặc bộ bằng chứng.

1.7.7. Đối chiếu các phương án và phạm vi đóng góp

Bảng 1.2 đối chiếu ba cách tiếp cận cho cùng bài toán. Cơ sở dữ liệu và chữ ký số vẫn có thể đáp ứng quản lý nội bộ và kiểm tra tính toàn vẹn. Blockcerts bổ sung cách đóng gói bằng chứng; việc neo lên một mạng công khai nhằm tạo cam kết lịch sử có thể được kiểm tra ngoài cơ sở dữ liệu của đơn vị cấp. Giá trị bổ sung này phụ thuộc ranh giới tin cậy, khả năng truy cập dữ liệu issuer và chính sách thu hồi [1], [9], [11].

Bảng 1.2. Đối chiếu các phương án quản lý và kiểm chứng văn bằng

Tiêu chí	CSDL và chữ ký số	Chứng thư theo chuẩn VC	Blockcerts neo Bitcoin
Bằng chứng toàn vẹn	Chữ ký trên dữ liệu; kiểm tra khóa	Phụ thuộc cơ chế bảo vệ của credential	Target hash, Merkle proof và anchor
Lịch sử cam kết	Phụ thuộc kho log và miền quản trị	Chuẩn VC không tự quy định sổ cái	Có cam kết trên chuỗi; regtest vẫn do lab kiểm soát
Thu hồi	CSDL hoặc danh sách trạng thái	Cơ chế trạng thái theo hệ triển khai	Nguyên mẫu dùng Revocation List và dấu vết riêng

Tiêu chí	CSDL và chữ ký số	Chứng thư theo chuẩn VC	Blockcerts neo Bitcoin
Chi phí vận hành	Không phát sinh phí neo Blockchain	Phụ thuộc cơ chế bảo vệ được chọn	Thêm node/RPC, phí và confirmation nếu dùng mainnet
Phạm vi đồ án	Phương án đối chiếu về thiết kế	Khung dữ liệu để đối chiếu	Nguyên mẫu được hiện thực và đo trên regtest

Đồ án kế thừa Blockcerts và các chuẩn có sẵn; đóng góp tập trung vào tích hợp Maker–Checker, phân quyền đối tượng, outbox, checkpoint/phục hồi và chuỗi bằng chứng tái lập. Báo cáo chưa có thí nghiệm đối chứng giữa ba phương án trên, nên không khẳng định Blockchain nhanh hơn hoặc an toàn hơn trong mọi tình huống. Việc kiểm tra trên regtest chứng minh logic cam kết và xác minh, chưa chứng minh mức phân tán niềm tin của Bitcoin mainnet.

1.8. Tổng kết Chương I

1.8.1. Kết quả lý luận chính

Chương I đã trình bày Blockchain như một kiến trúc sổ cái phân tán kết hợp liên kết băm, chữ ký số, mạng ngang hàng và đồng thuận; đồng thời phân tích các giới hạn về dữ liệu đầu vào, quản trị khóa, quyền riêng tư và khả năng suy rộng kết quả. Đối với bài toán văn bằng, chương đã làm rõ vai trò của Blockcerts V3, Merkle proof, Bitcoin OP_RETURN, mối quan hệ giữa tệp kết quả Blockcerts với W3C Verifiable Credentials và ranh giới pháp lý của văn bằng, chứng chỉ số tại Việt Nam [9], [24]–[27].

1.8.2. Cơ sở chuyển sang Chương II

Từ các cơ sở trên, thiết kế phù hợp cho nguyên mẫu là giữ dữ liệu nghiệp vụ và dữ liệu cá nhân ngoài chuỗi, chỉ neo bằng chứng mật mã tối thiểu lên Bitcoin, đồng thời duy trì quy trình phê duyệt, phân quyền, thu hồi, audit và cơ chế phục hồi ở lớp ứng dụng. Chương II chuyển các nguyên tắc này thành yêu cầu chức năng, mô hình đe dọa, kiến trúc, máy trạng thái và mô hình dữ liệu bám sát mã nguồn hiện hành.

CHƯƠNG II. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích bài toán

2.1.1. Bối cảnh nghiệp vụ

Quy trình quản lý văn bằng có ba nhóm rủi ro chính. Thứ nhất, dữ liệu có thể bị nhập sai hoặc sửa trái phép trong hệ thống tập trung. Thứ hai, bên tuyển dụng hoặc tổ chức tiếp nhận thường phải phụ thuộc vào cổng tra cứu hoặc xác minh thủ công của đơn vị cấp. Thứ ba, nếu một chủ thể vừa lập vừa duyệt yêu cầu phát hành thì nguy cơ lạm quyền khó được kiểm soát. Blockchain chỉ hỗ trợ kiểm chứng tính toàn vẹn của dữ liệu sau khi dữ liệu đã được chấp nhận; nó không tự chứng minh dữ liệu đầu vào là đúng [1], [10]. Vì vậy hệ thống phải kết hợp kiểm soát quy trình Maker–Checker, quản trị khóa, dữ liệu ngoài chuỗi và bằng chứng blockchain.

Hệ thống được thiết kế cho một cơ sở đào tạo phát hành chứng thư Blockcerts V3 để hỗ trợ quy trình quản lý và xác minh. Tài khoản, dữ liệu người học, thông tin văn bằng và trạng thái nghiệp vụ lưu trong PostgreSQL; chứng thư JSON lưu ngoài chuỗi; Bitcoin chỉ nhận Merkle root của lô hoặc dấu vết thu hồi tối thiểu qua OP_RETURN. Chứng thư Blockcerts của nguyên mẫu là tệp kết quả kỹ thuật phục vụ nghiên cứu, không được đồng nhất với văn bằng, chứng chỉ số có giá trị pháp lý theo Thông tư 10/2026/TT-BGDĐT [25].

2.1.2. Phạm vi

Phạm vi đồ án gồm đăng nhập, quản lý tài khoản, lập–duyệt yêu cầu, phát hành đơn/lô tối đa 500 chứng thư, cổng Holder, xác minh công khai, thu hồi, audit và triển khai pilot. Bitcoin chạy regtest [18]. Hệ thống không thay thế nghiệp vụ xét tốt nghiệp, không tự xác lập giá trị pháp lý cho văn bằng số, không xác thực danh tính ngoài các tài khoản do đơn vị tạo và không được xem là kiến trúc môi trường vận hành thực tế hoặc Bitcoin mainnet. Dữ liệu thử nghiệm dùng dữ liệu giả lập; thông tin bí mật và dữ liệu cá nhân không cần thiết bị loại khỏi bộ bằng chứng công khai.

2.2. Tác nhân và yêu cầu chức năng

2.2.1. Tác nhân

Hệ thống có ba vai trò xác thực và một tác nhân công khai:

• **Maker:** lập yêu cầu đơn hoặc lô; xem trạng thái và audit; không được duyệt hoặc thu hồi.

• **Checker:** duyệt/từ chối yêu cầu, quản lý tài khoản, xem audit và thu hồi chứng thư đã phát hành.

• **Student:** đăng nhập cổng Holder để xem/tải chứng thư được liên kết với mình; không được vào cổng quản trị.

• **Người xác minh:** không cần đăng nhập; tải hoặc dán JSON chứng thư để nhận kết luận.

Không có vai trò Admin độc lập trong phiên bản mã nguồn được báo cáo. Tên "cổng Admin" chỉ mô tả cổng nghiệp vụ phía cơ sở đào tạo dành cho Maker và Checker, không phải một role thứ tư. Các tác nhân và nhóm chức năng tương ứng được khái quát trên Hình 2.1.



Hình 2.1. Tác nhân và nhóm chức năng

2.2.2. Yêu cầu chức năng

Bảng 2.1. Danh sách yêu cầu chức năng

Mã	Yêu cầu	Tác nhân
FR-01	Đăng nhập và nhận JWT	Maker, Checker, Student
FR-02	Tạo/list tài khoản, không trả password hash	Checker
FR-03	Tạo một yêu cầu phát hành	Maker
FR-04	Tạo lô 1–500 yêu cầu	Maker
FR-05	Duyệt hoặc từ chối yêu cầu	Checker
FR-06	Phát hành Blockcerts V3 và neo Bitcoin	Worker sau khi Checker duyệt

Mã	Yêu cầu	Tác nhân
FR-07	Xem danh sách, chi tiết và trạng thái	Maker, Checker
FR-08	Xem/tải chứng thư của mình	Student
FR-09	Xác minh chứng thư JSON	Công khai
FR-10	Thu hồi chứng thư đã phát hành	Checker
FR-11	Công bố Issuer Profile và Revocation List	Công khai
FR-12	Ghi và đọc audit nghiệp vụ	Hệ thống; Maker/Checker đọc
FR-13	Kiểm tra sức khỏe dịch vụ	Công khai có giới hạn thông tin

Bảng 2.1 là cơ sở truy vết từ yêu cầu chức năng sang use case, API và kiểm thử ở các phần tiếp theo.

2.2.3. Quy tắc nghiệp vụ

Yêu cầu mới có trạng thái pending_approval. Maker không được gọi API duyệt; Checker lấy danh tính từ JWT thay vì tin trường actor do client gửi. Khi Checker duyệt, các thay đổi trạng thái DB, bản ghi issuance_batches và bản ghi issuance_outbox được commit trong cùng một transaction; phiếu chuyển queued, còn dispatcher chịu trách nhiệm đưa job sang BullMQ. Worker lần lượt xử lý processing, tạo bằng chứng/anchor và kết thúc ở issued hoặc trạng thái lỗi/phục hồi tương ứng.

Đối với lô, hệ thống tách requestBatchId (nhóm phiếu Maker tạo) khỏi issuanceBatchId (một lần Checker thực sự duyệt và phát hành). Một requestBatchId có thể sinh nhiều issuanceBatchId khi Checker duyệt từng phần; mỗi chứng thư chỉ thuộc một lô phát hành cụ thể. Các chứng thư cùng issuanceBatchId có thể dùng chung Merkle root và transaction id nhưng có certUid riêng. Vì vậy cột txid không đặt unique. Cơ chế này đồng thời tránh xung đột khi duyệt từng phần và tạo khóa idempotency ổn định cho worker.

2.3. Yêu cầu phi chức năng và mô hình đe dọa

2.3.1. Yêu cầu phi chức năng

• **An toàn:** băm mật khẩu; JWT có thời hạn; RBAC ở backend; bí mật ngoài Git; giới hạn tần suất và kích thước yêu cầu.

• **Toàn vẹn:** sửa payload, proof hoặc transaction id phải không còn kết luận VALID.

• **Truy vết:** lưu người lập, người duyệt, người thu hồi, transaction và audit.

• **Hiệu năng:** tối đa 500 chứng thư/lô trong phạm vi nguyên mẫu; một transaction neo cho mỗi lô.

• Khả dụng: tách request HTTP khỏi tác vụ phát hành dài; dùng cơ chế outbox giao dịch, BullMQ, checkpoint, retry/reconciliation và chính sách khởi động lại.

• **Vận hành:** /health chỉ phản ánh liveness của backend; kiểm tra readiness/dependency được thực hiện bằng script triển khai đối với PostgreSQL, Redis, Bitcoin, Nginx và migration.

• **Tái lập:** dùng Compose, migration, scripts, freeze và checksum tệp kết quả.

• **Riêng tư:** không ghi hồ sơ cá nhân trực tiếp lên Bitcoin; hạn chế dữ liệu trong log/bằng chứng; dữ liệu thử nghiệm là dữ liệu giả lập và secret nằm ngoài Git.

2.3.2. Tài sản và đối thủ

Tài sản gồm WIF phát hành, JWT secret, RPC/database credential, tài khoản, chứng thư, trạng thái phát hành–thu hồi, audit và cấu hình triển khai. Đối thủ có thể là người ngoài gửi chứng thư giả, tài khoản nội bộ vượt quyền, kẻ lấy được khóa, hoặc người vận hành sửa DB. Các bề mặt chính là API công khai, cổng đăng nhập, reverse proxy, queue, RPC Bitcoin, tệp workspace và chuỗi cung ứng phần mềm. Các đe dọa chính và biện pháp kiểm soát được tổng hợp tại Bảng 2.2.

Bảng 2.2. Đe dọa và kiểm soát

Đe dọa	Hậu quả	Kiểm soát thiết kế	Giới hạn còn lại
Maker tự duyệt	Phát hành không có kiểm soát chéo	RBAC, Checker-only approve	Tài khoản Checker bị chiếm vẫn nguy hiểm
Sửa chứng thư	Văn bằng giả	Hash, proof và anchor	Không phát hiện dữ liệu sai từ đầu vào hợp lệ
Chiếm mật khẩu	Mạo danh	bcrypt, JWT, throttling	Chưa có MFA
Lộ WIF	Neo bằng chứng giả	Bí mật ngoài Git, RPC cục bộ	Chưa dùng HSM/KMS

Đe dọa	Hậu quả	Kiểm soát thiết kế	Giới hạn còn lại
Sửa DB thu hồi	Kết luận trạng thái sai	Checkpoint/ idempotency, OP_RETURN thu hồi, audit hash chain	Chưa có indexer tái tạo toàn bộ trạng thái từ blockchain
DoS Verify	Cạn tài nguyên	body limit, throttling, Nginx	Chưa thử tải tấn công dài hạn
Lộ dữ liệu runtime	Rò rỉ thông tin	Workspace ngoài Git, quyền tệp, che thông tin nhạy cảm trong nhật ký/bằng chứng	Cần chính sách lưu giữ, xóa và giám sát truy cập
Sửa hoặc xóa audit DB	Mất khả năng truy trách nhiệm	Hash chain previousHash/entry Hash và endpoint kiểm tra toàn vẹn	DBA vẫn có thể viết lại cả chuỗi hoặc xóa đuôi nếu head hash không neo ra hệ thống độc lập

OWASP khuyến nghị từ chối mặc định, kiểm tra quyền trên mọi yêu cầu và kiểm thử logic phân quyền [16]. Vì vậy frontend chỉ ẩn chức năng để cải thiện trải nghiệm; backend mới là lớp quyết định quyền.

2.4. Lựa chọn công nghệ và kiến trúc

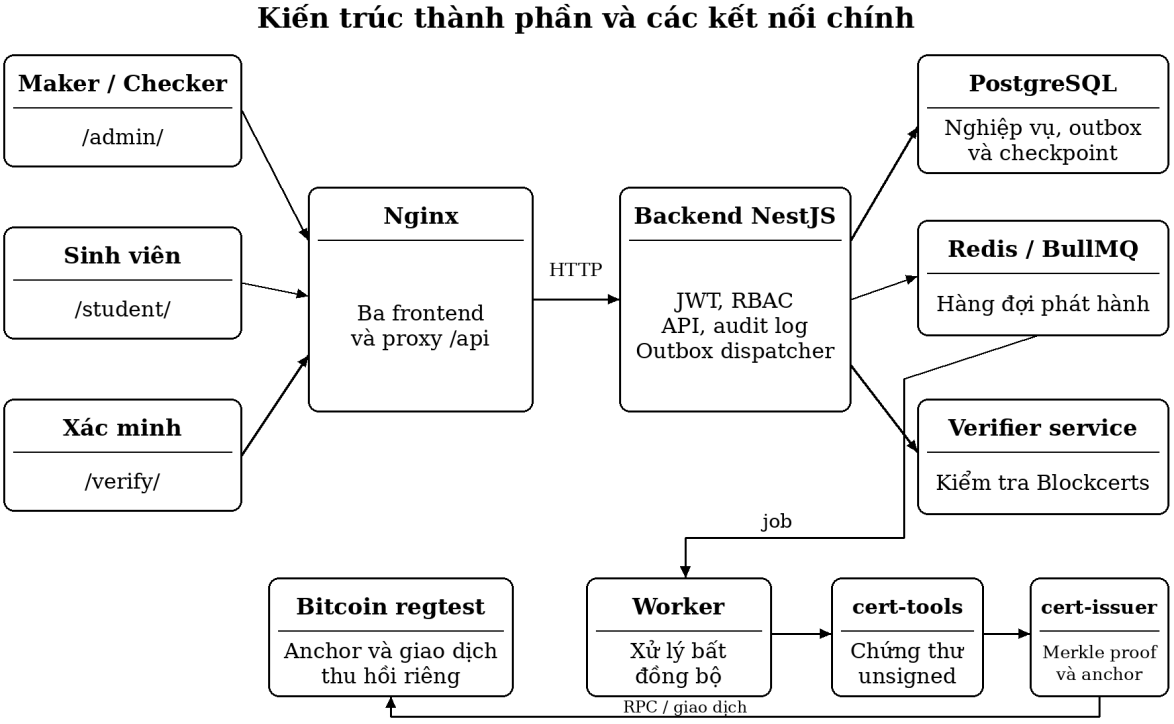
2.4.1. Lựa chọn mô hình Blockcerts–Bitcoin

Blockcerts biểu diễn chứng thư dưới dạng JSON-LD/Verifiable Credential và gắn bằng chứng Merkle. W3C VC Data Model v2.0 được dùng để mô tả mô hình issuer–holder–verifier và các nguyên tắc hiện hành [9], trong khi tệp kết quả Blockcerts V3 thực tế của nguyên mẫu vẫn dùng context VC v1; vì vậy cấu trúc chứng thư được đối chiếu đồng thời với VC v1.1, schema/context Blockcerts và bộ MerkleProof2019 [24], [11], [14]. Trong chứng thư hiện tại, proof.type là DataIntegrityProof, cryptosuite là merkle-proof-2019; receipt nằm trong proofValue.

Bitcoin regtest được chọn thay vì hợp đồng thông minh. Giá trị on-chain của quy trình phát hành chỉ là Merkle root; cơ chế thu hồi của nguyên mẫu dùng payload REVOKE:<certUid> trong OP_RETURN. Đây là mở rộng riêng của

nguyên mẫu, không được mô tả như cơ chế thu hồi chuẩn chung của Blockcerts. Workflow, Issuer Profile, Revocation List và trạng thái nghiệp vụ vẫn nằm ngoài chuỗi.

2.4.2. Kiến trúc thành phần



Hình 2.2. Kiến trúc thành phần

Hình 2.2 thể hiện kiến trúc thành phần và các luồng kết nối chính. Hệ thống gồm ba frontend React; Nginx reverse proxy; backend NestJS; PostgreSQL; Redis/BullMQ; worker; container cert-tools; container cert-issuer; verifier service; Bitcoin Core regtest. Các thành phần dữ liệu và RPC chỉ bind loopback. Cổng quản trị, Holder và Verify được tách để giảm nhầm lẫn quyền và cho phép triển khai độc lập.

NestJS cung cấp module, guard và validation [19]. PostgreSQL cung cấp transaction và ràng buộc dữ liệu [20]. TypeORM quản lý entity/migration [22]. BullMQ dùng Redis để chuyển tác vụ dài khỏi request HTTP [21]. Việc chọn công nghệ không có nghĩa framework tự bảo đảm an toàn; cấu hình, kiểm thử và quy trình vận hành vẫn quyết định kết quả.

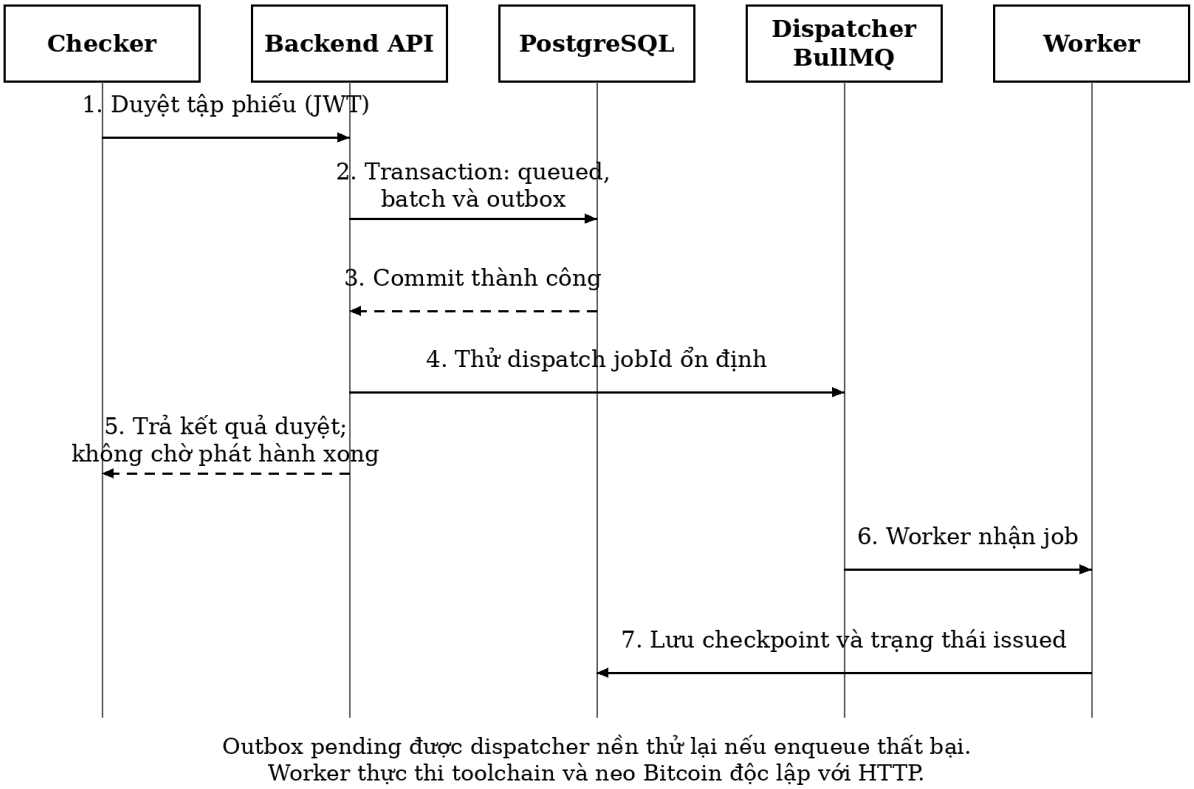
2.4.3. Luồng dữ liệu

Maker gửi studentCode, thông tin văn bằng và địa chỉ P2PKH regtest (trường pubkey). Backend lấy hồ sơ Student từ DB, lưu phiếu và chờ Checker duyệt; worker sau đó tạo roster và gọi toolchain. Chứng thư đã ký được lưu ngoài chuỗi; DB lưu certUid, txid, merkleRoot, trạng thái và actor. Bitcoin chỉ nhận root

phát hành. Student tải JSON qua Holder API. Verify portal gửi JSON tới backend; backend phối hợp verifier service, Bitcoin RPC và dữ liệu thu hồi để kết luận.

2.5. Thiết kế Maker–Checker và máy trạng thái

2.5.1. Luồng phát hành



Hình 2.3. Trình tự phát hành theo lô

1. Theo trình tự trên Hình 2.3, Maker đăng nhập và gửi một yêu cầu hoặc mảng tối đa 500 phần tử.

2. Backend lấy actor từ JWT, kiểm tra DTO và lưu pending_approval.

3. Checker xem danh sách, duyệt hoặc từ chối.

4. Khi duyệt, backend commit nguyên tử trạng thái queued, issuance_batches và issuance_outbox; dispatcher sau đó enqueue job với jobId ổn định.

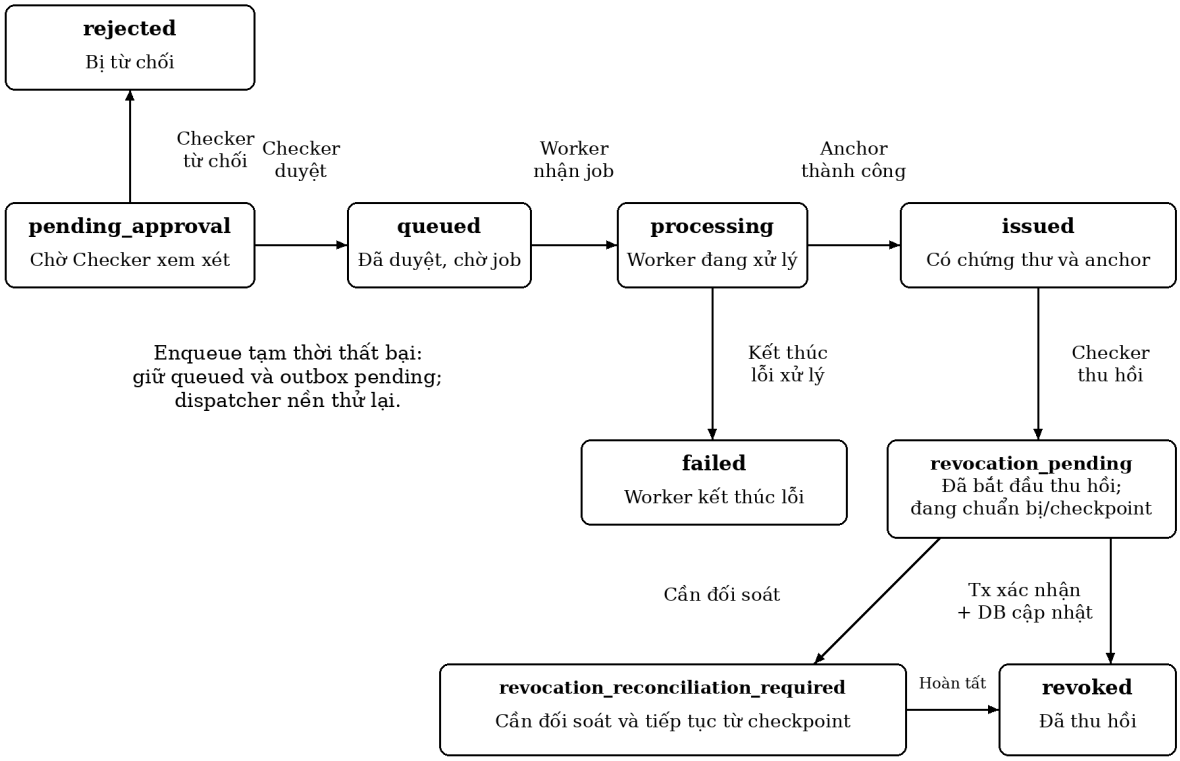
5. Worker nhận job theo issuanceBatchId, đặt processing và tạo chứng thư unsigned theo các chunk riêng.

6. Các output được gom để cert-issuer tạo một cây Merkle và một anchor.

7. Regtest tạo một block; worker giải mã proof, ánh xạ bằng nonce và cập nhật issued.

8. Holder tải chứng thư; bên thứ ba có thể Verify.

2.5.2. Trạng thái



Hình 2.4. Máy trạng thái yêu cầu phát hành

Theo Hình 2.4, luồng phát hành chính là pending_approval → queued → processing → issued; nhánh từ chối là pending_approval → rejected. Lỗi enqueue tạm thời được giữ trong outbox để dispatcher thử lại, không tự động đồng nghĩa với chứng thư failed. Worker kết thúc lỗi có thể chuyển sang failed hoặc trạng thái phục hồi của lô. Khi thu hồi bắt đầu, issued chuyển sang revocation_pending; nếu đã có checkpoint nhưng chưa hoàn tất thì cần revocation_reconciliation_required. Reconciler tiếp tục từ raw transaction/txid đã lưu; khi giao dịch được xác nhận và DB cập nhật thành công, trạng thái cuối là revoked. Bảng 2.3 giải thích các trạng thái này.

Bảng 2.3. Ý nghĩa trạng thái

Trạng thái	Ý nghĩa	Tác nhân chuyển
pending_approval	Chờ Checker xem xét	Maker tạo
rejected	Bị từ chối, có lý do	Checker
queued	Đã duyệt và chờ worker	Checker/service
processing	Toolchain đang xử lý	Worker
issued	Có chứng thư và anchor	Worker

Trạng thái	Ý nghĩa	Tác nhân chuyển
failed	Worker kết thúc lỗi	Worker
revoked	Chứng thư mất hiệu lực	Checker
revocation_pending	Đã bắt đầu thu hồi; có thể đang chuẩn bị/checkpoint giao dịch	Revocation service
revocation_reconciliation_required	Thu hồi đã có tác động/checkpoint nhưng cần đối soát và tiếp tục	Revocation service / reconciler

2.6. Thiết kế phát hành Blockcerts V3

2.6.1. Cấu trúc chứng thư

Chứng thư gồm @context, type, URI issuer, issuanceDate, id, credentialSubject, nội dung hiển thị và proof. type chứa VerifiableCredential và BlockcertsCredential; credentialSubject.id là URI định danh, không phải địa chỉ Bitcoin trần. Trường pubkey đầu vào chứa địa chỉ P2PKH regtest theo toolchain. Nonce dẫn xuất ổn định từ batchId và ID bản ghi, dùng ánh xạ roster với output; Holder đối chiếu studentId với sub của JWT.

DataIntegrityProof biểu diễn thông tin về thời điểm tạo, mục đích assertionMethod, verification method và bằng chứng theo mô hình Data Integrity [13]. Với toolchain Blockcerts của nguyên mẫu, cryptosuite là merkle-proof-2019; proofValue bao gói receipt Merkle gồm target hash, đường dẫn và anchor Bitcoin theo bộ suite tương ứng [11], [14]. Bằng chứng này được tạo bởi toolchain, không phải hợp đồng thông minh.

2.6.2. Gom lô và ánh xạ kết quả

Phương án chạy toàn bộ toolchain cho từng hồ sơ gây chi phí khởi động container và tăng transaction. Thiết kế cuối chia bước tạo unsigned certificate thành chunk 10 hồ sơ, tối đa 16 chunk đồng thời trên máy thử nghiệm. Mỗi chunk có workspace riêng; sau đó toàn bộ output được gom vào một thư mục để cert-issuer tạo một root và một transaction.

Tên người nhận có thể trùng; tên file do tool tạo không được dùng làm khóa. Mỗi dòng roster có nonce ổn định, lấy 32 ký tự hex đầu của SHA-256("datn-blockcerts:" + batchId + ":" + ID bản ghi). Worker đọc nonce từ output để ánh xạ DB. requestBatchId truy vết nhóm phiếu; issuanceBatchId định danh lô; certUid

định danh chứng thư. Nếu mở giới hạn trên 500, cần chia thành nhiều anchor job thay vì tạo lô vượt phạm vi đã kiểm thử.

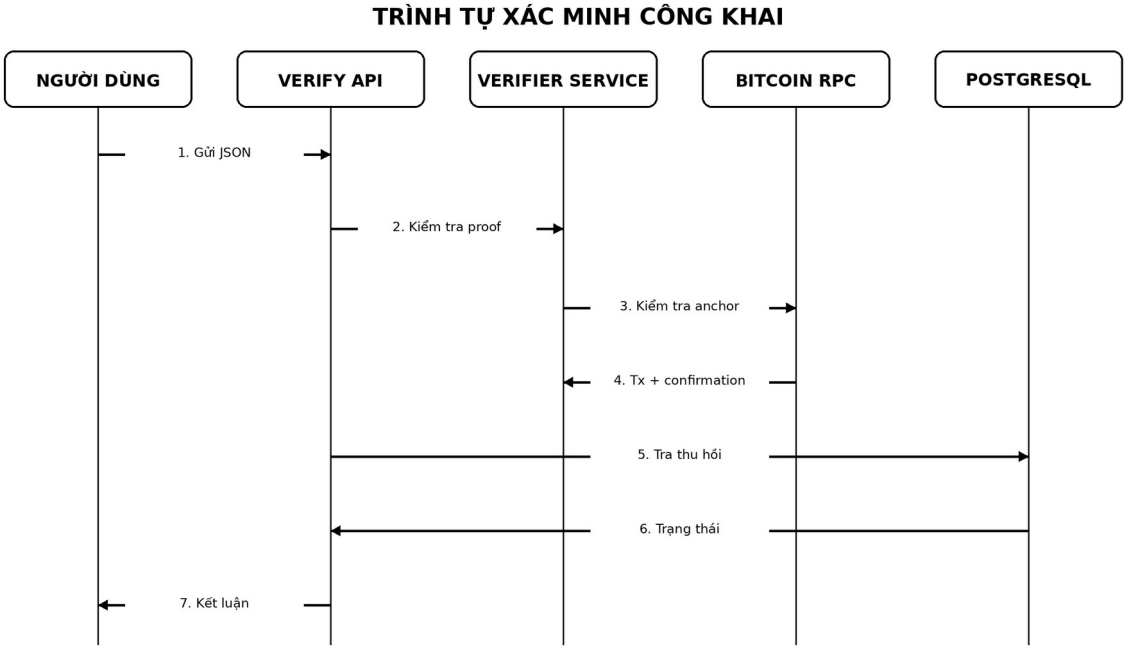
2.6.3. Tách nhóm yêu cầu và lô phát hành

Phiên bản hiện hành tách requestBatchId và issuanceBatchId vì hai định danh có vòng đời khác nhau. requestBatchId được sinh khi Maker tạo nhóm phiếu và giữ ổn định để truy vết nguồn nghiệp vụ. issuanceBatchId chỉ sinh khi Checker duyệt một tập phiếu cụ thể; mỗi lần duyệt từng phần tạo một giá trị mới. Do đó một nhóm phiếu có thể liên kết với nhiều lô phát hành mà không tái sử dụng khóa chính hoặc kéo nhầm phiếu vào lần chạy lại.

Khi duyệt, dịch vụ khóa các phiếu đang chờ, cập nhật queued, tạo issuance_batches và issuance_outbox trong cùng transaction PostgreSQL. Outbox sau đó được dispatcher chuyển sang BullMQ bằng jobId ổn định; nếu enqueue thất bại, bản ghi outbox vẫn ở pending để tiến trình nền thử lại. Đây là ranh giới quan trọng giữa tính nguyên tử trong DB và tác động ngoài DB/Redis.

2.7. Thiết kế xác minh và thu hồi

2.7.1. Xác minh



Hình 2.5. Trình tự xác minh công khai

Theo Hình 2.5, Verify API tiếp nhận toàn bộ JSON. Backend kiểm tra semantic tối thiểu, sau đó gửi chứng thư đến verifier service. Verifier giải mã proofValue bằng LDMerkleProof2019, kiểm tra cấu trúc/chứng thư; adapter regtest lấy transaction theo txid, kiểm tra đầu vào gắn với issuer, tìm OP_RETURN, so Merkle root và yêu cầu ít nhất một confirmation. Nếu lớp mật mã/anchor đạt,

backend lấy certUid để tra thu hồi. Kết luận gồm VALID, INVALID, REVOKED hoặc INDETERMINATE.

INVALID dùng khi nội dung không khớp bằng chứng; REVOKED dùng khi chứng thư vốn hợp lệ nhưng đã bị thu hồi; INDETERMINATE dùng khi không đủ bằng chứng để kết luận, ví dụ transaction/proof không thể kiểm tra. Phân biệt này tránh biến lỗi kết nối thành tuyên bố chứng thư giả.

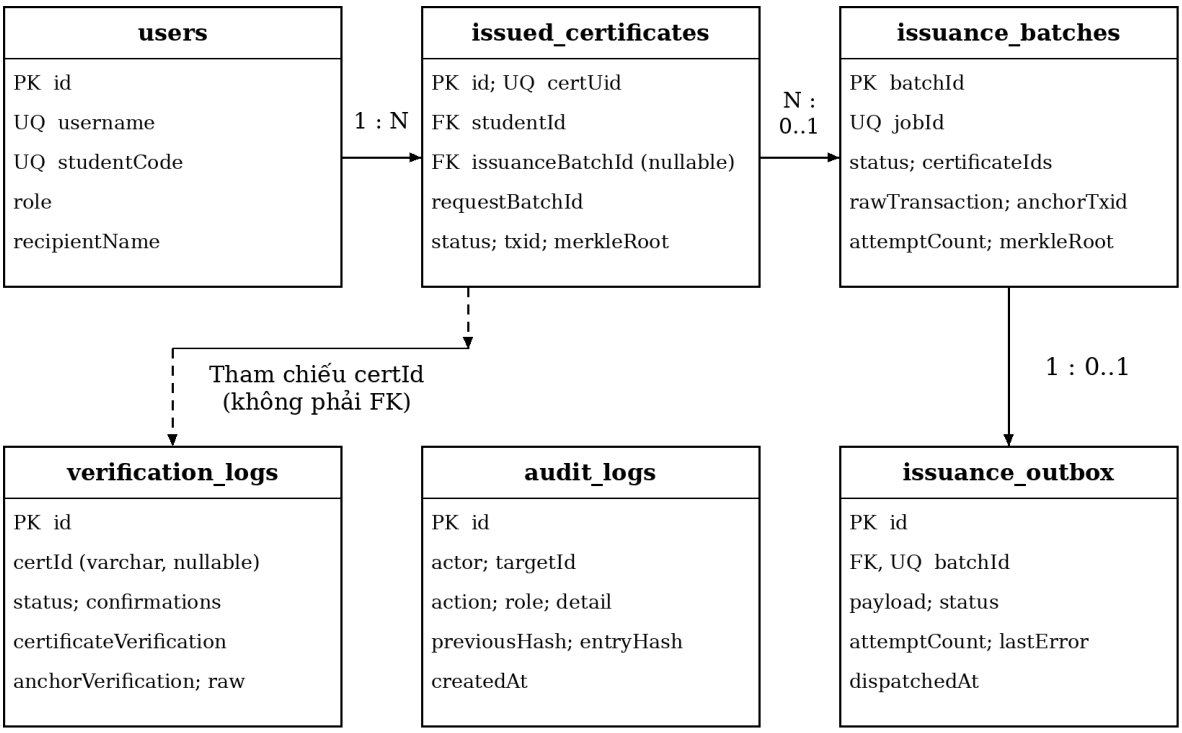
2.7.2. Thu hồi

Checker chỉ có thể khởi tạo thu hồi đối với chứng thư ở trạng thái cho phép. Service dùng PostgreSQL advisory lock để tuần tự hóa việc tiêu UTXO, chuyển bản ghi sang revocation_pending, tạo và ký đúng một raw transaction chứa payload REVOKE:<certUid>, tính trước txid dự kiến và lưu revocationRawTransaction, revocationIntentTxid, revocationPreparedAt vào DB trước khi gọi sendrawtransaction. Checkpoint bền vững trước broadcast bảo đảm lần retry không tạo một giao dịch thay thế.

Sau broadcast, service luôn tra lại đúng txid đã checkpoint; nếu transaction chưa có trên node thì chỉ phát lại raw transaction cũ. Khi có ít nhất một confirmation, DB mới chuyển sang revoked và lưu revocationTxid, actor, lý do và thời gian. Nếu lỗi xảy ra sau khi tiến trình thu hồi đã bắt đầu, trạng thái được giữ ở revocation_pending hoặc revocation_reconciliation_required; reconciler định kỳ tiếp tục từ checkpoint. Revocation List hiện đọc trạng thái từ DB, vì vậy môi trường vận hành thực tế vẫn cần backup/restore và indexer để tái dựng trạng thái từ blockchain.

2.8. Thiết kế dữ liệu ngoài chuỗi

Hình 2.6 khái quát quan hệ giữa tài khoản, phiếu/chứng thư, lô phát hành, outbox, nhật ký xác minh và audit trong mô hình dữ liệu hiện hành.



PK: khóa chính UQ: ràng buộc duy nhất FK: khóa ngoại trong migration

actor và targetId là tham chiếu nghiệp vụ; certId có thể thuộc chứng thư ngoài hệ thống.

Hình 2.6. Mô hình dữ liệu

Hình 2.6 phân biệt khóa ngoại trong migration với tham chiếu logic. issuance_outbox.batchId đồng thời là FK và unique nên mỗi batch có tối đa một outbox; số lần thử được lưu trong attemptCount. verification_logs.certId là chuỗi nullable, có thể lấy từ chứng thư ngoài hệ thống, không phải FK tới issued_certificates.id. actor/targetId của audit là tham chiếu nghiệp vụ và cũng không được mô tả như khóa ngoại cưỡng chế.

2.8.1. Bảng users

Bảng lưu id, username duy nhất, passwordHash, role, studentCode, recipientName, ngày sinh, email, niên khóa và createdAt theo phạm vi hồ sơ Student. studentCode là định danh nghiệp vụ duy nhất; chứng thư liên kết tới Student bằng khóa ngoại studentId. recipientName chỉ là ảnh chụp dữ liệu hiển thị tại thời điểm lập phiếu, không được dùng để quyết định quyền sở hữu.

2.8.2. Bảng issued_certificates

Mỗi dòng vừa là yêu cầu nghiệp vụ vừa là chứng thư sau phát hành. Nhóm đầu vào gồm dữ liệu văn bằng, studentId và requestBatchId; nhóm blockchain gồm certUid, txid, merkleRoot, issuanceBatchId; nhóm Maker–Checker gồm requestedBy, approvedBy, rejectedBy, rejectReason; nhóm trạng thái gồm status,

errorMessage; nhóm thu hồi gồm revocationTxid, raw transaction/intent txid checkpoint, số lần thử, lỗi, người thực hiện, lý do và thời điểm.

Thiết kế một bảng giúp nguyên mẫu đơn giản hơn nhưng trộn vòng đời yêu cầu với tệp kết quả phát hành. Hệ thống hiện đã dùng khóa ngoại Student, tách nhóm yêu cầu khỏi lô phát hành và có bảng lô/outbox riêng; khi mở rộng môi trường vận hành thực tế vẫn nên cân nhắc tách issuance_requests, certificates và revocation_requests để giảm coupling và đơn giản hóa chính sách lưu giữ dữ liệu.

2.8.3. verification_logs và audit_logs

verification_logs lưu định danh kỹ thuật, trạng thái và kết quả từng lớp kiểm tra; dữ liệu thử nghiệm xác nhận log không chứa họ tên, email hoặc mã sinh viên của credential. audit_logs lưu action, actor, role, target, detail, txid và thời gian; mỗi dòng có previousHash và entryHash tạo chuỗi hash SHA-256 để phát hiện sửa/xóa ở giữa chuỗi. Hash chain chỉ tạo tính tamper-evident: DBA vẫn có thể viết lại toàn bộ chuỗi hoặc xóa đuôi nếu head hash không được neo/đối chiếu với kho độc lập. Ngoài ra, audit hiện không luôn commit cùng transaction với nghiệp vụ chính, nên cần cảnh báo/giám sát để tránh mất log âm thầm khi triển khai thực tế.

2.8.4. Bảng issuance_batches

Bảng issuance_batches lưu batchId, jobId, danh sách certificate ID, trạng thái lô, raw transaction/anchor txid, Merkle root, số lần thử và các mốc thời gian. Máy trạng thái của lô gồm pending_enqueue, queued, processing, anchor_prepared, reconciliation_required, completed và failed. Bảng này là checkpoint bền vững cho tiến trình worker và recovery.

2.8.5. Bảng issuance_outbox

Bảng issuance_outbox lưu một payload enqueue cho mỗi batchId, trạng thái pending/dispatched, số lần thử, lỗi gần nhất và thời điểm dispatch. Bản ghi outbox được tạo cùng transaction khi Checker duyệt phiếu; dispatcher nền mới thực hiện tác động sang Redis/BullMQ. Cách tổ chức này tránh tình trạng DB đã queued nhưng job bị mất nếu tiến trình lỗi đúng giữa commit DB và thao tác enqueue. Các thực thể dữ liệu chính và vai trò của chúng được tổng hợp tại Bảng 2.4.

Bảng 2.4. Mô hình dữ liệu chính

Bảng	Khóa/định danh	Chức năng
users	UUID, username, studentCode	Tài khoản, vai trò và hồ sơ Student

Bảng	Khóa/định danh	Chức năng
issued_certificates	UUID, certUid, requestBatchId, issuanceBatchId	Vòng đời phiếu/chứng thư và checkpoint thu hồi
issuance_batches	batchId, jobId	Trạng thái lô, checkpoint worker và anchor
issuance_outbox	UUID, batchId	Cơ chế outbox giao dịch để đưa tác vụ vào BullMQ bền vững
verification_logs	UUID, certId	Kết quả xác minh đã tối thiểu hóa dữ liệu
audit_logs	UUID, targetId, entryHash	Nhật ký truy trách nhiệm và hash chain

2.9. Thiết kế API và phân quyền

Bảng 2.5. API chính và quyền

Phương thức	Endpoint	Quyền	Chức năng
POST	/api/auth/login	Công khai	Đăng nhập và nhận JWT
POST	/api/auth/register	Checker	Tạo tài khoản qua AuthController
GET	/api/admin/users	Checker	Liệt kê tài khoản
POST	/api/admin/users	Checker	Tạo tài khoản qua cổng quản trị
PATCH	/api/admin/users/:id/student-profile	Checker	Cập nhật hồ sơ Student cho phép
GET	/api/audit	Maker, Checker	Đọc audit theo phạm vi vai trò/đối tượng
GET	/api/audit/integrity	Maker, Checker	Kiểm tra chuỗi hash audit
POST	/api/issue/request	Maker	Lập một yêu cầu phát hành
POST	/api/issue/batch/request	Maker	Lập lô từ 1 đến 500 yêu cầu

Phương thức	Endpoint	Quyền	Chức năng
GET	/api/issue/ students/:studentCode	Maker	Tra cứu hồ sơ Student chỉ đọc
POST	/api/issue/:id/approve	Checker	Duyệt một yêu cầu
POST	/api/issue/batch/approve	Checker	Duyệt một tập phiếu
POST	/api/issue/batch/:batchId/ retry	Checker	Retry lô đã thất bại
POST	/api/issue/:id/reject	Checker	Từ chối yêu cầu
GET	/api/issue	Maker, Checker	Danh sách theo phạm vi vai trò
GET	/api/issue/:id	Maker, Checker	Chi tiết theo phạm vi vai trò
GET	/api/issue/holder/certificates	Student	Danh sách chứng thư của subject hiện tại
POST	/api/revoke/check	Công khai	Kiểm tra trạng thái thu hồi theo certUid
POST	/api/revoke/:id	Checker	Thu hồi chứng thư đã phát hành
POST	/api/verify	Công khai	Xác minh JSON Blockcerts
GET	/api/blockcerts/issuers/kma/ profile.json	Công khai	Issuer Profile
GET	/api/blockcerts/issuers/kma/ revocation-list.json	Công khai	Revocation List
GET	/health	Công khai	Liveness tối thiểu của backend

Bảng 2.5 tổng hợp bề mặt API chính và quyền tối thiểu theo mã nguồn hiện hành; quyền chi tiết vẫn được kiểm tra tại backend.

Route tĩnh batch/request, batch/approve, holder/certificates và revoke/check phải đứng trước route tham số :id. ValidationPipe loại hoặc từ chối trường ngoài DTO. Actor của approve/revoke lấy từ JWT [15], không lấy từ body. Với endpoint

list/chi tiết/audit, Checker có phạm vi quản trị rộng hơn; Maker bị lọc theo requestedBy và không thể đọc trực tiếp yêu cầu hoặc audit của Maker khác. Endpoint /health chỉ cung cấp liveness tối thiểu của backend; kiểm tra PostgreSQL, Redis, Bitcoin, Nginx và số migration thuộc script readiness của quy trình triển khai.

2.10. Thiết kế ba cổng giao diện

Cổng quản trị phục vụ Maker và Checker. Maker có màn hình lập phiếu đơn/lô, danh sách và audit; Checker có màn hình duyệt, văn bằng, thu hồi, người dùng và audit. Menu thay đổi theo role nhưng backend vẫn kiểm tra quyền. Cổng Holder chỉ nhận tài khoản Student, hiển thị cả chứng thư còn hiệu lực và đã thu hồi để giữ lịch sử. Cổng Verify không yêu cầu tài khoản, nhận file/dữ liệu JSON và hiển thị trạng thái cùng chi tiết kiểm tra.

Ba frontend dùng chung API nhưng build độc lập. Khi triển khai sau Nginx, mỗi ứng dụng dùng base path riêng. Giao diện không tự tính hợp lệ ở client; nó gửi chứng thư tới backend để logic nhất quán và tránh phụ thuộc khả năng trình duyệt.

2.11. Thiết kế an toàn và triển khai

Mật khẩu được băm bằng bcrypt; JWT có thời hạn và được kiểm tra tại backend [15], [17]. AuthGuard chạy trước RolesGuard tại route bảo vệ. Quyền không chỉ kiểm tra ở mức role mà còn ở mức đối tượng: Maker chỉ xem phiếu/audit thuộc phạm vi của mình, Student chỉ xem chứng thư liên kết với subject JWT, còn Checker thực hiện các chức năng quản trị và thu hồi. Frontend chỉ ẩn/hiện chức năng để hỗ trợ trải nghiệm, không phải ranh giới an toàn.

Helmet bổ sung security header; throttling giới hạn tần suất; CORS dùng allowlist; body có giới hạn; PostgreSQL, Redis và Bitcoin RPC chỉ bind loopback. Dịch vụ xác minh áp dụng safe-fetch để kiểm tra URL, giao thức, địa chỉ loopback/private/link-local/metadata, DNS và chuyển hướng trước khi tải tài nguyên ngoài; lỗi hạ tầng được phân loại INDETERMINATE thay vì tự động kết luận INVALID [23].

Thiết kế pilot vẫn dùng HTTP trong mạng riêng. HTTPS/HSTS, secret manager, HSM/KMS, MFA, luân chuyển khóa, backup/restore, HA, giám sát tập trung và kiểm thử xâm nhập độc lập nằm ngoài phạm vi nguyên mẫu nhưng là điều kiện trước môi trường vận hành thực tế. Đối với dữ liệu cá nhân, triển khai thực tế phải xác định mục đích xử lý, phạm vi công bố, thời hạn lưu giữ và biện pháp bảo vệ phù hợp Luật 91/2025/QH15 và Nghị định 356/2025/NĐ-CP [26], [27].

2.11.1. Ranh giới tin cậy và dữ liệu nhạy cảm

Trình duyệt được xem là vùng không tin cậy; backend và worker là vùng xử lý đặc quyền; PostgreSQL là nguồn trạng thái nghiệp vụ; Redis là cơ chế điều phối; Bitcoin RPC và kho cấu hình Blockcerts thuộc vùng quản trị. JWT secret, mật khẩu cơ sở dữ liệu, RPC credential, WIF và token thử nghiệm chỉ tồn tại trong cấu hình runtime ngoài Git. Báo cáo/bằng chứng không công bố secret và chỉ giữ dữ liệu kỹ thuật cần cho tái lập.

2.11.2. Bảo vệ trạng thái, outbox và phục hồi

Các thao tác duyệt/từ chối dùng transaction hoặc cập nhật có điều kiện để tránh cạnh tranh. Khi duyệt, trạng thái phiếu, issuance_batches và issuance_outbox được commit cùng transaction; enqueue sang BullMQ được thực hiện sau đó và có dispatcher thử lại. Worker dùng jobId, checkpoint raw transaction/txid và trạng thái lô để tránh tạo anchor thứ hai khi retry. Thu hồi áp dụng nguyên tắc tương tự: checkpoint bền vững trước broadcast và reconciler chỉ tiếp tục giao dịch đã chuẩn bị.

2.11.3. Audit và giới hạn tính toàn vẹn

Audit hash chain giúp phát hiện sửa hoặc xóa bản ghi ở giữa chuỗi khi thực hiện kiểm tra toàn vẹn. Tuy nhiên, cơ chế này không biến PostgreSQL thành kho log bất biến: quản trị viên đủ quyền vẫn có thể viết lại cả chuỗi hoặc xóa phần đuôi nếu head hash không được neo ra hệ thống độc lập. Ngoài ra, một số audit được ghi sau transaction nghiệp vụ nên chưa bảo đảm atomicity tuyệt đối giữa sự kiện và log; môi trường vận hành thực tế cần append-only logging hoặc hệ thống giám sát độc lập.

2.12. Tiêu chí chấp nhận và kế hoạch kiểm thử

Bảng 2.6. Tiêu chí chấp nhận

Mã	Tiêu chí	Bằng chứng dự kiến
AC-01	Chứng thư gốc VALID	Kịch bản 1
AC-02	Sửa nội dung/proof/txid không còn VALID	Kịch bản 1
AC-03	Maker tự duyệt bị 403	Kịch bản 1
AC-04	10, 100, 500 chứng thư/lô đều phát hành đủ	Kịch bản 2
AC-05	Mỗi lô dùng đúng một giao dịch	Kiểm toán tệp kết quả
AC-06	Student/Maker không thu hồi được	Kịch bản 3

Mã	Tiêu chí	Bằng chứng dự kiến
AC-07	Checker thu hồi và Verify trả REVOKED	Kịch bản 3
AC-08	Revocation List và audit nhất quán	Kịch bản 3
AC-09	Build, kiểm thử và readiness/dependency check đạt	Lượt tái kiểm chứng 30/09/2026; Bảng 3.6
AC-10	Duyệt từng phần cùng requestBatchId tạo các issuanceBatchId khác nhau	Functional E2E
AC-11	Lỗi hoặc khởi động lại sau khi quảng bá không tạo giao dịch thứ hai	Recovery E2E
AC-12	Maker không đọc phiếu/audit của Maker khác; Student không đọc chứng thư của Student khác	Functional E2E

Bảng 2.6 xác định các tiêu chí chấp nhận dùng để đối chiếu với bằng chứng ở Chương 3.

2.13. Kết luận Chương 2

Chương 2 đã chuyển cơ sở lý thuyết thành thiết kế bám sát mã nguồn hiện hành: ba vai trò xác thực và một tác nhân công khai; ba cổng web; Maker–Checker; phân quyền mức role và mức đối tượng; tách requestBatchId/issuanceBatchId; cơ chế outbox giao dịch; checkpoint/idempotency cho phát hành và thu hồi; xác minh bốn trạng thái; safe-fetch chống SSRF; audit hash chain và sáu nhóm dữ liệu chính. Thiết kế chỉ neo bằng chứng tối thiểu lên Bitcoin regtest, không dùng hợp đồng thông minh và không đồng nhất chứng thư Blockcerts của nguyên mẫu với văn bằng số có giá trị pháp lý. Các giới hạn về quản trị khóa, audit atomicity, indexer thu hồi, bảo vệ dữ liệu cá nhân và môi trường regtest được giữ rõ để Chương 3 đánh giá đúng phạm vi.

CHƯƠNG III. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Môi trường và mục tiêu tái lập

3.1.1. Môi trường thực nghiệm và mốc mã nguồn

Đợt tái kiểm chứng cuối được thực hiện ngày 30/09/2026 trên Ubuntu Server 22.04.5 LTS, 4 vCPU Intel Xeon Gold 6130 và khoảng 19 GiB RAM. Bitcoin chạy ở chế độ regtest để chủ động tạo block mà không sử dụng tiền thật [18]. Mọi kết quả định lượng trong chương này được gắn với commit 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1, trạng thái sourceDirty=false và dấu vân tay của cây mã nguồn 6e0cd862daf5641b26995575d8e7ff9d755daf43396191dfc838fc6ffb0997cc. Việc ghi đồng thời commit, trạng thái cây nguồn và fingerprint nhằm tránh trường hợp mã chưa commit làm thay đổi kết quả nhưng không xuất hiện trong định danh phiên bản.

Mục tiêu của chương không chỉ mô tả các thành phần đã xây dựng, mà tái hiện đường đi từ một máy Ubuntu mới đến hệ thống có thể sử dụng qua Web, sau đó kiểm thử phát hành, xác minh, thu hồi, khả năng phục hồi và hiệu năng. Vì vậy, phần triển khai được trình bày theo đúng thứ tự phụ thuộc thay vì tách rời thành các mô tả mô-đun. Các thành phần phần mềm và vai trò của chúng được tổng hợp tại Bảng 3.1.

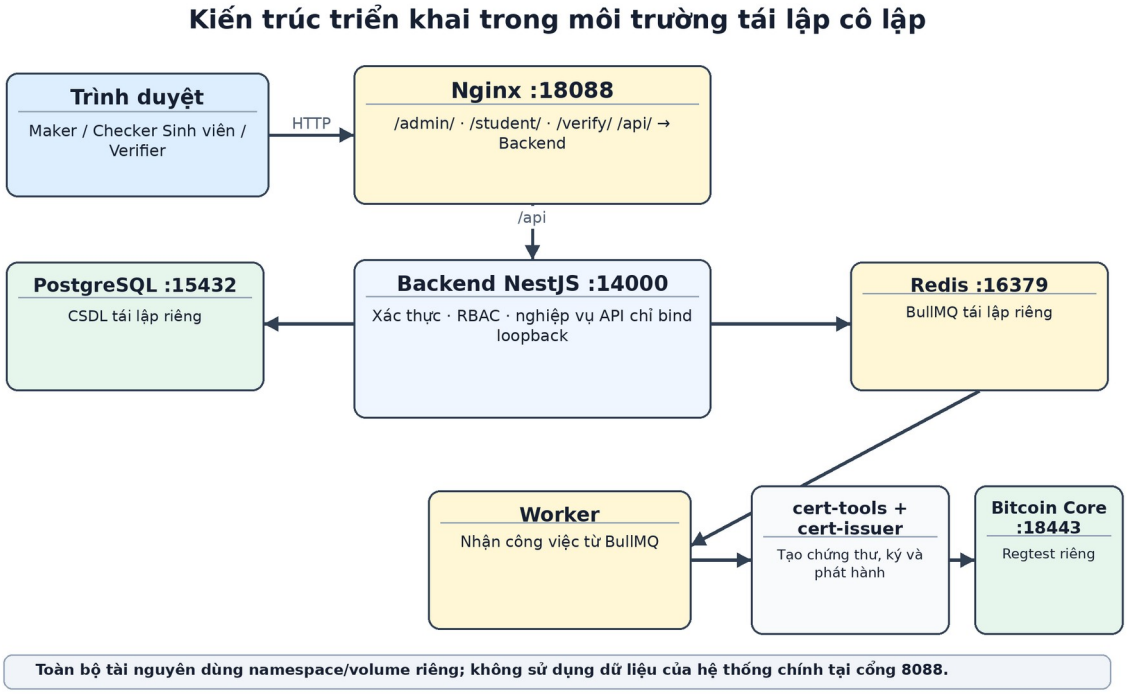
Bảng 3.1. Thành phần phần mềm của lượt tái kiểm chứng

Thành phần	Phiên bản/dòng phiên bản	Vai trò
Node.js	22 trở lên (kịch bản tái lập không cố định phiên bản vá)	Runtime backend và build frontend
NestJS	11	REST API, JWT, RBAC và nghiệp vụ
PostgreSQL	17-bookworm	Dữ liệu nghiệp vụ, trạng thái và log
Redis	7.4-bookworm	Nền lưu trữ cho BullMQ
BullMQ	6 (^6.3.1)	Hàng đợi phát hành và retry
Bitcoin Core	31.1	Sổ cái regtest và RPC
cert-tools	3.0.2	Sinh chứng thư unsigned từ roster

Thành phần	Phiên bản/dòng phiên bản	Vai trò
cert-issuer	3.13.1	Tạo Merkle proof và neo Bitcoin
React/Vite	19/7	Ba cổng giao diện Web
Nginx	Bản hệ thống Ubuntu	Reverse proxy và base path

3.1.2. Kiến trúc triển khai

Hình 3.1 cho thấy ba cổng giao diện tách theo nhóm sử dụng nhưng cùng đi qua backend NestJS. PostgreSQL là nguồn trạng thái nghiệp vụ; Redis/BullMQ điều phối công việc bất đồng bộ; cert-tools và cert-issuer tạo tệp kết quả Blockcerts; Bitcoin Core giữ anchor regtest. Cách phân tách này giúp việc kiểm tra quyền diễn ra tại backend thay vì phụ thuộc vào việc ẩn hay hiện nút trên giao diện.



Hình 3.1. Kiến trúc triển khai của hệ thống trong môi trường thực nghiệm

3.2. Tái lập hệ thống từ máy Ubuntu mới

Quy trình tái lập được tổ chức thành các cổng kiểm soát. Chỉ chuyển sang bước sau khi bước hiện tại đạt điều kiện chấp nhận. Ở mỗi bước, báo cáo trình bày mục tiêu, câu lệnh cần thực hiện và kết quả cần đối chiếu ngay sau khi chạy; Phụ lục 3 chỉ tổng hợp nhanh các lệnh chính để thuận tiện tra cứu. Chuỗi phụ thuộc từ môi trường mới đến bộ bằng chứng được minh họa trên Hình 3.2.

Quy ước trình bày trong mục này: mỗi bước gồm mục đích, câu lệnh cần chạy và kết quả dùng để đối chiếu. Dấu $ chỉ biểu diễn dấu nhắc của terminal,

người thực hiện không gõ ký tự này. Các giá trị mật khẩu, JWT secret, Bitcoin RPC credential và WIF được sinh riêng trong runtime và không được chép vào báo cáo. Các khung lệnh được trình bày theo cùng quy cách với mẫu triển khai thực nghiệm: lệnh đặt trong khung đơn, sau đó là phần kết quả kiểm tra.



Hình 3.2. Chuỗi tái lập từ Ubuntu mới đến bộ bằng chứng thực nghiệm

3.2.1. Chuẩn bị công cụ và kiểm tra tài nguyên

Máy mới cần Git, Node.js/npm, Docker Engine, Docker Compose và Nginx. Trước khi dựng stack phải kiểm tra Docker daemon, CPU, RAM, dung lượng đĩa và các cổng dự kiến. Nếu Docker chưa hoạt động hoặc cổng đang bị stack khác chiếm, quy trình dừng tại đây; không đổi cổng tùy ý vì Issuer Profile, CORS và URL công khai có liên hệ với cấu hình.

Bước 1 - Cập nhật Ubuntu và cài các công cụ nền

Tiến hành chạy các lệnh dưới đây:

```
$ sudo apt update
$ sudo apt install -y git curl ca-certificates gnupg nginx python3
python3-pip openssl jq
```

Kết quả mong đợi khi chạy thành công:

```
Các gói được cài đặt hoặc xác nhận đang ở phiên bản sẵn có trong Ubuntu.
Lệnh kết thúc với mã 0; không xuất hiện lỗi "command not found".
```

Bước 2 - Cài Docker Engine và Docker Compose plugin

Tiến hành chạy lần lượt các lệnh sau:

```
$ sudo install -m 0755 -d /etc/apt/keyrings
$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
$ sudo chmod a+r /etc/apt/keyrings/docker.asc
$ echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $
{VERSION_CODENAME}) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
$ sudo apt update
$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
$ sudo usermod -aG docker "$USER"
```

Sau khi thêm tài khoản vào nhóm docker, đăng xuất/đăng nhập lại phiên shell (hoặc mở terminal mới) để quyền nhóm có hiệu lực. Sau đó kiểm tra:

```
$ docker --version
$ docker compose version
$ docker info >/dev/null && echo DOCKER_OK
```

Kết quả mong đợi khi chạy thành công:

```
Docker version ...
Docker Compose version v2... hoặc mới hơn
DOCKER_OK
```

Bước 3 - Cài Node.js và kiểm tra các công cụ

Bộ kịch bản tái lập yêu cầu Node.js/npm khả dụng nhưng không khóa cứng bản runtime. Trên Ubuntu 22.04 có thể cài Node.js 22 LTS để tái lập ổn định; nếu máy thực nghiệm dùng một bản tương thích mới hơn thì phải ghi lại chính xác kết quả node --version trong bằng chứng. Tiến hành chạy:

```
$ curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
$ sudo apt install -y nodejs
$ node --version
$ npm --version
$ nginx -v
$ python3 --version
```

Kết quả mong đợi khi chạy thành công:

```
v22.x.x hoặc bản tương thích mới hơn đã được ghi nhận
npm tương thích với bản Node.js đang sử dụng
nginx version: nginx/...
Python 3.x.x
```

Bước 4 - Kiểm tra tài nguyên máy

Tiến hành chạy các lệnh dưới đây để ghi nhận tài nguyên trước khi thực nghiệm:

```
$ uname -a
$ nproc
$ free -h
$ df -h /
```

Kết quả của lượt tái kiểm chứng chính thức:

```
Máy thực nghiệm chính thức: Ubuntu Server 22.04.5 LTS.
CPU khả dụng: 4 vCPU.
RAM khả dụng xấp xỉ 19 GiB.
Phân vùng hệ thống đủ dung lượng cho ảnh dựng, vùng lưu trữ và tệp kết
quả.
```

3.2.2. Lấy mã nguồn và cố định phiên bản

Repository được checkout đúng commit của lượt tái kiểm chứng. Sau checkout, git status --short phải không có thay đổi ở mã nguồn nghiệp vụ. Script source-manifest.py tính dấu vân tay trên tập tệp được kiểm soát; tệp kết quả sinh sau khi chạy không được đưa vào fingerprint nguồn.

Bước 1 - Tải repository về máy mới

Tiến hành chạy:

```
$ cd ~
$ git clone https://github.com/ebkk24/DATN-AT190226.git
$ cd DATN-AT190226
$ git fetch --all --tags
```

Kết quả mong đợi khi chạy thành công:

```
Repository DATN-AT190226 được tạo trên máy và có thể truy cập các thư
mục backend, frontend-* và experiments/repro.
```

Bước 2 - Checkout đúng mốc mã nguồn dùng cho lượt tái kiểm chứng

Tiến hành chạy:

```
$ git checkout 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1
$ git rev-parse HEAD
$ git status --short
```

Kết quả mong đợi khi chạy thành công:

```
2816de8aac13492b614dbcc6b3b3e1f3eba52bc1
(git status --short không in thêm dòng nào khi working tree sạch)
```

Nếu git status --short hiển thị tệp sửa hoặc tệp chưa theo dõi thì không dùng lượt đó làm bằng chứng chính thức; phải làm sạch cây nguồn hoặc tạo fingerprint mới và ghi nhận rõ trong bằng chứng.

3.2.3. Tạo cấu hình runtime và bảo vệ bí mật

Tệp cấu hình mẫu được sao chép thành cấu hình runtime riêng. Mật khẩu PostgreSQL/Redis, JWT secret, Bitcoin RPC credential và WIF phát hành không được đưa vào Git hoặc ảnh chụp báo cáo. Tệp runtime đặt quyền đọc hạn chế; WIF chỉ cấp cho thành phần cần ký. Issuer Profile chỉ công bố khóa công khai và endpoint cần thiết. Các nhóm cấu hình runtime và nguyên tắc bảo vệ được tổng hợp tại Bảng 3.2.

Bước 1 - Kiểm tra namespace tái lập và quyền thực thi script

Tiến hành chạy:

```
$ chmod +x experiments/repro/scripts/*.sh
$ test ! -e experiments/repro/runtime/.env && echo RUNTIME_CLEAN || echo
RUNTIME_EXISTS
$ grep -n "datn-blockcerts-repro" experiments/repro/scripts/prepare.sh |
head
```

Kết quả mong đợi khi chạy thành công:

```
RUNTIME_CLEAN (đối với máy mới).
Các dòng cấu hình chỉ tới namespace datn-blockcerts-repro.
```

Bước 2 - Không tự ghi bí mật vào mã nguồn

Bộ prepare.sh tự sinh mật khẩu PostgreSQL/Redis, Bitcoin RPC credential và JWT secret bằng OpenSSL; khóa phát hành regtest cũng được sinh trong runtime. Vì vậy người thực hiện không cần sửa .env.example bằng bí mật thật. Sau bước prepare, chỉ kiểm tra quyền tệp, không sao chép giá trị bí mật vào báo cáo.

Bảng 3.2. Nhóm cấu hình runtime và nguyên tắc bảo vệ

Nhóm	Ví dụ	Yêu cầu
Cơ sở dữ liệu	POSTGRES_PASSWORD, DATABASE_URL	Sinh riêng cho môi trường; không công bố
Redis	REDIS_PASSWORD, REDIS_URL	Không ghi vào tệp kết quả công khai
JWT	JWT_SECRET, JWT_EXPIRES_IN	Secret sinh ngẫu nhiên; thời hạn token cấu hình riêng; không công bố secret
Bitcoin RPC	BITCOIN_RPC_USER/PASSWORD	Chỉ dùng trong mạng nội bộ
Blockcerts	Issuer URL, revocation URL, WIF	URL nhất quán; WIF tuyệt đối bí mật

Nhóm	Ví dụ	Yêu cầu
Frontend	API base URL	Không chứa credential

3.2.4. Dựng PostgreSQL, Redis và Bitcoin regtest

Bộ experiments/repro dựng một stack tách biệt nhằm tránh phụ thuộc trạng thái của môi trường phát triển trước đó. PostgreSQL, Redis và Bitcoin Core phải đạt health check trước khi backend chạy. Đây là readiness ở mức triển khai, khác với endpoint /health của backend vốn chỉ là liveness và trả trạng thái tiến trình ứng dụng.

Bước 1 - Chuẩn bị toàn bộ stack tái lập từ đầu

Trên máy mới, tiến hành chạy lệnh sau. Tùy CPU, tốc độ mạng và cache image, bước này có thể mất vài phút:

```
$ cd ~/DATN-AT190226
$ bash experiments/repro/scripts/prepare.sh --reset
```

Kết quả mong đợi khi chạy thành công:

```
[1/8] Cài phụ thuộc, kiểm thử biên dịch và build từ mã nguồn hiện có
[2/8] Tạo cây dự án runtime tách biệt
[3/8] Khởi động PostgreSQL, Redis và Bitcoin regtest của stack riêng
[4/8] Tạo khóa phát hành dùng riêng cho regtest và cấp UTXO
[5/8] Sinh cấu hình Blockcerts cho URL và địa chỉ của stack kiểm thử
[6/8] Chạy 10 migration trên cơ sở dữ liệu kiểm thử
[7/8] Tạo cấu hình Nginx cổng 18088
[8/8] Kiểm tra phạm vi cách ly
PREPARE_PASS stack=datn-blockcerts-repro api=14000 web=18088
key_mode=600
```

Dòng PREPARE_PASS là cổng kiểm soát của bước chuẩn bị. Nếu script dừng ở một bước trước đó thì không tiếp tục start.sh; phải xử lý lỗi rồi chạy lại prepare.sh.

Bước 2 - Kiểm tra quyền của tệp bí mật

Tiến hành chạy:

```
$ stat -c '%a %n' experiments/repro/runtime/.env
$ stat -c '%a %n'
experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Kết quả mong đợi khi chạy thành công:

```
600 experiments/repro/runtime/.env
600 experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Trong lượt tái kiểm chứng, health.sh còn kiểm tra backend, Nginx, ba cổng web, Issuer Profile và số migration trong PostgreSQL. Điều kiện đạt là ba container healthy, các endpoint trả HTTP thành công và đủ 10 migration đã được áp dụng.

3.2.5. Áp dụng migration, cài phụ thuộc và build

Backend chạy synchronize=false; schema được quản lý bằng migration [22]. Phiên bản hiện tại có mười migration, trong đó migration cuối tách requestBatchId và issuanceBatchId. Sau migration, backend và ba frontend được build độc lập. Build thành công chỉ chứng minh mã có thể biên dịch và đóng gói; kiểm thử nghiệp vụ được thực hiện ở bước sau.

Bước 1 - Xác nhận ba container hạ tầng đang healthy

Tiến hành chạy:

```
$ docker ps --filter 'name=datn-repro' \
  --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
$ for c in datn-repro-postgres datn-repro-redis \
  datn-repro-bitcoin-core; do
    echo -n "$c: "
    docker inspect -f '{{.State.Health.Status}}' "$c"
  done
```

Kết quả mong đợi khi chạy thành công:

```
datn-repro-postgres: healthy
datn-repro-redis: healthy
datn-repro-bitcoin-core: healthy
```

Bước 2 - Kiểm tra đủ 10 migration

Nạp biến môi trường runtime vào shell hiện tại rồi truy vấn bảng migration:

```
$ set -a
$ source experiments/repro/runtime/.env
$ set +a
$ docker exec \
  -e PGPASSWORD="$POSTGRES_PASSWORD" \
  datn-repro-postgres \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -Atc 'select count(*) from typeorm_migrations'
```

Kết quả mong đợi khi chạy thành công:

```
10
```

Bước 3 - Kiểm tra sản phẩm build của ba frontend

Tiến hành chạy:

```
$ test -f frontend-admin/dist/index.html && echo ADMIN_BUILD_OK
$ test -f frontend-client/dist/index.html && echo STUDENT_BUILD_OK
$ test -f frontend-verify/dist/index.html && echo VERIFY_BUILD_OK
$ test -d backend/dist && echo BACKEND_BUILD_OK
```

Kết quả mong đợi khi chạy thành công:

```
ADMIN_BUILD_OK
STUDENT_BUILD_OK
VERIFY_BUILD_OK
BACKEND_BUILD_OK
```

3.2.6. Khởi tạo Blockcerts regtest và preflight

Trước khi cho người dùng thao tác trên Web, môi trường phát hành phải chứng minh rằng Issuer Profile, khóa phát hành, Bitcoin RPC và toolchain cert-tools/cert-issuer phối hợp đúng. Preflight sử dụng dữ liệu giả lập để tạo chứng thư mẫu, neo Merkle root và kiểm tra receipt. Chỉ khi proof, chain=bitcoinRegtest và OP_RETURN khớp thì hệ thống mới được coi là sẵn sàng cho demo nghiệp vụ.

Bước 1 - Kiểm tra địa chỉ phát hành và Issuer Profile đã được sinh

Tiến hành chạy:

```
$ grep '^ISSUING_ADDRESS=' experiments/repro/runtime/.env
$ cat experiments/repro/runtime/project/blockcerts/issuer/\
issuing-address.txt
$ python3 -m json.tool \
  experiments/repro/runtime/project/blockcerts/issuer/profile.json \
  | head -n 40
```

Kết quả mong đợi khi chạy thành công:

```
ISSUING_ADDRESS=<địa_chỉ_Bitcoin_regtest>
<địa_chỉ_Bitcoin_regtest>
Issuer Profile chứa URL
http://127.0.0.1:18088/api/blockcerts/issuers/kma/...
```

Bước 2 - Không hiển thị WIF; chỉ kiểm tra quyền truy cập

Tiến hành chạy:

```
$ stat -c '%a %n'
experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Kết quả mong đợi khi chạy thành công:

```
600 experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Không chạy cat đối với pk_issuer.txt trong ảnh chụp hoặc phiên trình bày, vì đây là WIF phát hành của môi trường regtest. Chỉ địa chỉ công khai và Issuer Profile được phép xuất hiện trong bằng chứng.

Khóa riêng không được đưa vào profile hoặc frontend. Việc dùng regtest cho phép chủ động tạo block và tái lập kết quả, nhưng không đại diện cho phí, mempool hay độ trễ của Bitcoin mainnet [18].

3.2.7. Triển khai backend và ba cổng web

Sau build, Nginx phục vụ ba base path cho cổng quản trị, cổng Student và cổng Verify, đồng thời chuyển tiếp API tới backend. Cổng quản trị phục vụ Maker/Checker; cổng Student chỉ hiển thị chứng thư của sinh viên đang đăng nhập; cổng Verify công khai không yêu cầu đăng nhập. Ở lượt tái kiểm chứng, các endpoint Web được kiểm tra cùng /health và Issuer Profile trước khi thực hiện E2E. Điều kiện hoàn thành giai đoạn triển khai được tổng hợp tại Bảng 3.3.

Bước 1 - Khởi động backend và Nginx của stack tái lập

Tiến hành chạy:

```
$ bash experiments/repro/scripts/start.sh
```

Kết quả mong đợi khi chạy thành công:

```
START_PASS api=14000 web=18088 fault_after_broadcast=0
```

Bước 2 - Chạy readiness/dependency check

Tiến hành chạy:

```
$ bash experiments/repro/scripts/health.sh
```

Kết quả mong đợi khi chạy thành công:

```
HEALTH_PASS containers=3 migrations=10 api=200 admin=200 student=200
verify=200
```

Bước 3 - Kiểm tra trực tiếp các cổng web bằng HTTP

Tiến hành chạy:

```
$ curl -I http://127.0.0.1:18088/admin/
$ curl -I http://127.0.0.1:18088/student/
$ curl -I http://127.0.0.1:18088/verify/
$ curl -fsS
http://127.0.0.1:18088/api/blockcerts/issuers/kma/profile.json | head
```

Kết quả mong đợi khi chạy thành công:

```
Ba yêu câù HEAD trả HTTP 200.
Issuer Profile trả JSON và không yêu câù đăng nhập.
```

Bảng 3.3. Điều kiện hoàn thành giai đoạn triển khai

Hạng mục	Điều kiện đạt	Ý nghĩa
/health	HTTP 200, backend đang chạy	Liveness của ứng dụng
PostgreSQL/ Redis/Bitcoin	Container healthy	Phụ thuộc hạ tầng sẵn sàng
Migration	10/10 đã áp dụng	Schema khớp mã nguồn
Admin/ Student/Verify	Truy cập được qua Nginx	Frontend được phục vụ đúng base path
Issuer Profile	Truy cập công khai trong môi trường demo	Verifier có dữ liệu issuer

3.2.8. Khởi tạo tài khoản thử nghiệm và kiểm tra quyền

Sau khi health check đạt, hệ thống cần có tài khoản để tái hiện luồng Web. Script bootstrap tạo một Checker trực tiếp trong DB kiểm thử, sau đó dùng API được bảo vệ để tạo Maker, Student và một Student thứ hai phục vụ ca isolation.

Tiến hành chạy câu lệnh dưới đây:

```
$ node experiments/repro/scripts/prepare-accounts.mjs
```

Kết quả mong đợi khi chạy thành công:

```
{"ok":true,"roles":4,"privateMode":"600"}
```

Kiểm tra quyền tệp chứa tài khoản thử nghiệm:

```
$ stat -c '%a %n' experiments/repro/runtime/private/accounts.json
```

Kết quả mong đợi:

```
600 experiments/repro/runtime/private/accounts.json
```

Khi cần đăng nhập trên giao diện, người vận hành có thể đọc accounts.json ngay trên máy cục bộ. Tuy nhiên không chụp hoặc sao chép mật khẩu từ tệp này vào báo cáo, Git hoặc tệp kết quả công khai. Lưu ý: khi chạy run-tests.sh ở bước kế tiếp, script tự dừng/khởi động lại ứng dụng và chạy prepare-accounts.mjs thêm một lần; vì vậy accounts.json có thể được thay bằng bộ tài khoản mới. Trước khi demo web phải thực hiện bước reset demo ở Mục 3.5 rồi đọc lại tệp này.

3.2.9. Chạy kiểm thử chức năng, an toàn và phục hồi

Bộ run-tests.sh thực hiện tuần tự unit/integration test backend, test verifier, bootstrap tài khoản, Functional E2E và Recovery E2E. Đây là cổng kiểm soát trước khi dùng số liệu benchmark. Script tự gọi start.sh, health.sh và prepare-accounts.mjs; người thực hiện không cần bootstrap tài khoản thủ công lần thứ hai trong cùng lượt test.

Tiến hành chạy:

```
$ bash experiments/repro/scripts/run-tests.sh
```

Dòng kết thúc của lượt chạy đạt:

```
TEST_PASS default=PASS benchmark=SKIPPED
```

Sau đó đọc tệp tổng hợp của lượt chạy:

```
$ python3 -m json.tool experiments/repro/runtime/results/summary.json
```

Kết quả của lượt tái kiểm chứng chính thức:

```
{
  "default": "PASS",
  "functional": true,
  "recovery": true,
  "backend_suites": 12,
  "backend_tests": 49,
  "ssrf_tests": 10,
  "verification_status_tests": 6,
  "verifier_tests": 16
}
```

Nếu một test thất bại thì không bỏ qua bằng cờ cưỡng bức. Cần đọc log trong experiments/repro/runtime/logs/, xử lý nguyên nhân và chạy lại từ bước phù hợp.

3.2.10. Chạy đo hiệu năng và cố định bộ bằng chứng

Benchmark là bước tốn thời gian hơn, chỉ chạy sau khi kiểm thử mặc định đạt. Runner phát hành 10, 100 và 500 chứng thư, mỗi kích thước lặp ba lần, rồi kiểm toán tệp kết quả và bước xác minh đầy đủ.

Tiến hành chạy:

```
$ bash experiments/repro/scripts/run-tests.sh --benchmark
```

Dòng kết thúc của lượt benchmark đạt:

```
TEST_PASS default=PASS benchmark=PASS
```

Đọc kết quả tổng hợp và kiểm toán tệp kết quả:

```
$ python3 -m json.tool experiments/repro/runtime/results/summary.json
$ python3 -m json.tool experiments/repro/runtime/results/benchmark-
artifact-audit.json | head -n 80
```

Kết quả của bộ bằng chứng chính thức:

```
summary.json (các trường cần đối chiếu):
{
  "benchmark": "PASS",
  "benchmark_runs": 9,
  "benchmark_certificates": 1830,
  "full_verifier_valid": 1830
}
benchmark-artifact-audit.json (kết quả tổng hợp):
{
  "passed": true,
  "fullVerifierValid": 1830
}
```

Các số liệu thời gian, thông lượng và tài nguyên được phân tích riêng ở Mục 3.9. Không dùng kết quả từ một working tree khác để thay cho bộ bằng chứng của cây nguồn đã chốt.

3.2.11. Dừng dịch vụ và làm sạch môi trường tái lập

Sau khi hoàn tất thực nghiệm, lựa chọn mức dừng phù hợp. Không sử dụng docker system prune vì có thể ảnh hưởng stack khác trên máy.

Chỉ dừng backend/Nginx nhưng giữ container và dữ liệu:

```
$ bash experiments/repro/scripts/stop.sh --app-only
```

Dừng cả container nhưng giữ volume:

```
$ bash experiments/repro/scripts/stop.sh --down
```

Xóa toàn bộ stack và volume của namespace tái lập (thao tác phá hủy):

```
$ bash experiments/repro/scripts/stop.sh --purge
```

Sau khi purge, chỉ dữ liệu thuộc namespace datn-blockcerts-repro bị xóa; stack chính tại cổng 4000/8088 không thuộc phạm vi kịch bản tái lập.

3.3. Hiện thực backend, dữ liệu và hàng đợi

3.3.1. Các mô-đun nghiệp vụ

Backend gồm các module xác thực, quản trị tài khoản, phát hành, xác minh, thu hồi, audit và Issuer Profile. Route cần bảo vệ gắn AuthGuard trước RolesGuard để req.user được tạo trước khi kiểm tra vai trò. Maker và Checker cùng dùng cổng quản trị nhưng không có role Admin độc lập. Checker tạo tài khoản; Maker lập phiếu; Student chỉ đọc chứng thư của chính mình; người xác minh là tác nhân công khai.

3.3.2. Mô hình dữ liệu và liên kết người học

issued_certificates vừa giữ yêu cầu nghiệp vụ vừa giữ kết quả phát hành. Quyền sở hữu chứng thư được xác định bằng khóa ngoại studentId và studentCode duy nhất; recipientName chỉ là snapshot hiển thị nên hai sinh viên trùng tên vẫn được tách biệt. requestBatchId biểu diễn nhóm phiếu Maker tạo, còn issuanceBatchId chỉ được sinh tại một lần Checker duyệt. Vì vậy một requestBatchId có thể sinh nhiều issuanceBatchId khi duyệt từng phần. Các thực thể dữ liệu trực tiếp tham gia luồng được tổng hợp tại Bảng 3.4.

Bảng 3.4. Các thực thể dữ liệu trực tiếp tham gia luồng phát hành

Bảng	Vai trò trong luồng	Điểm quan trọng
users	Tài khoản, role, Student profile	studentCode duy nhất; không trả passwordHash
issued_certificates	Phiếu + tệp chứng thư sau phát hành	studentId, requestBatchId, issuanceBatchId, txid, trạng thái, checkpoint thu hồi
issuance_batches	Trạng thái một lần phát hành	jobId ổn định, giao dịch thô, anchorTxid, attemptCount
issuance_outbox	Giao hàng job DB → Redis/BullMQ	pending/dispatched; retry nếu enqueue lỗi
verification_logs	Kết quả xác minh	Không lưu PII thô trong lượt E2E cuối
audit_logs	Actor/action/target	Chuỗi previousHash/entryHash phát hiện sửa cục bộ

3.3.3. Outbox giao dịch và phục hồi phát hành

Khi Checker duyệt, các bản ghi chứng thư, IssuanceBatch và IssuanceOutbox được ghi trong cùng transaction PostgreSQL. Sau commit, dispatcher mới đẩy job vào BullMQ. Nếu enqueue thất bại, outbox vẫn ở trạng thái pending và dispatcher nền thử lại. Job sử dụng jobId ổn định và chính sách tối đa 5 attempts với exponential backoff, giảm nguy cơ tạo nhiều công việc tương đương do lỗi mạng tạm thời.

Worker duy trì checkpoint theo lô. Trước khi broadcast, raw transaction và dữ liệu anchor được lưu bền vững; nếu tiến trình lỗi sau thời điểm node đã nhận giao dịch, recovery dùng lại transaction đã checkpoint thay vì tạo giao dịch khác. Recovery E2E của lượt 30/09 xác nhận transaction ID không đổi và số block chỉ tăng một lần.

3.3.4. Audit và dữ liệu xác minh

Audit ghi actor, role, action và target cho các bước chính. Hash chain giúp phát hiện việc sửa hoặc xóa ở giữa chuỗi, nhưng đây là tamper-evident chứ không phải bất biến tuyệt đối: DBA có thể viết lại cả chuỗi hoặc xóa phần đuôi nếu head hash không được neo ra nơi độc lập. Ngoài ra, audit hiện được ghi sau một số transaction nghiệp vụ; lỗi audit không rollback nghiệp vụ chính. Đây là giới hạn cần được nêu rõ thay vì coi audit là giao dịch nguyên tử với nghiệp vụ.

Functional E2E còn kiểm tra verification_logs không chứa họ tên, email hoặc mã sinh viên lấy từ chứng thư thử nghiệm. Tệp chứng thư trong workspace được yêu cầu có quyền 0600 trong lượt kiểm tra. Các kiểm soát này giảm rủi ro rò rỉ dữ liệu ngoài blockchain nhưng không thay thế chính sách retention, mã hóa backup và giám sát truy cập.

3.4. Hiện thực phát hành, xác minh và thu hồi Blockcerts

3.4.1. Phát hành theo lô và gom lô Merkle

Maker tạo phiếu đơn hoặc lô tối đa 500 bản ghi. Backend xác thực dữ liệu, đối chiếu Student và đặt pending_approval. Khi Checker duyệt, các phiếu được gán một issuanceBatchId mới và chuyển queued. Worker chia bước sinh unsigned certificate thành chunk 10 hồ sơ, tối đa 16 chunk song song trên máy thử nghiệm; sau đó gom toàn bộ output để gọi cert-issuer một lần. Kết quả là một Merkle tree và một transaction Bitcoin cho toàn bộ issuance batch.

Mỗi chứng thư có certUid, target hash và đường dẫn Merkle riêng nhưng dùng chung Merkle root và txid với các chứng thư trong cùng lô. Merkle root mới được neo vào OP_RETURN; toàn bộ hồ sơ và dữ liệu cá nhân không được ghi trực

tiếp lên blockchain. Đây là lý do cột txid không thể có unique constraint ở cấp chứng thư.

3.4.2. Xác minh công khai

Cổng Verify gửi JSON đến POST /api/verify. Backend và verifier kiểm tra semantic, target hash, Merkle proof, anchor Bitcoin, Issuer Profile và trạng thái thu hồi. Kết quả được phân biệt thành VALID, INVALID, REVOKED và INDETERMINATE. INVALID chỉ dùng khi có bằng chứng xác định nội dung/proof/anchor sai; lỗi mạng, timeout hoặc phụ thuộc tạm thời không đủ dữ kiện phải trả INDETERMINATE để tránh kết luận âm tính giả.

Dịch vụ tải tài nguyên ngoài áp safe-fetch: chặn loopback, private, link-local và metadata IP; kiểm tra lại sau redirect; giới hạn timeout/kích thước và xử lý kết quả DNS trộn. Bộ hồi quy cuối có 10 ca SSRF và 6 ca phân loại trạng thái, tổng 16/16 đạt. Kết quả này chứng minh các ca đã định nghĩa, không thay thế kiểm thử xâm nhập độc lập [23].

3.4.3. Thu hồi có điểm kiểm soát và đối soát trạng thái

Cơ chế thu hồi là phần mở rộng của nguyên mẫu: Checker yêu cầu thu hồi, service tạo payload REVOKE:<certUid> trong OP_RETURN. Chuỗi này không nên được mô tả như một cơ chế chuẩn bắt buộc của Blockcerts; verifier bên ngoài không mặc nhiên hiểu protocol thu hồi riêng của hệ thống.

Để tránh tạo giao dịch thứ hai khi lỗi xảy ra, service dùng advisory lock để tuần tự hóa việc tiêu UTXO. Khi bắt đầu, bản ghi chuyển revocation_pending. Raw transaction đã ký và revocationIntentTxid được checkpoint bền vững trước sendrawtransaction. Nếu sau đó xảy ra timeout hoặc crash, trạng thái chuyển revocation_reconciliation_required và tiến trình nền chỉ tra/broadcast lại đúng raw transaction đã lưu. Khi transaction có ít nhất một confirmation, bản ghi mới chuyển revoked và revocationTxid được chốt bằng intent txid. Thứ tự này quan trọng vì PostgreSQL và Bitcoin không nằm trong một transaction ACID chung.

3.5. Tái hiện quy trình nghiệp vụ trên web từ đầu đến cuối

Mục này trình bày luồng sử dụng mà người bảo vệ hoặc người vận hành có thể tái hiện trực tiếp: tạo tài khoản thử nghiệm, Maker lập phiếu, Checker phê duyệt, Worker phát hành, Student tải chứng thư, Verify kiểm tra, sửa JSON để chứng minh tính toàn vẹn và cuối cùng thu hồi. Hình 3.3–3.12 được chụp trong cùng một lượt demo sạch sau khi chạy demo-reset.sh --quick, dùng một Student và số hiệu KMA-2026-0001 xuyên suốt. Các khẳng định về gia cố an toàn, phục hồi

và số liệu định lượng vẫn được đối chiếu bằng mã nguồn cùng kết quả E2E ngày 30/09/2026, không suy ra chỉ từ ảnh giao diện.

Bước chuẩn bị - Đưa môi trường về trạng thái demo sạch

Trước khi thao tác trên Web, cần xóa dữ liệu nghiệp vụ cũ trong namespace tái lập, tạo lại bốn tài khoản thử nghiệm và kiểm tra health. Không dùng --full cho mỗi lần demo vì chế độ đó xóa toàn bộ volume/runtime.

Tiến hành chạy:

```
$ bash experiments/repro/scripts/demo-reset.sh --dry-run
$ bash experiments/repro/scripts/demo-reset.sh --quick
```

Kết quả mong đợi:

```
DEMO_RESET_DRY_RUN scope=PASS containers=PASS schema=PASS mode=quick
HEALTH_PASS containers=3 migrations=10 api=200 admin=200 student=200
verify=200
DEMO_READY mode=quick health=PASS accounts=READY data=CLEAN
```

Sau khi xuất hiện DEMO_READY, accounts.json đã được tạo lại và là bộ credential phải dùng cho toàn bộ phần Web bên dưới. Không dùng credential đã đọc trước khi chạy run-tests.sh hoặc benchmark.

Địa chỉ truy cập: trên chính máy tái lập, dùng http://127.0.0.1:18088/admin/ cho Maker/Checker, http://127.0.0.1:18088/student/ cho sinh viên và http://127.0.0.1:18088/verify/ để xác minh công khai. Khi truy cập hệ thống chính từ máy khác, dùng http://<IP>:8088/admin/, http://<IP>:8088/student/ và http://<IP>:8088/verify/; không dùng 127.0.0.1. <IP> là 192.168.4.69 trong mạng LAN hoặc 100.105.56.19 qua Tailscale.

3.5.1. Khởi tạo tài khoản Checker, Maker và Student

Bước 1 - Xác nhận tài khoản và dữ liệu Student dùng cho demo. Script demo-reset.sh --quick đã tạo một Checker, một Maker, một Student chính và một Student thứ hai để kiểm tra isolation. Tài khoản không được đăng ký công khai; Checker là vai trò có quyền tạo tài khoản.

Kiểm tra tên đăng nhập và mã sinh viên mà không in mật khẩu ra terminal:

```
$ python3 - <<'PY'
import json
p = "experiments/repro/runtime/private/accounts.json"
a = json.load(open(p))
for role in ("checker", "maker", "student"):
    print(role, "=", a[role]["username"])
print("studentCode =", a["student"]["studentCode"])
PY
```

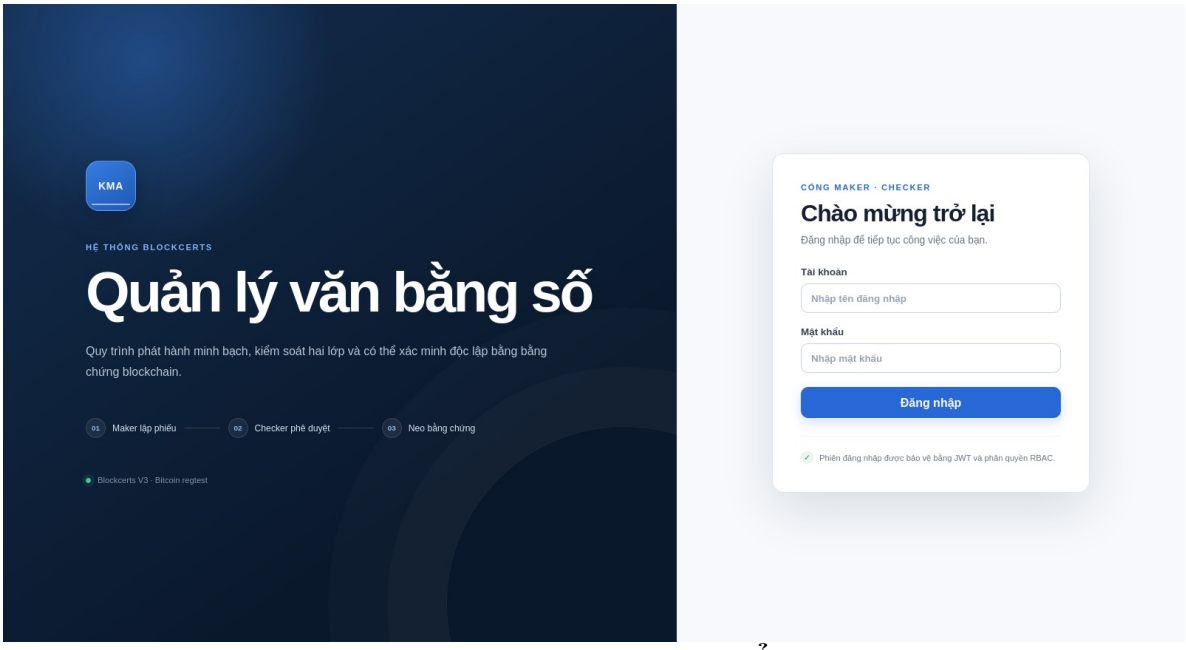
Kết quả: terminal in ra username của ba vai trò và studentCode của Student. Khi cần đăng nhập, mở accounts.json trực tiếp trên máy lab để lấy password tương ứng; không chụp mật khẩu vào báo cáo.

3.5.2. Maker đăng nhập, tra cứu Student và lập phiếu

Bước 2 - Maker đăng nhập cổng quản trị. Mở địa chỉ dưới đây trên trình duyệt, sau đó đăng nhập bằng đối tượng maker trong accounts.json:

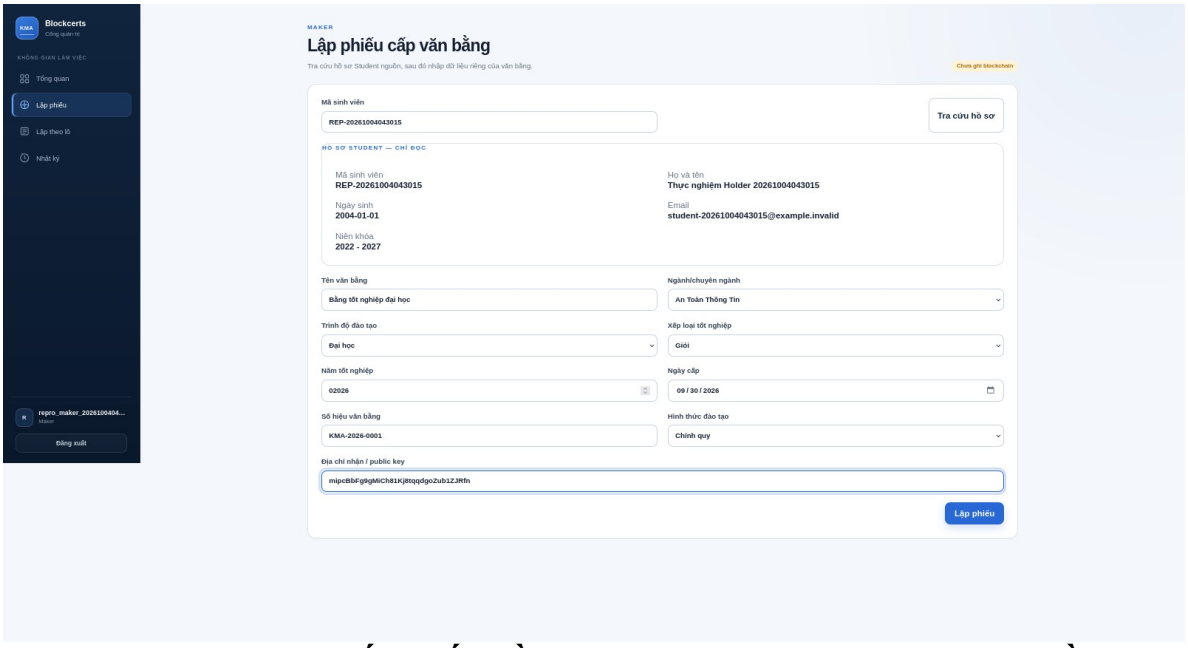
```
http://127.0.0.1:18088/admin/
```

Kết quả: giao diện quản trị mở thành công; Maker không thấy chức năng phê duyệt hoặc thu hồi. Hình 3.3 minh họa màn hình đăng nhập.



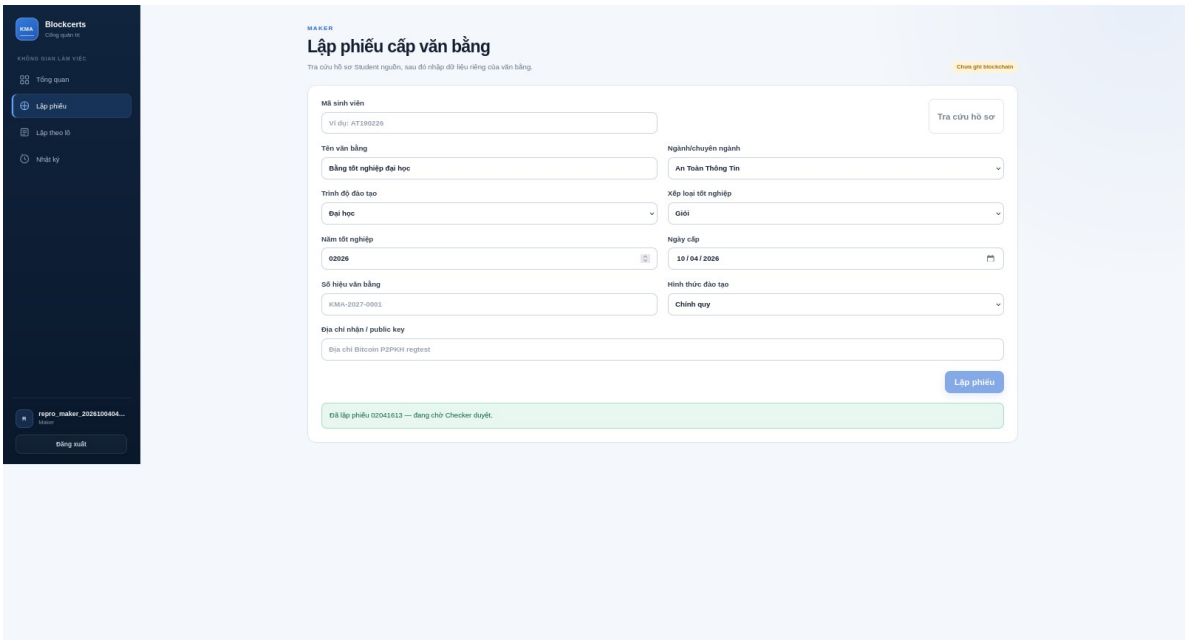
Hình 3.3. Màn hình đăng nhập cổng quản trị

Bước 3 - Maker tra cứu Student và lập phiếu theo Hình 3.4. Nhập studentCode lấy ở Bước 1, bấm tra cứu và đối chiếu hồ sơ chỉ đọc. Sau đó nhập tối thiểu các trường nghiệp vụ: tên văn bằng, ngành/chuyên ngành, trình độ, xếp loại, năm tốt nghiệp, ngày cấp, số hiệu, hình thức đào tạo và địa chỉ P2PKH regtest trong trường pubkey. Không sửa hồ sơ Student tại màn hình Maker.



Hình 3.4. Maker đối chiếu hồ sơ Student và nhập dữ liệu văn bằng

Kết quả sau khi gửi: backend tạo phiếu ở trạng thái pending_approval. requestBatchId được tạo cho nhóm phiếu, còn issuanceBatchId chưa tồn tại trước khi Checker duyệt. Hình 3.5 thể hiện thông báo lập phiếu thành công.



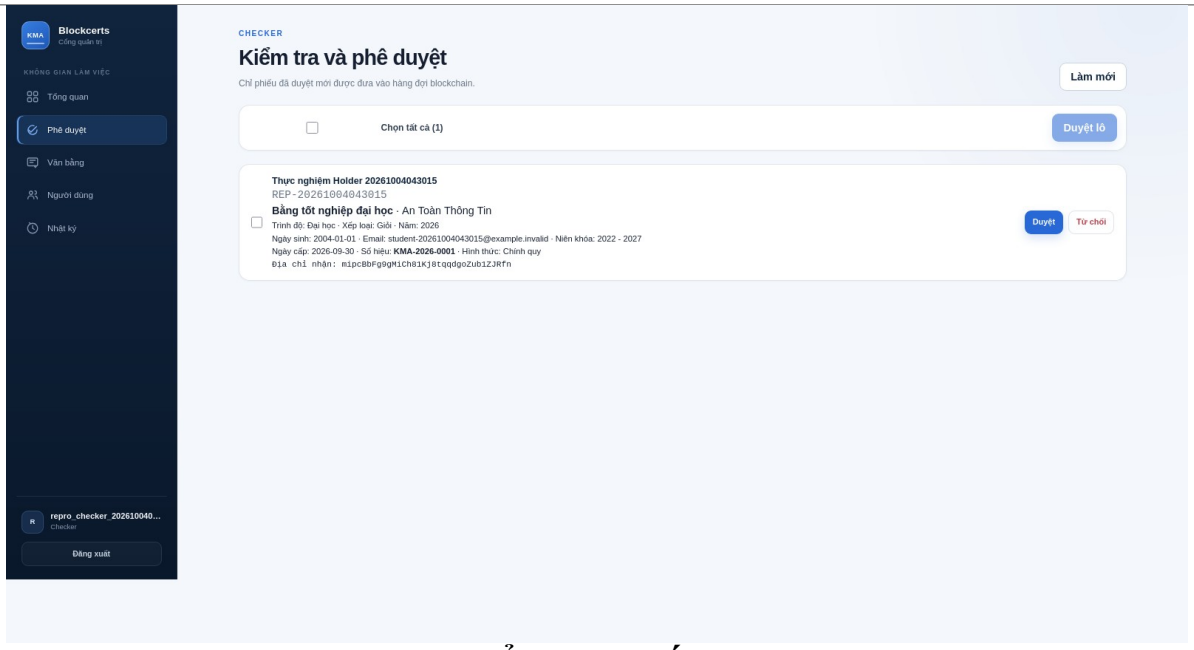
Hình 3.5. Phiếu được tạo và chờ Checker phê duyệt

3.5.3. Checker kiểm tra, phê duyệt và Worker phát hành

Bước 4 - Checker đăng nhập và phê duyệt theo Hình 3.6. Mở lại cổng quản trị, đăng xuất Maker nếu cần rồi đăng nhập bằng đối tượng checker trong accounts.json. Vào danh sách chờ duyệt, chọn phiếu vừa tạo và đối chiếu người lập, Student snapshot cùng dữ liệu văn bằng. Với nhóm nhiều phiếu, Checker có thể duyệt một tập con; mỗi lượt duyệt tạo issuanceBatchId mới nhưng giữ requestBatchId gốc.

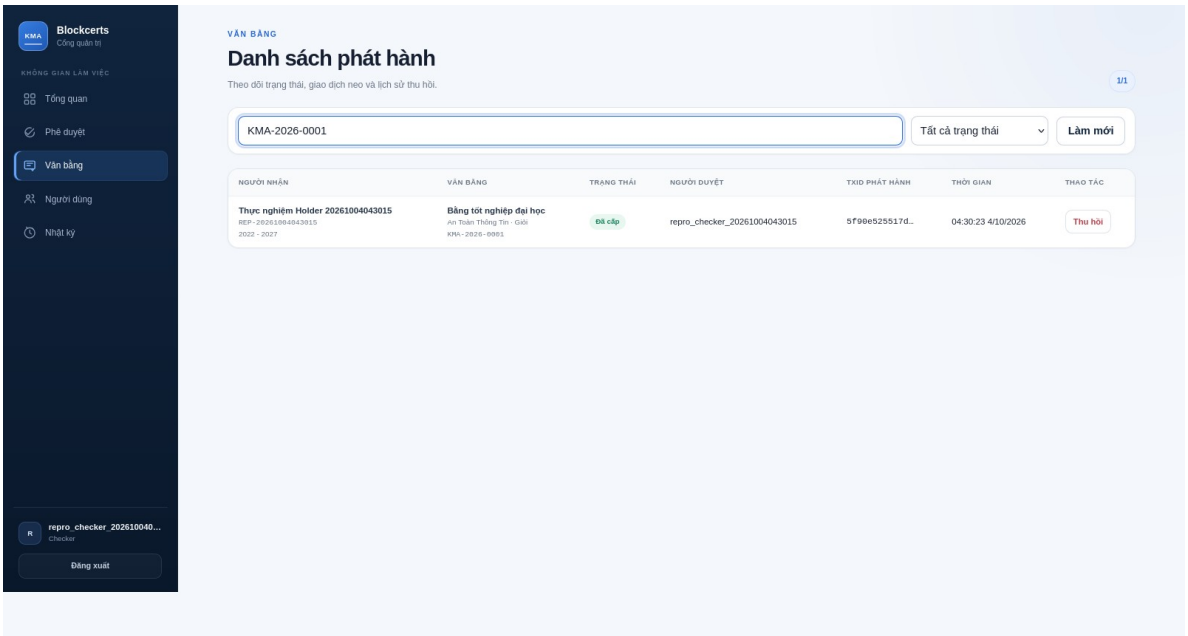
Địa chỉ sử dụng:

```
http://127.0.0.1:18088/admin/
```



Hình 3.6. Checker kiểm tra phiếu đang chờ phê duyệt

Bước 5 - Chờ Worker phát hành. Sau khi Checker bấm phê duyệt, bản ghi chuyển queued rồi processing. Dispatcher và Worker xử lý bất đồng bộ, tạo credential, cây Merkle, transaction neo và cuối cùng cập nhật issued. Không nhấn duyệt lặp lại khi job đang chạy. Kết quả đạt khi danh sách hiển thị trạng thái đã cấp/issued và có txid, Merkle root tương ứng; Hình 3.7 minh họa trạng thái sau phát hành.



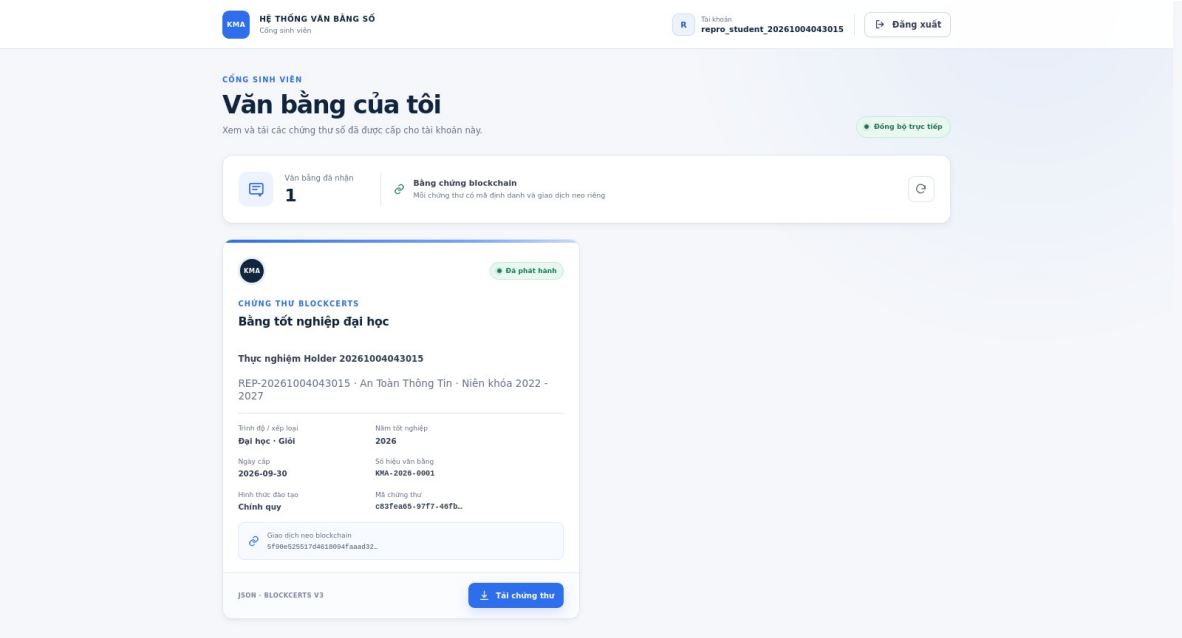
Hình 3.7. Danh sách văn bằng sau khi Worker phát hành

3.5.4. Student xem và tải chứng thư

Bước 6 - Student đăng nhập cổng Holder. Mở địa chỉ dưới đây và đăng nhập bằng đối tượng student trong accounts.json. Backend lấy subject từ JWT và chỉ trả chứng thư có studentId gắn với tài khoản hiện tại; không chấp nhận studentId tùy ý từ query làm căn cứ phân quyền.

```
http://127.0.0.1:18088/student/
```

Kết quả: Student thấy đúng chứng thư vừa được phát hành, có thể tải tệp kết quả JSON xuống máy. Functional E2E dùng Student thứ hai để xác nhận tài khoản khác không đọc được chứng thư này. Hình 3.8 minh họa danh sách và thao tác tải.



Hình 3.8. Student xem và tải chứng thư thuộc đúng tài khoản

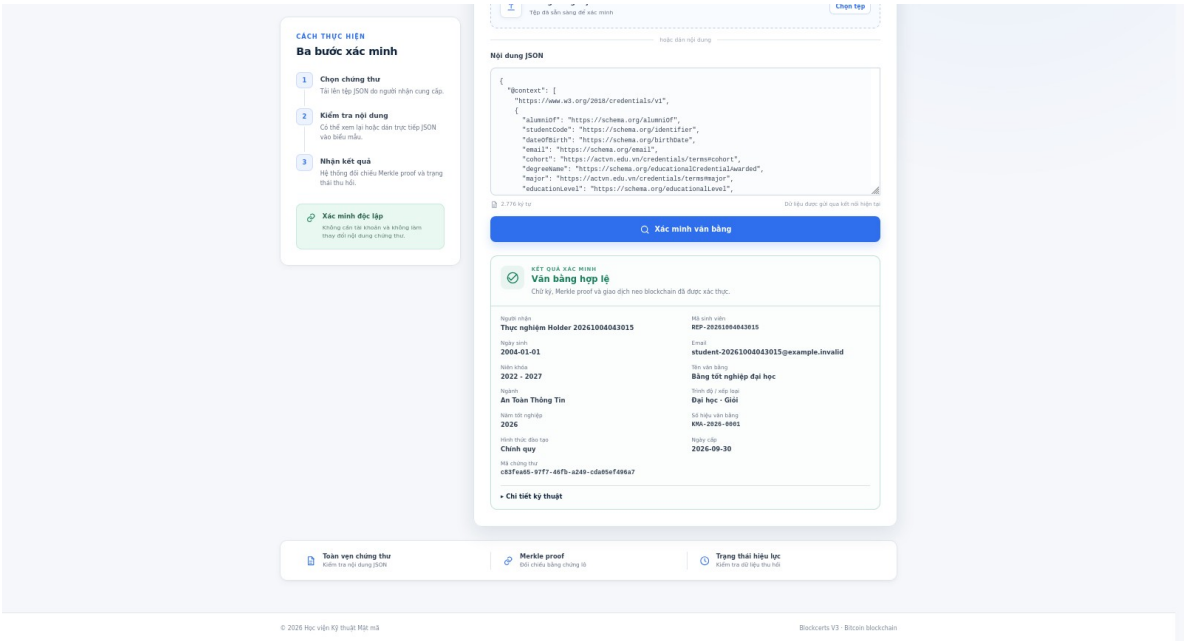
Tệp JSON tải xuống là tệp kết quả Blockcerts dùng cho bước xác minh công khai. Student vẫn có thể nhìn thấy chứng thư đã thu hồi để bảo toàn lịch sử, nhưng trạng thái phải được hiển thị rõ ràng.

3.5.5. Xác minh chứng thư hợp lệ

Bước 7 - Xác minh tệp gốc. Không cần đăng nhập; mở cổng Verify, chọn tệp JSON vừa tải ở Bước 6 và thực hiện xác minh:

```
http://127.0.0.1:18088/verify/
```

Kết quả mong đợi: trạng thái VALID. Verifier kiểm tra semantic, target hash, Merkle proof, anchor Bitcoin, Issuer Profile và trạng thái thu hồi. Hình 3.9 minh họa kết quả hợp lệ.



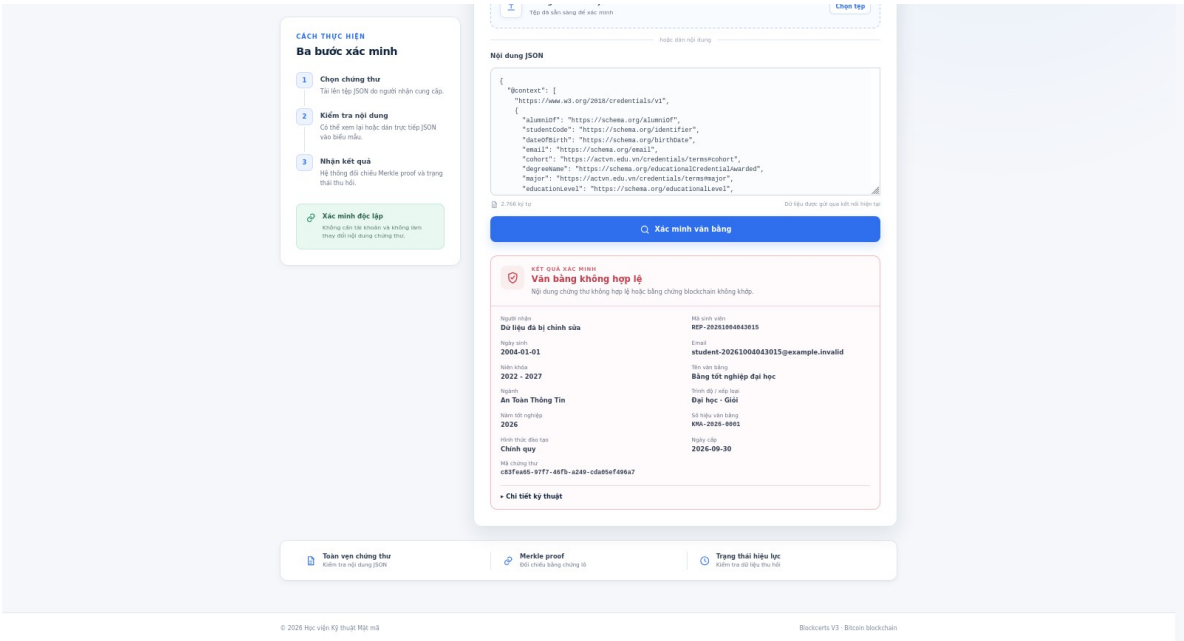
Hình 3.9. Cổng Verify xác nhận chứng thư hợp lệ

3.5.6. Sửa nội dung chứng thư và kiểm tra lại

Bước 8 - Tạo bản sao đã sửa để kiểm tra tính toàn vẹn. Không sửa tệp gốc. Giả sử tệp tải về có tên certificate.json trong thư mục Downloads, chạy:

```
$ cd ~/Downloads
$ cp certificate.json certificate-tampered.json
$ jq \
  '.credentialSubject.name = "TEN_DA_SUA_DE_KIEM_THU"' \
  certificate.json > certificate-tampered.json
```

Sau đó tải certificate-tampered.json lên cổng Verify. Kết quả mong đợi: INVALID vì nội dung được bảo vệ đã thay đổi trong khi proof/anchor vẫn thuộc tệp gốc. Hình 3.10 minh họa kết quả phát hiện sửa đổi.

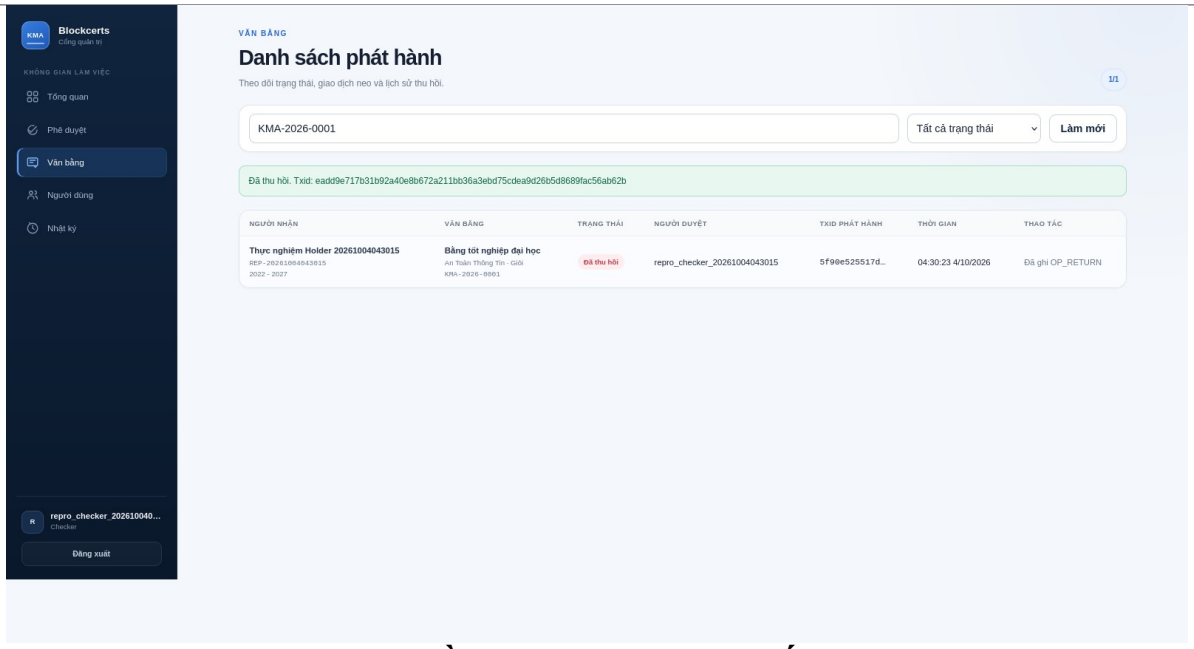


Hình 3.10. Verifier phát hiện chứng thư đã bị thay đổi

3.5.7. Checker thu hồi và xác minh lại tệp gốc

Bước 9 - Checker thu hồi chứng thư. Quay lại cổng quản trị, đăng nhập Checker, mở chứng thư đã phát hành, nhập lý do thu hồi và xác nhận. Hệ thống checkpoint raw transaction/intent txid trước broadcast, xác nhận giao dịch trên regtest rồi cập nhật trạng thái. Hình 3.11 minh họa bản ghi sau thu hồi.

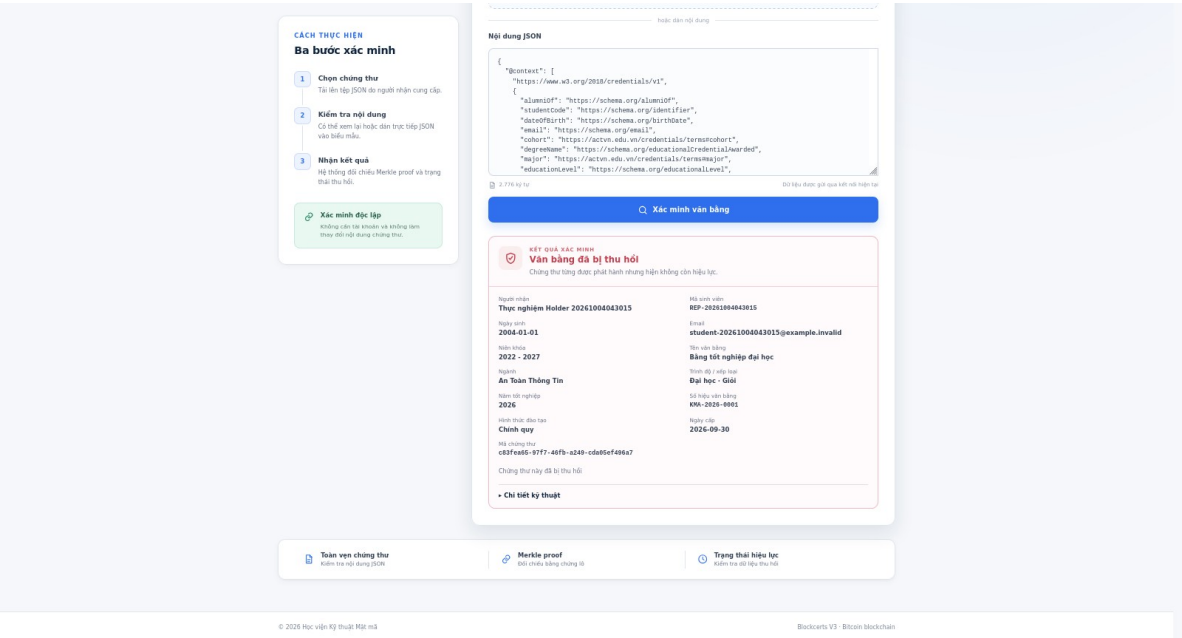
```
http://127.0.0.1:18088/admin/
```



Hình 3.11. Checker thu hồi chứng thư và hệ thống cập nhật trạng thái

Bước 10 - Xác minh lại chính tệp gốc chưa sửa theo Hình 3.12. Mở cổng Verify, tải lại certificate.json ban đầu (không dùng bản tampered). Proof và anchor phát hành vẫn hợp lệ nhưng trạng thái nghiệp vụ đã mất hiệu lực, nên kết quả cuối phải là REVOKED. Đây là khác biệt giữa tài liệu bị sửa (INVALID) và tài liệu nguyên vẹn nhưng đã bị thu hồi (REVOKED).

```
http://127.0.0.1:18088/verify/
```



Hình 3.12. Cổng Verify trả trạng thái REVOKED cho chứng thư gốc

3.6. Kiểm soát an toàn và khả năng phục hồi

Các cơ chế an toàn được kiểm tra ở backend và E2E, không chỉ dựa trên giao diện. Bảng 3.5 tổng hợp rủi ro, kiểm soát và bằng chứng hiện có trong lượt tái kiểm chứng.

Bảng 3.5. Kiểm soát an toàn và bằng chứng kiểm thử

Rủi ro/thuộc tính	Kiểm soát hiện tại	Bằng chứng
Maker tự duyệt	Checker-only approve + RolesGuard	Maker gọi approve nhận HTTP 403
IDOR giữa Maker	List/detail/audit lọc theo requestedBy	Maker B không thấy phiếu của Maker A trong danh sách, không đọc được chi tiết hoặc nhật ký liên quan
Holder đọc nhầm chứng thư	studentId/studentCode lấy từ JWT	Student thứ hai không thấy chứng thư của Student thứ nhất
SSRF qua URL trong credential	safe-fetch, kiểm tra DNS/IP/redirect/timeout	10/10 ca SSRF đạt
Queue mất job sau commit DB	outbox giao dịch + bộ điều phối nền	Outbox pending được dispatch lại
Worker lỗi quanh broadcast	raw tx/txid checkpoint + jobId ổn định	Recovery E2E giữ nguyên txid, chỉ tăng 1 block

Rủi ro/thuộc tính	Kiểm soát hiện tại	Bằng chứng
Thu hồi lặp hoặc timeout	advisory lock + checkpoint trước broadcast + reconciliation	Retry dùng lại revocationIntentTxid
Audit bị sửa cục bộ	previousHash/ entryHash	Kiểm tra chain phát hiện thay đổi giữa chuỗi
Rò PII qua verify log	redaction và assertion E2E	Log không chứa name/email/studentCode của credential thử
Lộ secret	runtime ngoài Git, quyền tệp hạn chế	Preflight + file chứng thư 0600 trong E2E

Hai giới hạn cần được giữ rõ. Thứ nhất, audit hash chain không biến log thành sổ bất biến và audit chưa nguyên tử với mọi transaction nghiệp vụ. Thứ hai, các bộ test chứng minh những ca đã định nghĩa nhưng không thay thế kiểm thử xâm nhập, kiểm thử dài hạn, kiểm tra backup/restore hoặc mô hình đối phương có quyền quản trị host.

3.7. Phương pháp kiểm thử và chuỗi bằng chứng

3.7.1. Phân lớp kiểm thử

Kết quả được báo cáo theo đúng phạm vi của từng lớp: unit/service test chứng minh hành vi mã cô lập; functional E2E chứng minh chuỗi API và hạ tầng phối hợp; recovery E2E kiểm tra điểm lỗi sau broadcast; benchmark đo thời gian trong cấu hình regtest cụ thể; bước xác minh đầy đủ kiểm tra độc lập toàn bộ tệp kết quả đã phát hành. Không cộng các loại kiểm thử khác bản chất thành một con số "tổng test hệ thống". Kết quả của từng lớp kiểm thử được tổng hợp tại Bảng 3.6.

Bảng 3.6. Kết quả kiểm thử tự động và E2E

Lớp kiểm thử	Kết quả	Điểm được chứng minh
Backend Jest	12/12 suite, 49/49 test	RBAC, DTO, state transition, outbox/checkpoint, revocation, audit, issuer/verifier
Verifier	16/16 test	10 ca SSRF; 6 ca phân loại trạng thái

Lớp kiểm thử	Kết quả	Điểm được chứng minh
Functional E2E	Đạt	Maker–Checker, partial approval, object isolation, Holder, Verify, revoke
Recovery E2E	Đạt	Phục hồi sau lỗi quanh broadcast giữ nguyên txid và không neo lặp
Build	Backend + 3 frontend đạt	Mã biên dịch và đóng gói được
Health/readiness	Đạt, đủ 10 migration	Stack phụ thuộc và endpoint sẵn sàng

3.7.2. Cố định bộ bằng chứng

Benchmark và E2E sinh tệp kết quả có batch ID, transaction ID, Merkle root và checksum. Sau kiểm toán, freeze-evidence.py đóng băng các payload và lập MANIFEST.json với kích thước, SHA-256 từng tệp và hash tổng hợp. Bộ công bố cuối còn có benchmark-summary.json và SHA256SUMS để kiểm tra 37 tệp; các tệp tổng hợp bổ sung được lập sau lượt chạy và cần được giữ cùng phiên bản công bố. Tổng benchmark chính thức là 9 lần chạy, 1.830 chứng thư và 9 transaction neo; bước xác minh đầy đủ đọc lại toàn bộ chứng thư. Bảng 3.7 và Phụ lục 4 trình bày các chỉ số và nơi lưu bằng chứng.

Bảng 3.7. Chỉ số kiểm soát cuối của bộ bằng chứng

Kiểm soát	Kết quả
Lần chạy benchmark	9/9 đạt
Chứng thư phát hành	1.830/1.830
Giao dịch neo	9
Xác minh đầy đủ	1.830/1.830 VALID
Payload manifest	35/35 khớp
Tệp SHA-256	37/37 khớp

3.8. Kết quả Kịch bản 1 – tính toàn vẹn và Maker–Checker

Kịch bản bắt đầu từ chứng thư gốc hợp lệ, lần lượt sửa dữ liệu hoặc bằng chứng và thử hành vi Maker tự duyệt. Sáu trên sáu ca đạt. Bộ tái kiểm chứng hiện tại phân loại các trường hợp sửa proof/txid xác định là INVALID; chỉ lỗi hạ tầng hoặc thiếu dữ kiện mới trả INDETERMINATE. Kết quả chi tiết được trình bày tại Bảng 3.8.

Bảng 3.8. Kết quả Kịch bản 1

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S1-01	Chứng thư gốc	VALID	VALID	Đạt
S1-02	Sửa tên người nhận	Không VALID	INVALID	Đạt
S1-03	Sửa tên văn bằng	Không VALID	INVALID	Đạt
S1-04	Sửa proofValue	Không VALID	INVALID	Đạt
S1-05	Sửa txid trong proof	Không VALID	INVALID	Đạt
S1-06	Maker tự duyệt	HTTP 403	HTTP 403	Đạt

3.9. Kết quả Kịch bản 2 – hiệu năng phát hành theo lô Merkle

3.9.1. Kết quả chín lần chạy

Mỗi kích thước 10, 100 và 500 chứng thư được lặp ba lần. Đồng hồ phía client bắt đầu trước khi gửi yêu cầu duyệt và dừng khi lần truy vấn trạng thái quan sát thấy toàn bộ lô issued. Runner kiểm tra trạng thái mỗi 2 giây, do đó thời gian đo gồm chi phí API và độ trễ phát hiện hoàn tất, có thể xấp xỉ một chu kỳ polling cộng thời gian truy vấn; đây không phải timestamp hoàn tất nội bộ chính xác của worker. Mỗi lần tạo tập certificate ID/số hiệu riêng trên dữ liệu giả lập và đúng một transaction anchor. Tải là số chứng thư trong một lô, không phải số người dùng đồng thời; các bản ghi benchmark cùng tham chiếu hồ sơ Student thử nghiệm. Bảng 3.9 trình bày chín lần chạy.

Với n = 3 lần chạy, gọi t_j là thời gian quan sát của lần j và N là số chứng thư trong lô. Thời gian trung bình là tổng các t_j chia n. SD trong Bảng 3.10 được tính bằng căn bậc hai của trung bình bình phương độ lệch (mẫu số n), để mô tả chính ba lần chạy đã công bố; không dùng nó như ước lượng không chệch cho quần thể. Giây/chứng thư của mỗi lần là t_j/N; thông lượng của mỗi lần là N/t_j. Giá trị thông lượng trong bảng là trung bình của ba giá trị N/t_j, không phải lấy N chia thời gian trung bình. Với ba lần lặp và cơ chế polling, SD nhỏ không đủ để kết luận hiệu năng ổn định trong vận hành dài hạn.

Bảng 3.9. Chín lần chạy hiệu năng tái kiểm chứng

Lô–lần	Số CT	Thời gian (s)	s/CT	CT/s	Tx
10–1	10	6,106	0,6106	1,6377	1
10–2	10	6,102	0,6102	1,6388	1
10–3	10	6,094	0,6094	1,6410	1
100–1	100	10,379	0,1038	9,6348	1
100–2	100	10,368	0,1037	9,6451	1
100–3	100	10,548	0,1055	9,4805	1
500–1	500	35,943	0,0719	13,9109	1
500–2	500	36,112	0,0722	13,8458	1
500–3	500	35,798	0,0716	13,9673	1

Cả chín lần chạy đều phát hành đủ số chứng thư dự kiến và mỗi lần chỉ tạo một transaction neo.

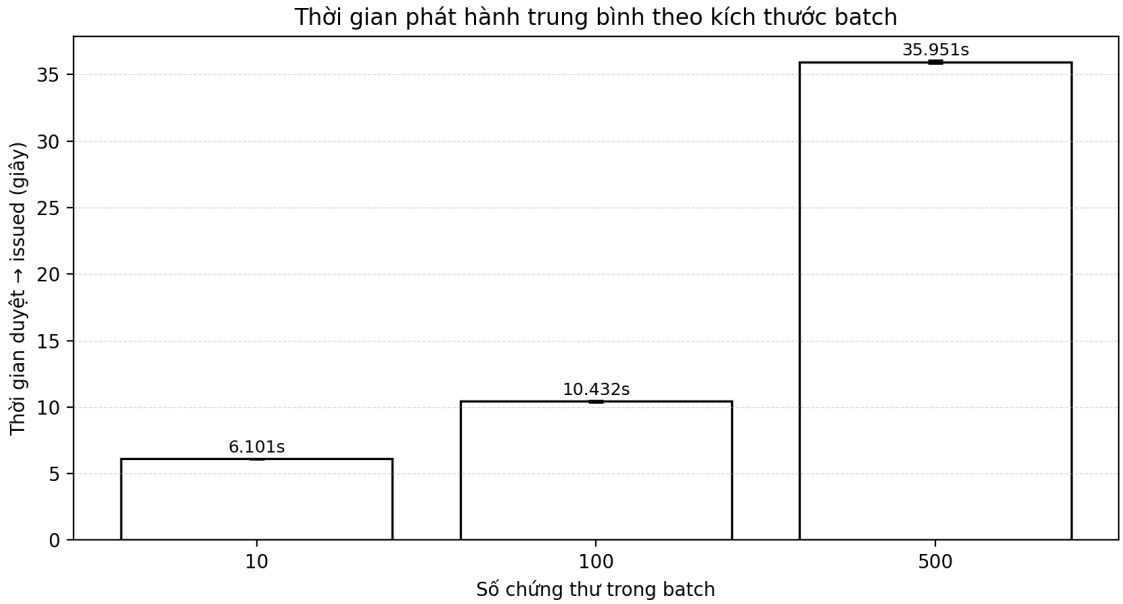
3.9.2. Giá trị tổng hợp

Bảng 3.10. Tổng hợp theo kích thước lô

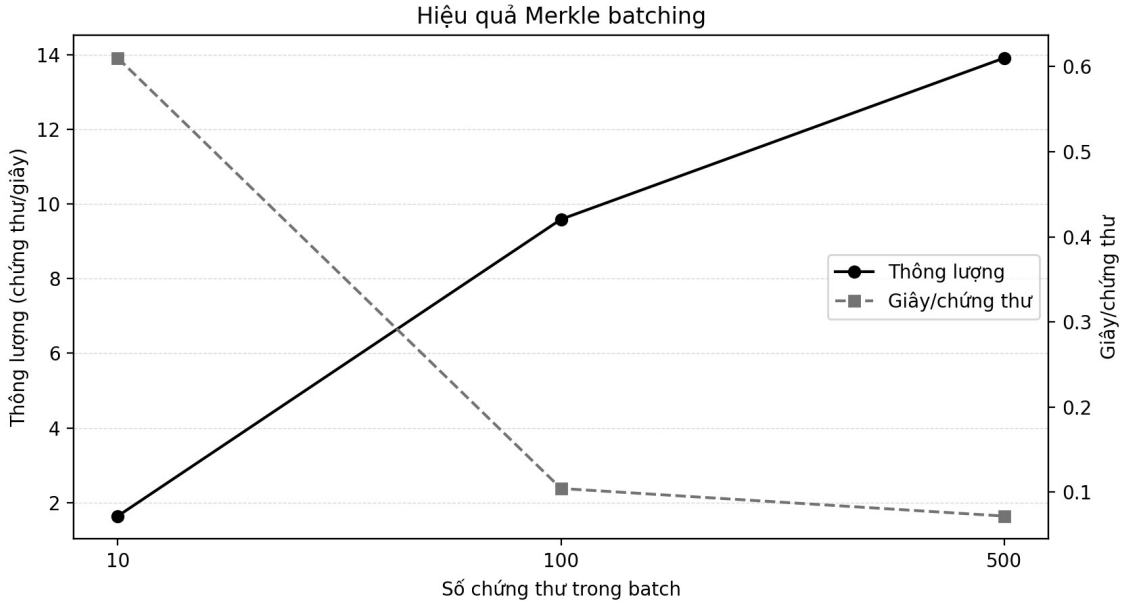
Kích thước lô	Tổng CT (3 lần)	TB ± SD (s)	s/CT	CT/s	Tx/run
10	30	6,101 ± 0,005	0,6101	1,6392	1,00
100	300	10,432 ± 0,082	0,1043	9,5868	1,00
500	1.500	35,951 ± 0,128	0,0719	13,9080	1,00

Bảng 3.10 mô tả mức thay đổi của thời gian quan sát và thông lượng khi kích thước lô tăng trong cùng cấu hình thử nghiệm; dấu ± biểu diễn độ lệch chuẩn mô tả của ba lần chạy, không phải khoảng tin cậy.

Trong ba kích thước đều sử dụng batching, số chứng thư tăng từ 10 lên 500 (50 lần), còn thời gian quan sát trung bình tăng khoảng 5,89 lần. Thông lượng trung bình tăng từ 1,6392 lên 13,9080 chứng thư/giây; thời gian quan sát trung bình trên mỗi chứng thư giảm khoảng 88,21%. Đây là xu hướng hiệu quả theo kích thước lô trong cấu hình regtest đã đo, không phải mức cải thiện so với phát hành từng chứng thư: đồ án chưa chạy phương án đối chứng không batching. Kết quả cũng không phải cam kết hiệu năng vận hành thực tế. Hình 3.13 và Hình 3.14 trực quan hóa hai xu hướng.



Hình 3.13. Thời gian phát hành trung bình theo kích thước lô



Hình 3.14. Thông lượng và thời gian trung bình trên mỗi chứng thư

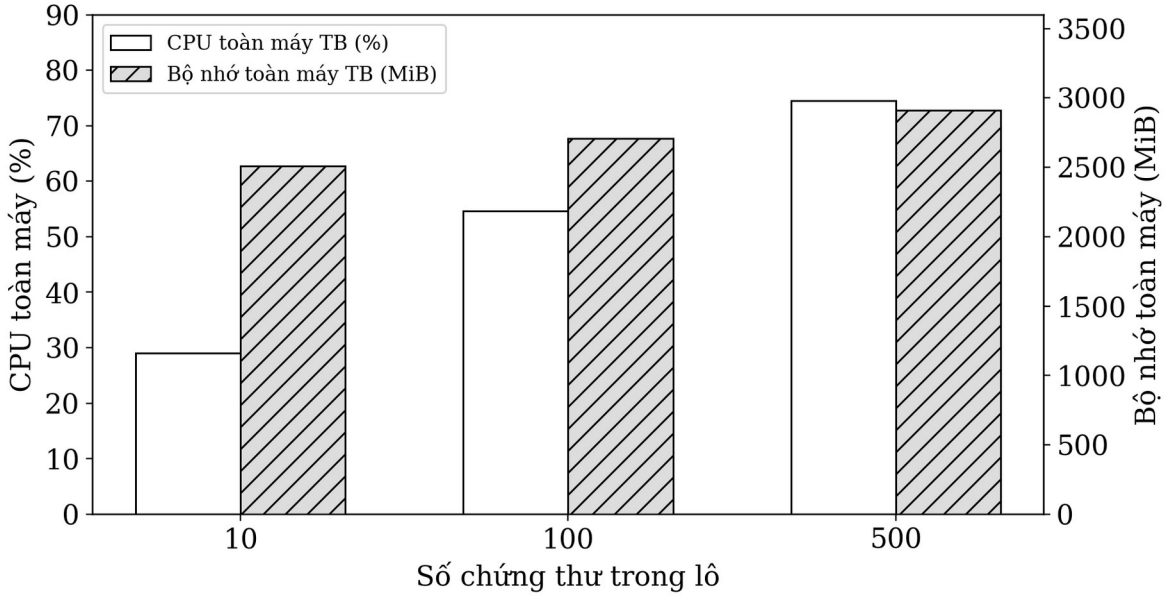
3.9.3. Tài nguyên

Bảng 3.11. Tài nguyên hệ thống theo kích thước lô

Kích thước lô	CPU TB / đỉnh (%)	RAM TB / đỉnh (MiB)	RSS backend TB / đỉnh (MiB)
10	29,00 / 42,71	2508,9 / 2543,7	166,9 / 169,4
100	54,62 / 100,00	2705,6 / 3042,8	201,5 / 205,3
500	74,45 / 100,00	2912,7 / 3323,4	221,3 / 235,6

Bảng 3.11 cung cấp số liệu CPU, bộ nhớ hệ thống và RSS backend để đặt kết quả thời gian vào đúng bối cảnh tài nguyên.

CPU được lấy từ /proc/stat ở mức toàn máy; bộ nhớ dùng là MemTotal − MemAvailable từ /proc/meminfo, cũng ở mức toàn máy. RSS backend được lấy riêng từ /proc/<pid>/status. Runner lấy mẫu định kỳ 1 giây và tại đầu/cuối lượt đo; TB là trung bình mẫu trong lượt, sau đó tổng hợp ba lượt; cột đỉnh là trung bình của các giá trị cực đại từng lượt. Bộ nhớ được chia cho 2^20 byte nên đơn vị đúng là MiB. Số liệu toàn máy có thể gồm các tiến trình khác và không được diễn giải là tài nguyên riêng của backend. CPU cực đại quan sát chạm 100% ở nhóm 100 và 500; đỉnh bộ nhớ trung bình của nhóm 500 khoảng 3.323,4 MiB. Hình 3.15 thể hiện cấu hình 4 vCPU với tối đa 16 chunk xử lý đồng thời, không phải 16 người dùng đồng thời.



Hình 3.15. Mức sử dụng tài nguyên theo kích thước lô

3.9.4. Kiểm toán tệp kết quả

Kiểm toán sau benchmark không chỉ đếm số file. Script đối chiếu certificate ID, target hash, Merkle proof, issuanceBatchId, transaction đã xác nhận và Merkle root trong OP_RETURN; bước xác minh đầy đủ sau đó đọc toàn bộ chứng thư bằng cert-verifier-js. Kết quả cho thấy 1.830/1.830 proof khớp và 1.830/1.830 chứng thư được kết luận VALID. Kết quả kiểm toán được tổng hợp tại Bảng 3.12.

Bảng 3.12. Kết quả kiểm toán tệp kết quả

Chỉ số	Kết quả
Lần chạy	9/9 đạt
Chứng thư phát hành	1.830/1.830

Chỉ số	Kết quả
Certificate ID/target hash duy nhất	1.830/1.830
Merkle proof được đối chiếu	1.830/1.830
Xác minh đầy đủ bằng cert-verifier-js	1.830/1.830 VALID
Lô phát hành/giao dịch đã xác nhận	9/9
Payload trong manifest	35/35
Tệp công bố qua SHA256SUMS	37/37

3.10. Kết quả Kịch bản 3 – thu hồi, RBAC và audit

Kịch bản thứ ba kiểm tra quyền thu hồi và tính nhất quán giữa Bitcoin, PostgreSQL, Revocation List, Verify và audit. Chỉ Checker được phép thu hồi. Tệp gốc trước thu hồi phải VALID và sau thu hồi phải REVOKED. Tám ca chính được tổng hợp tại Bảng 3.13.

Bảng 3.13. Kết quả Kịch bản 3

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S3-01	Trước thu hồi	VALID	VALID	Đạt
S3-02	Student thu hồi	HTTP 403	HTTP 403	Đạt
S3-03	Maker thu hồi	HTTP 403	HTTP 403	Đạt
S3-04	Checker thu hồi	HTTP 201	HTTP 201	Đạt
S3-05	Sau thu hồi	REVOKED	REVOKED	Đạt
S3-06	Tx thu hồi xác nhận	≥1 confirmation	1	Đạt
S3-07	Có trong Revocation List	Có	Có	Đạt
S3-08	Audit đủ role/action	4 action	request/ approve/ issue/revoke	Đạt

Ngoài tám ca trên, functional E2E còn kiểm tra các điều kiện gia cố an toàn không nên gộp vào ba kịch bản cũ: duyệt từng phần, Maker-to-Maker isolation, Student isolation và privacy của verification log. Các kiểm tra bổ sung được tổng hợp tại Bảng 3.14.

Bảng 3.14. Một số kiểm tra E2E bổ sung của phiên bản hiện tại

Kiểm tra	Quan sát cuối	Kết quả
Hai lần duyệt từng phần cùng requestBatchId	Sinh hai issuanceBatchId khác nhau	Đạt
Maker B list phiếu Maker A	Không xuất hiện	Đạt
Maker B đọc trực tiếp phiếu Maker A	HTTP 404	Đạt
Maker B đọc audit mục tiêu của Maker A	Không xuất hiện	Đạt
Student khác đọc chứng thư	Không xuất hiện	Đạt
Verification log chứa name/email/studentCode	Không tìm thấy	Đạt
Quyền tệp chứng thư trong workspace	0600	Đạt

3.11. Đánh giá kết quả, hạn chế và hướng phát triển

3.11.1. Mức đáp ứng

Bảng 3.15. Đối chiếu yêu cầu chính và bằng chứng

Yêu cầu	Bằng chứng	Kết quả
Maker không tự duyệt	S1-06 + functional E2E	Đạt
Phát hiện sửa chứng thư/bằng chứng	S1-02 đến S1-05	Đạt
Phát hành lô 10/100/500	9 lần chạy, 1.830 chứng thư	Đạt
Một giao dịch mỗi lô	9 lô / 9 giao dịch	Đạt
Duyệt từng phần không xung đột lô	Functional E2E	Đạt
Holder/Maker object isolation	Functional E2E	Đạt

Yêu cầu	Bằng chứng	Kết quả
Chỉ Checker thu hồi	S3-02 đến S3-04	Đạt
Verify nhận biết thu hồi	S3-05 đến S3-07	Đạt
Phục hồi sau lỗi quanh broadcast	Recovery E2E	Đạt
Build/test/readiness	49/49 backend; 16/16 verifier; 3 frontend; 10 migration	Đạt

Bảng 3.15 cho thấy các yêu cầu chính đều có bằng chứng tương ứng trong phạm vi thử nghiệm; các giới hạn còn lại được nêu ở mục tiếp theo.

3.11.2. Hạn chế

Thứ nhất, Bitcoin regtest không phản ánh phí, độ trễ, cạnh tranh mempool, reorg và kinh tế an ninh của mainnet. Thứ hai, benchmark chạy trên một máy 4 vCPU và mỗi kích thước chỉ lặp ba lần; kết quả chưa đại diện tải dài hạn, nhiều node hoặc mức xử lý đồng thời lớn. Thứ ba, khóa phát hành chưa được bảo vệ bằng HSM/KMS; hệ thống chưa có MFA, secret manager, quy trình xoay khóa và HTTPS/HSTS công khai hoàn chỉnh.

Thứ tư, Revocation List vẫn lấy trạng thái từ PostgreSQL và chưa có indexer đủ khả năng tái dựng toàn bộ trạng thái thu hồi chỉ từ blockchain. Payload REVOKE:<certUid> là extension của nguyên mẫu, nên khả năng tương tác với verifier Blockcerts bên ngoài cần được đánh giá riêng. Thứ năm, audit hash chain chỉ giúp phát hiện sửa cục bộ; audit chưa atomic với mọi thay đổi nghiệp vụ và head hash chưa được neo ra kho append-only độc lập.

Thứ sáu, recovery E2E kiểm tra fault point quanh broadcast nhưng chưa bao phủ kill tiến trình ở mọi thời điểm, mất RPC kéo dài, hỏng volume, backup/restore hoặc thảm họa host. Thứ bảy, 10 ca SSRF bao phủ các trường hợp đã định nghĩa nhưng không thay thế kiểm thử xâm nhập độc lập. Ảnh giao diện ở Mục 3.5 minh họa cùng một lượt demo sạch; kết quả định lượng và kiểm thử hành vi được gắn với lượt tái kiểm chứng ngày 30/09/2026. Ảnh không tự chứng minh snapshot mã nguồn hoặc thay thế log/E2E; khi tái chụp cần ghi lại ngày, commit và dữ liệu demo.

3.11.3. Hướng phát triển

Hướng tiếp theo là tách rõ bảng request, batch, certificate và revocation; xây indexer phục hồi trạng thái từ blockchain; neo head hash audit ra kho độc lập; đưa

khóa vào HSM/KMS; tách worker đặc quyền khỏi API; bổ sung MFA, HTTPS/HSTS, backup/restore tự động, giám sát tập trung và kiểm thử xâm nhập. Trước khi dùng testnet/mainnet cần xác định chính sách confirmation, phí, quyền riêng tư, tương thích verifier và quy trình pháp lý của cơ sở đào tạo. Với dữ liệu cá nhân, cần bổ sung retention, mã hóa backup và quy trình thực hiện quyền của chủ thể dữ liệu theo phạm vi pháp luật áp dụng.

3.12. Kết luận Chương 3

Chương 3 đã tái lập nguyên mẫu từ môi trường Ubuntu mới đến hệ thống Web hoạt động, mô tả chính xác cơ chế phát hành Blockcerts theo lô, cơ chế outbox giao dịch, checkpoint/reconciliation, xác minh công khai và thu hồi. Luồng Web được trình bày xuyên suốt từ Maker lập phiếu, Checker duyệt, Worker phát hành, Student nhận chứng thư, Verify xác minh, phát hiện sửa đổi và thu hồi.

Đợt tái kiểm chứng cuối đạt 12/12 bộ kiểm thử backend với 49/49 ca đạt; 16/16 ca của bộ xác minh đạt; kiểm thử E2E chức năng và phục hồi đều thành công. Chín lần đo hiệu năng đã phát hành 1.830 chứng thư, kiểm tra đầy đủ 1.830/1.830 chứng thư hợp lệ và xác nhận tính toàn vẹn của bộ bằng chứng.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết quả đạt được và mức hoàn thành mục tiêu

Đồ án đã hoàn thành mục tiêu phân tích, thiết kế và xây dựng nguyên mẫu hỗ trợ quản lý, phát hành, xác minh và thu hồi chứng thư Blockcerts dựa trên Bitcoin. Hệ thống triển khai ba vai trò xác thực Maker, Checker và Student; tách cổng quản trị, cổng Holder và cổng Verify; áp dụng Maker–Checker, RBAC kết hợp kiểm soát mức đối tượng, phát hành theo lô, Merkle proof, cơ chế outbox giao dịch, checkpoint/reconciliation và audit hash chain. Blockchain chỉ lưu bằng chứng tối thiểu, còn dữ liệu nghiệp vụ và dữ liệu cá nhân được quản lý ngoài chuỗi.

Kết quả tái kiểm chứng tại commit 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1 cho thấy 12/12 bộ kiểm thử backend với 49/49 ca đạt, 16/16 ca kiểm thử bộ xác minh đạt, kiểm thử E2E chức năng và phục hồi đều thành công. Chín lần đo hiệu năng đã phát hành 1.830/1.830 chứng thư, tạo chín giao dịch neo và xác minh đầy đủ 1.830/1.830 chứng thư ở trạng thái hợp lệ.

Các kết quả trên cho thấy nguyên mẫu đáp ứng các yêu cầu đã đặt ra trong phạm vi Bitcoin regtest và dữ liệu giả lập: phân tách quyền lập–duyệt, phát hiện sửa đổi, phát hành theo lô, cô lập dữ liệu người dùng, xác minh công khai và thu hồi. Tuy nhiên, kết quả không chứng minh hệ thống sẵn sàng cho môi trường vận hành thực tế hoặc Bitcoin mainnet. Chứng thư Blockcerts trong đồ án là tệp kết quả kỹ thuật phục vụ nghiên cứu; giá trị pháp lý của văn bằng số còn phụ thuộc thẩm quyền, quy trình nghiệp vụ và việc tuân thủ quy định hiện hành [25].

2. Đóng góp của đồ án

Đóng góp của đồ án tập trung ở năm điểm. Thứ nhất, xây dựng mô hình nghiệp vụ Maker–Checker có phân quyền mức vai trò và mức đối tượng, giảm nguy cơ một chủ thể tự lập rồi tự duyệt. Thứ hai, hiện thực phát hành Blockcerts V3 theo lô, dùng một Merkle root và một Bitcoin transaction cho mỗi lô trong môi trường thử nghiệm. Thứ ba, tách requestBatchId và issuanceBatchId, kết hợp cơ chế outbox giao dịch, job ID ổn định và checkpoint để hỗ trợ duyệt từng phần và phục hồi. Thứ tư, xây dựng chuỗi xác minh phân biệt VALID, INVALID, REVOKED và INDETERMINATE, đồng thời bổ sung kiểm soát SSRF, audit hash chain và cơ chế thu hồi idempotent. Thứ năm, xây dựng bộ tái lập và bằng chứng có dấu vân tay mã nguồn, manifest, checksum, functional/recovery E2E, benchmark và bước xác minh đầy đủ để các kết luận thực nghiệm có thể được đối chiếu.

3. Hạn chế và hướng phát triển

Hệ thống vẫn có các giới hạn cần được xử lý trước khi triển khai thực tế. Regtest không phản ánh đầy đủ phí giao dịch, vùng nhớ giao dịch chờ (mempool), tái tổ chức chuỗi (reorg) và độ phân tán của mạng chính (mainnet); khóa phát hành chưa được bảo vệ bằng HSM/KMS; chưa có MFA, HTTPS/HSTS công khai, HA, backup/restore được kiểm chứng và giám sát tập trung. Audit hash chain mới hỗ trợ phát hiện sửa cục bộ, chưa neo head hash ra kho độc lập; Revocation List vẫn phụ thuộc PostgreSQL và chưa có indexer phục hồi đầy đủ từ blockchain. Kiểm thử SSRF và fault-injection đã được bổ sung nhưng chưa thay thế kiểm thử xâm nhập độc lập hoặc thử tải dài hạn. Ngoài ra, việc xử lý dữ liệu người học cần có chính sách retention, phân quyền, mã hóa bản sao lưu và quy trình đáp ứng quyền của chủ thể dữ liệu phù hợp Luật Bảo vệ dữ liệu cá nhân và văn bản hướng dẫn [26], [27]. Hướng phát triển ưu tiên là tách worker đặc quyền khỏi API, đưa khóa vào HSM/KMS, hoàn thiện backup/indexer, bổ sung MFA và TLS, mở rộng kiểm thử mức xử lý đồng thời–disaster recovery, đánh giá khả năng tương tác với verifier ngoài hệ thống và chỉ chuyển sang testnet/mainnet sau khi có đánh giá pháp lý, chi phí và chính sách vận hành.

TÀI LIỆU THAM KHẢO

[1] D. Yaga, P. Mell, N. Roby và K. Scarfone, "Blockchain Technology Overview," NISTIR 8202, National Institute of Standards and Technology, 2018. DOI: 10.6028/NIST.IR.8202.

[2] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Có tại: https://bitcoin.org/bitcoin.pdf (truy cập ngày 02/10/2026).

[3] R. C. Merkle, "Secrecy, Authentication, and Public Key Systems," Luận án tiến sĩ, Stanford University, 1979.

[4] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.

[5] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-5, 2023. DOI: 10.6028/NIST.FIPS.186-5.

[6] M. Castro và B. Liskov, "Practical Byzantine Fault Tolerance," Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI), 1999, tr. 173–186. Có tại: https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance.

[7] D. Ongaro và J. Ousterhout, "In Search of an Understandable Consensus Algorithm," 2014 USENIX Annual Technical Conference, 2014, tr. 305–319. Có tại: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro.

[8] E. Androulaki và cộng sự, "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains," Proceedings of the Thirteenth EuroSys Conference, 2018. DOI: 10.1145/3190508.3190538.

[9] W3C, "Verifiable Credentials Data Model v2.0," W3C Recommendation, 15/05/2025. Có tại: https://www.w3.org/TR/2025/REC-vc-data-model-2.0-20250515/ (truy cập ngày 02/10/2026).

[10] T. T. A. Dinh, R. Liu, M. Zhang, G. Chen, B. C. Ooi và J. Wang, "Untangling Blockchain: A Data Processing View of Blockchain Systems," IEEE Transactions on Knowledge and Data Engineering, tập 30, số 7, tr. 1366–1385, 2018. DOI: 10.1109/TKDE.2017.2781227.

[11] Blockcerts, "cert-schema: JSON schemas and context for Blockcerts," commit 1e390ac, 31/07/2025. Có tại: https://github.com/blockchain-certificates/cert-schema/tree/1e390ace91fb2b09518e45760168f344ef3a4409 (truy cập ngày 02/10/2026).

[12] W3C, "JSON-LD 1.1," W3C Recommendation, 16/07/2020. Có tại: https://www.w3.org/TR/2020/REC-json-ld11-20200716/ (truy cập ngày 02/10/2026).
[13] M. Sporny và D. Longley (biên tập), "Verifiable Credential Data Integrity 1.0," W3C Recommendation, 15/05/2025. Có tại: https://www.w3.org/TR/vc-data-integrity/.
[14] Blockcerts, "jsonld-signatures-merkleproof2019: MerkleProof2019 suite," commit 26a9c30, 25/08/2026. Có tại: https://github.com/blockchain-certificates/jsonld-signatures-merkleproof2019/tree/26a9c3071355a9ccd83b0b90fc375a4d8cf1d4b8 (truy cập ngày 02/10/2026).
[15] M. Jones, J. Bradley và N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015. Có tại: https://www.rfc-editor.org/rfc/rfc7519.
[16] OWASP Foundation, "Authorization Cheat Sheet," OWASP Cheat Sheet Series. Có tại: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (truy cập ngày 02/10/2026).
[17] OWASP Foundation, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series. Có tại: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (truy cập ngày 02/10/2026).
[18] Bitcoin Project, "Bitcoin Developer Guide – Testing Applications." Có tại: https://developer.bitcoin.org/examples/testing.html (truy cập ngày 02/10/2026).
[19] NestJS, "NestJS Documentation." Có tại: https://docs.nestjs.com/ (truy cập ngày 02/10/2026).
[20] PostgreSQL Global Development Group, "PostgreSQL 17 Documentation – Transactions." Có tại: https://www.postgresql.org/docs/17/tutorial-transactions.html (truy cập ngày 02/10/2026).
[21] BullMQ, "BullMQ Documentation." Có tại: https://docs.bullmq.io/ (truy cập ngày 02/10/2026).
[22] TypeORM, "TypeORM Documentation – Migrations and Transactions." Có tại: https://typeorm.io/docs/advanced-topics/migrations/ và https://typeorm.io/docs/advanced-topics/transactions/ (truy cập ngày 02/10/2026).

[23] OWASP Foundation, “Server-Side Request Forgery Prevention Cheat Sheet,” OWASP Cheat Sheet Series. Có tại: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html (truy cập ngày 02/10/2026).
[24] W3C, “Verifiable Credentials Data Model v1.1,” W3C Recommendation, 09/11/2021. Có tại: https://www.w3.org/TR/2021/REC-vc-data-model-20211109/ (truy cập ngày 02/10/2026).
[25] Bộ Giáo dục và Đào tạo, “Thông tư số 10/2026/TT-BGDĐT ngày 26/02/2026 ban hành Quy chế văn bằng, chứng chỉ của hệ thống giáo dục quốc dân,” có hiệu lực từ 15/04/2026. Cổng Thông tin điện tử Chính phủ. Có tại: https://vanban.chinhphu.vn/?classid=1&docid=217140&pageid=27160&typegroupid=6 (truy cập ngày 02/10/2026).
[26] Quốc hội, “Luật số 91/2025/QH15 ngày 26/06/2025: Luật Bảo vệ dữ liệu cá nhân,” có hiệu lực từ 01/01/2026. Cổng Thông tin điện tử Chính phủ. Có tại: https://vanban.chinhphu.vn/?docid=214590&pageid=27160 (truy cập ngày 02/10/2026).
[27] Chính phủ, “Nghị định số 356/2025/NĐ-CP ngày 31/12/2025 quy định chi tiết một số điều và biện pháp thi hành Luật Bảo vệ dữ liệu cá nhân,” có hiệu lực từ 01/01/2026. Cổng Thông tin điện tử Chính phủ. Có tại: https://vanban.chinhphu.vn/?classid=1&docid=216387&pageid=27160 (truy cập ngày 02/10/2026).

PHỤ LỤC

Phụ lục 1. Danh mục API chính và quyền truy cập

Bảng PL1.1 đối chiếu trực tiếp các API với controller tại commit 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1. Quyền trên giao diện chỉ hỗ trợ trải nghiệm; quyết định cho phép hoặc từ chối được thực hiện ở backend bằng JWT, RolesGuard và kiểm tra phạm vi đối tượng. Maker khi đọc danh sách, chi tiết và audit chỉ được thấy dữ liệu thuộc phạm vi của mình; Student chỉ nhận chứng thư gắn với subject hiện tại.

Bảng PL1.1. API chính của nguyên mẫu

Phương thức	Đường dẫn	Quyền	Mục đích
POST	/api/auth/login	Công khai	Đăng nhập và nhận JWT
POST	/api/auth/register	Checker	Tạo tài khoản
GET	/api/admin/users	Checker	Liệt kê tài khoản
POST	/api/admin/users	Checker	Tạo tài khoản từ cổng quản trị
PATCH	/api/admin/users/:id/student-profile	Checker	Cập nhật hồ sơ Student
GET	/api/audit	Maker, Checker	Đọc audit theo phạm vi
GET	/api/audit/integrity	Maker, Checker	Kiểm tra chuỗi hash audit
POST	/api/issue/request	Maker	Lập một phiếu
POST	/api/issue/batch/request	Maker	Lập lô 1–500 phiếu
GET	/api/issue/ students/:studentCode	Maker	Tra cứu Student chỉ đọc
POST	/api/issue/:id/approve	Checker	Duyệt một phiếu
POST	/api/issue/batch/approve	Checker	Duyệt nhiều phiếu
POST	/api/issue/batch/:batchId/ retry	Checker	Retry lô thất bại
POST	/api/issue/:id/reject	Checker	Từ chối phiếu

Phương thức	Đường dẫn	Quyền	Mục đích
GET	/api/issue	Maker, Checker	Danh sách phiếu/văn bằng theo phạm vi
GET	/api/issue/:id	Maker, Checker	Chi tiết theo phạm vi
GET	/api/issue/holder/certificates	Student	Chứng thư của Student hiện tại
POST	/api/revoke/check	Công khai	Kiểm tra trạng thái thu hồi
POST	/api/revoke/:id	Checker	Thu hồi chứng thư
POST	/api/verify	Công khai	Xác minh chứng thư JSON
GET	/api/blockcerts/issuers/kma/profile.json	Công khai	Issuer Profile
GET	/api/blockcerts/issuers/kma/revocation-list.json	Công khai	Revocation List
GET	/health	Công khai	Liveness tối thiểu

Phụ lục 2. Khung cấu hình tái lập và bảo vệ bí mật

Các nhóm cấu hình trong Bảng PL2.1 được chia thành dữ liệu công khai, dữ liệu nội bộ và bí mật. Tệp ví dụ chỉ mô tả tên biến; giá trị thật phải được sinh riêng cho môi trường. Mật khẩu, JWT secret, Bitcoin RPC credential và WIF không được đưa vào Git, ảnh chụp hoặc bộ bằng chứng công khai. Tệp runtime và khóa riêng cần giới hạn quyền đọc; dữ liệu cá nhân thử nghiệm phải là dữ liệu giả lập.

Bảng PL2.1. Nhóm cấu hình và mức nhạy cảm

Nhóm	Ví dụ	Nguyên tắc
PostgreSQL	POSTGRES_PASSWORD, DATABASE_URL	Sinh riêng; chỉ mở trong phạm vi cần thiết
Redis	REDIS_PASSWORD, REDIS_URL	Không công bố; chỉ dùng cho queue/runtime

Nhóm	Ví dụ	Nguyên tắc
JWT	JWT_SECRET, JWT_EXPIRES_IN	JWT_SECRET ngẫu nhiên đủ mạnh; thời hạn token cấu hình riêng; không ghi log hoặc công bố secret
Bitcoin RPC	BITCOIN_RPC_USER, BITCOIN_RPC_PASSWORD	Chỉ backend/worker cần truy cập
Khóa phát hành	WIF trong storage/credentials	Không commit; quyền tệp hạn chế; production cần HSM/KMS
Issuer public	Issuer Profile, revocation URL	Được công bố; không chứa khóa riêng
Frontend	API base URL	Không chứa credential
Bằng chứng	Manifest, checksum, kết quả test	Loại secret và dữ liệu cá nhân không cần thiết trước khi công bố

Phụ lục 3. Quy trình tái lập và lệnh đại diện

Phụ lục này là bảng lệnh tra cứu nhanh; Mục 3.2 của Chương 3 mới là hướng dẫn đầy đủ theo từng bước, câu lệnh và kết quả đối chiếu. Không gọi trực tiếp các kịch bản nội bộ benchmark/E2E ngoài chuỗi chính nếu không có mục đích gỡ lỗi.

C1. Cố định mã nguồn

```
$ git clone https://github.com/ebkk24/DATN-AT190226.git
$ cd DATN-AT190226
$ git checkout 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1
$ git status --short
```

C2. Chuẩn bị stack tái lập. prepare.sh tự sinh runtime/.env và secret, build backend/frontend, dựng PostgreSQL/Redis/Bitcoin, sinh cấu hình Blockcerts và chạy migration; không có tệp .env.repro.example cần sao chép.

```
$ bash experiments/repro/scripts/prepare.sh --reset
$ bash experiments/repro/scripts/start.sh
$ bash experiments/repro/scripts/health.sh
```

Kết quả health đạt:

```
HEALTH_PASS containers=3 migrations=10 api=200 admin=200 student=200
verify=200
```

C3. Chạy kiểm thử mặc định. run-tests.sh tự tạo source manifest với --require-clean, tự start/health, bootstrap tài khoản, chạy backend/verifier test, Functional E2E và Recovery E2E.

```
$ bash experiments/repro/scripts/run-tests.sh
```

```
TEST_PASS default=PASS benchmark=SKIPPED
```

C4. Chạy benchmark đầy đủ. Runner tự chạy 10/100/500 × 3 với đúng --size/--repeat, sau đó kiểm toán tệp kết quả và bước xác minh đầy đủ; không gọi benchmark.mjs trống.

```
$ bash experiments/repro/scripts/run-tests.sh --benchmark
```

```
TEST_PASS default=PASS benchmark=PASS
```

C5. Chuẩn bị demo web sau khi test/benchmark. Lệnh quick giữ image, migration và blockchain regtest nhưng làm sạch dữ liệu nghiệp vụ, queue và tệp runtime, rồi tạo lại tài khoản.

```
$ bash experiments/repro/scripts/demo-reset.sh --dry-run
$ bash experiments/repro/scripts/demo-reset.sh --quick
```

```
DEMO_READY mode=quick health=PASS accounts=READY data=CLEAN
```

C6. Dừng hoặc xóa stack tái lập

```
$ bash experiments/repro/scripts/stop.sh --app-only
$ bash experiments/repro/scripts/stop.sh --down
# Chỉ khi cần xóa toàn bộ stack repro:
$ bash experiments/repro/scripts/stop.sh --purge
```

Khi một bước thất bại, không xóa checkpoint hoặc broadcast thủ công để "làm sạch" trạng thái. Với phát hành, kiểm tra issuance_batches, issuance_outbox, jobId và checkpoint của worker; với thu hồi, kiểm tra revocationIntentTxid, revocationRawTransaction và trạng thái revocation_pending/revocation_reconciliation_required. Chỉ retry sau khi xác định nguyên nhân và phải sử dụng lại định danh/giao dịch đã checkpoint.

Phụ lục 4. Bộ bằng chứng thực nghiệm chính thức

Bảng PL4.1 tổng hợp bộ tái kiểm chứng chính thức được lưu dưới experiments/results/final-revalidation-20260930/ tại commit công bố c56650d15f24a5464512c6c8deed509a04631577. Lượt chạy dùng cây mã nguồn sạch tại commit 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1, sourceDirty=false, runId 20260930T103044Z-3582ddaa và sourceTreeSha256 6e0cd862daf5641b26995575d8e7ff9d755daf43396191dfc838fc6ffb0997cc.

Checkout 2816de8 dùng để tái chạy mã; commit công bố dùng để đối chiếu bằng

chứng đã đóng băng. Không nhầm với thư mục revalidation-20260930/ của lượt trung gian có working tree bẩn.

Bảng PL4.1. Thành phần bộ bằng chứng và mục đích

Tệp/nhóm tệp	Nội dung kiểm chứng
source-manifest.json	Commit, trạng thái cây nguồn và fingerprint của tập tệp được kiểm soát
backend-jest.json	Kết quả 12/12 suite, 49/49 backend test
functional-e2e.json	Maker–Checker, duyệt từng phần, Holder, object isolation, verify và revoke
recovery.json	Phục hồi quanh broadcast, giữ nguyên txid và không tạo anchor thứ hai
benchmark.json	Dữ liệu thô của 9 lần chạy cho lô 10, 100 và 500
benchmark-summary.json	Giá trị tổng hợp thời gian, thông lượng và tài nguyên
benchmark-artifact-audit.json	Đối chiếu 1.830 chứng thư, bằng chứng Merkle, lô, giao dịch và manifest
manifests/*.json	Manifest theo từng lô/lần chạy, gồm ID, txid, gốc Merkle và tệp kết quả
SHA256SUMS	Checksum của bộ tệp công bố

Chỉ số chốt của lượt chạy: 9/9 lần chạy benchmark đạt; 1.830/1.830 chứng thư được phát hành; 9/9 lô tương ứng 9 giao dịch đã xác nhận; 1.830/1.830 Merkle proof được đối chiếu; bước xác minh đầy đủ kết luận 1.830/1.830 chứng thư VALID; manifest và SHA-256SUMS khớp với bộ tệp công bố. Các con số này áp dụng cho môi trường regtest và cấu hình máy thử nghiệm nêu trong Chương 3.

Phụ lục 5. Ma trận truy vết yêu cầu – hiện thực – bằng chứng

Bảng PL5.1 nối các yêu cầu quan trọng với thành phần mã và bằng chứng thực thi. Một yêu cầu chỉ được coi là đã kiểm chứng trong phạm vi đồ án khi có cả cơ chế hiện thực và kết quả thử nghiệm tương ứng; kết quả regtest không được suy rộng thành production/mainnet.

Bảng PL5.1. Ma trận truy vết yêu cầu và bằng chứng

Yêu cầu	Thành phần hiện thực	Bằng chứng chính
Maker không tự duyệt	JWT, RolesGuard, Checker-only approve	S1-06; functional E2E trả HTTP 403
Cô lập Maker theo đối tượng	Filter requestedBy ở list/detail/audit	Functional E2E: Maker B không thấy dữ liệu Maker A
Cô lập Student	studentId từ subject JWT	Functional E2E với hai Student
Duyệt từng phần	requestBatchId/ issuanceBatchId; migration 10	Hai lượt duyệt tạo hai issuanceBatchId khác nhau
Phát hành bền vững	issuance_batches, issuance_outbox, jobId ổn định	Backend test + functional E2E
Phục hồi sau broadcast	Điểm kiểm soát giao dịch thô/txid, đối soát trạng thái	Recovery E2E giữ nguyên txid
Phát hiện sửa chứng thư	Merkle proof + anchor verification	S1-02 đến S1-05 trả INVALID
Thu hồi đúng quyền	Checker-only revoke + checkpoint thu hồi	S3-02 đến S3-07; Verify trả REVOKED
Phân loại lỗi xác minh	VALID/INVALID/ REVOKED/ INDETERMINATE	16/16 verifier test
Chống SSRF	Safe-fetch, DNS/IP/redirect checks	10/10 regression test SSRF
Kiểm toán	Audit hash chain + integrity endpoint	Backend test; /api/audit/integrity
Tái lập và kiểm toán số liệu	Source manifest, manifest theo run, checksum	9/9 lần chạy; 1.830/1.830 xác minh đầy đủ; SHA256SUMS